FIRST EDITION · 2026

Python
A Complete Course

From the silicon up: a textbook for those who want to know not just how to write Python, but why Python is the way it is.

P

Volume I · Foundations & Core Language

With Numbered Examples, Exercises, and Complete Solutions

Preface

There are, by recent count, more than three hundred books on the Python programming language in print. This one differs from most of them in two respects.

The first is that it does not begin with Python. It begins, instead, with the machine. For three chapters you will read no Python at all. You will read about transistors, bits, instructions, processes, and the long history of how human beings learnt to make machines obey them. This is not preamble. It is the foundation, without which every later chapter would be a story told from the middle.

The second is that it does not assume you wish to be told. It assumes you wish to *understand*. Where another book might say "use a list", this book will tell you what a list is in memory, why appending to it is fast and prepending is not, when to choose a tuple instead, and what CPython's implementers did to make the choice matter as little as possible. The aim is fluency, not just fluency in writing Python but fluency in *thinking* about it.

How to Use This Book

Read each chapter in order. The examples are numbered (Example 4.3 is the third example in Chapter 4); the exercises follow each chapter; the complete solutions appear at the end of every chapter. Do the exercises before reading the solutions, even when you are certain of the answer. Programming is a motor skill; the muscle memory of typing `def` and `:` at the right moment is built only by typing them.

On Depth and Further Reading

Every topic in this book is the subject of its own books, papers, and lifelong careers. We cannot, and will not pretend to, cover everything. Where a subject deserves a book of its own, you will find a "Further Reading" box at the end of the section, pointing to the resources we judge to be the best on the internet. This book is a foundation; those resources are where you continue building.

Conventions

Cream code blocks are Python source. Grey blocks marked **Output** are what the interpreter prints. Yellow-bordered boxes contain Examples. Blue-bordered boxes contain Exercises. Indigo-bordered boxes contain links to Further Reading. Grey boxes contain Definitions of vocabulary you will use throughout the book.

Prerequisites

None, beyond curiosity and a willingness to read carefully. The mathematics will not exceed what you learnt in school. The first three chapters assume no programming background at all.

Contents

Volume I · Twenty-Two Chapters in Five Parts

PART I · THE GROUND

Foundations Beneath the Language

1. The Computer Beneath 1

2. The Lineage of Python 38

3. The Object Model 72

PART II · THE PRIMITIVES

Values and How They Behave

4. Numbers in Five Forms 108

5. Strings & Text 142

6. Booleans & Truthiness 186

7. None & the Sentinel 204

PART III · THE CONTAINERS

Structures for Collections

8. Lists in Depth 218

9. Tuples & Named Tuples 252

10. Dictionaries & Hash Tables 270

11. Sets & Frozensets 308

12. The collections Module 328

PART IV · FLOW & FUNCTION

The Shape of a Program

13. Control Flow 346

14. Iteration in Depth 368

15. Functions 394

16. Closures & Decorators 438

17. Comprehensions & Generators 462

PART V · LARGER PIECES

From One File to a Project

18. Modules, Packages, Environments 488

19. Classes & Objects 516

20. Files & the Outside World 562

21. Errors, Exceptions, Traces 590

22. A Workshop of Real Problems 618

CHAPTER 1 · PART I · THE GROUND

The Computer Beneath

Before you can program a machine, you should know what kind of machine it is. This chapter is a brief tour of the device, and of the five thousand years of effort that produced it, laying down only as much groundwork as the rest of the book will need.

> *A computer is a device which obeys instructions. We have built them so well that we now have to remember that this is all they do.*
>
> — Maurice Wilkes, designer of EDSAC (1949), the first practical stored-program computer

The book you are holding teaches a programming language called Python. But Python is, in the end, a thin layer of human convenience laid over a very specific kind of machine, and that machine has properties (speed, capacity, behaviour) which constrain everything you will ever do with it. If you want to be a competent programmer, you can begin with the language and learn what is beneath only when you need to. If you want to be a thoughtful one, you begin with what is beneath. We are taking the second route.

This chapter is not a course in computer architecture. A proper course in computer architecture is, by itself, a year of university study; we point at the best ones in the Further Reading boxes throughout. What you will find here is the minimum that the rest of this book assumes: a vocabulary for the parts of a computer, an intuition for their relative speeds, and a sense of the long human history that led to the device on your desk.

1.0 Before Electronics: Five Thousand Years of Calculating Machines

It is tempting, sitting in front of a glowing rectangle, to imagine that computing began with the silicon chip. It did not. The desire to delegate calculation to a machine is older than writing, and the lineage of devices that have answered that desire reaches back five thousand years. Knowing this lineage will not, by itself, make you a better Python programmer; but it will make you a less arrogant one, and that is its own kind of skill.

The Abacus, circa 2700 BCE

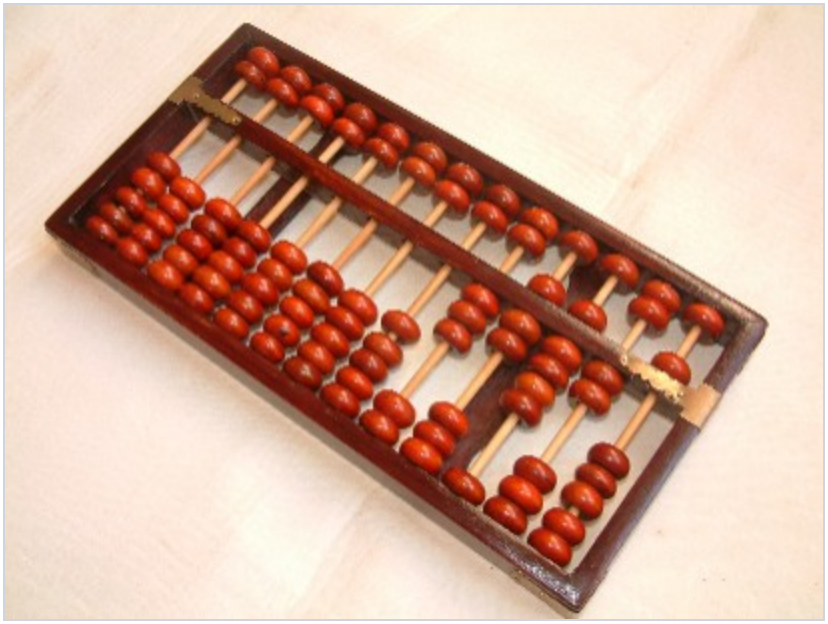


Figure 1.1. *A traditional Chinese suanpan abacus. The same instrument, in essence, was used for accounting in Mesopotamia, Egypt, Greece, Rome, China, Japan, and across the Islamic world.* en.wikipedia.org/wiki/Abacus

The abacus is the oldest surviving computing device. By moving beads along rods, a trained operator could perform addition, subtraction, multiplication, division, and even, with practice, square roots, at speeds that competed with early electronic calculators well into the 1940s. The abacus encodes numbers positionally, which is the same principle on which every modern digital computer rests. It is also, in a small but real sense, a stored-program device: the position of the beads represents both the operand and the intermediate state of the calculation.

The Antikythera Mechanism, circa 100 BCE



Figure 1.2. *The largest surviving fragment of the Antikythera mechanism, recovered in 1901 from a Roman-era shipwreck off the Greek island of Antikythera. Built around 100 BCE, it modelled the motions of the sun, moon, and planets with thirty-odd bronze gears.* en.wikipedia.org/wiki/Antikythera_mechanism

For nineteen centuries the Antikythera mechanism sat in a chest of bronze fragments at the bottom of the Aegean. When it was finally CT-scanned in the 2000s, the world discovered that ancient Greece had possessed an analog computer: a hand-cranked geared device that predicted eclipses, tracked the Olympic Games on a four-year cycle, and modelled the apparent motion of five planets. There is nothing else like it from antiquity. Whatever industrial culture produced it left no other example, and the line of mechanical computing fell silent for a thousand years.

Pascal, Leibniz, and the Mechanical Calculators

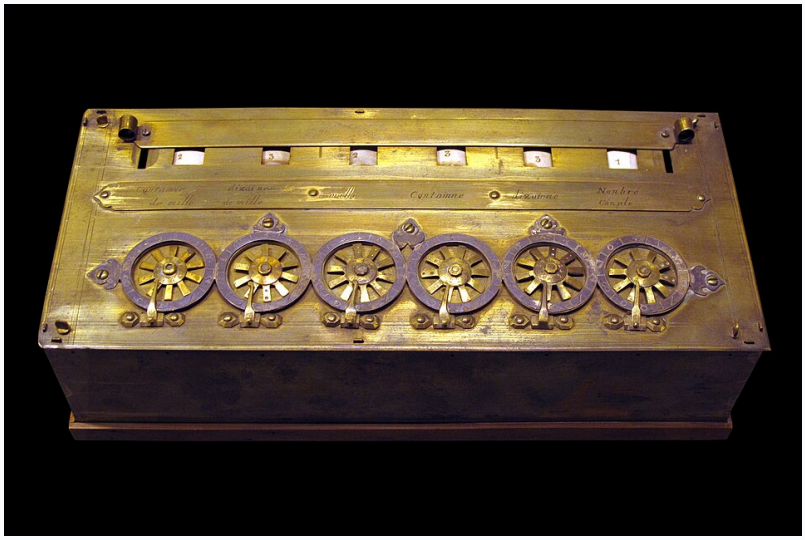


Figure 1.3. *The Pascaline, built by Blaise Pascal in 1642.* Pascal's calculator

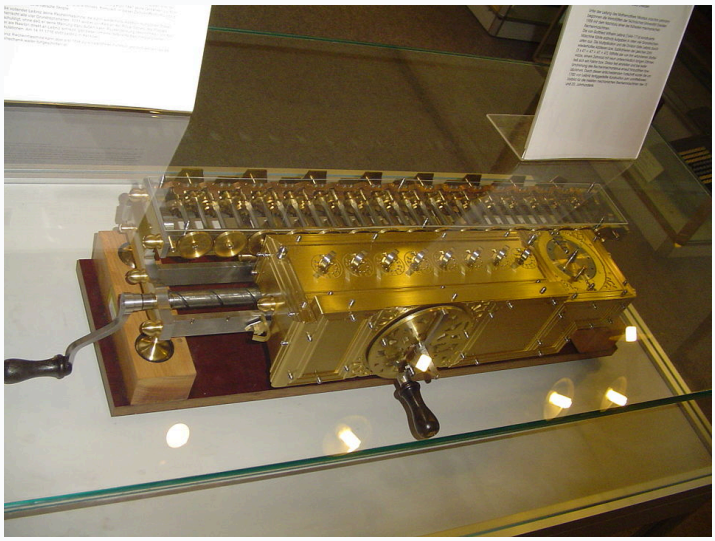


Figure 1.4. *Leibniz's Stepped Reckoner, 1673.* Stepped reckoner

In 1642, the nineteen-year-old Blaise Pascal built a mechanical calculator to help his father, a tax commissioner in Rouen, perform the arithmetic of public accounting. Three decades later, Gottfried Wilhelm Leibniz extended the idea with the *stepped reckoner*, which could multiply and divide. Leibniz also wrote, in 1703, the paper *Explication de l'Arithmétique Binaire*, in which he set out the binary number system, three hundred years before any device would exploit it.

Babbage, Lovelace, and the First Programs

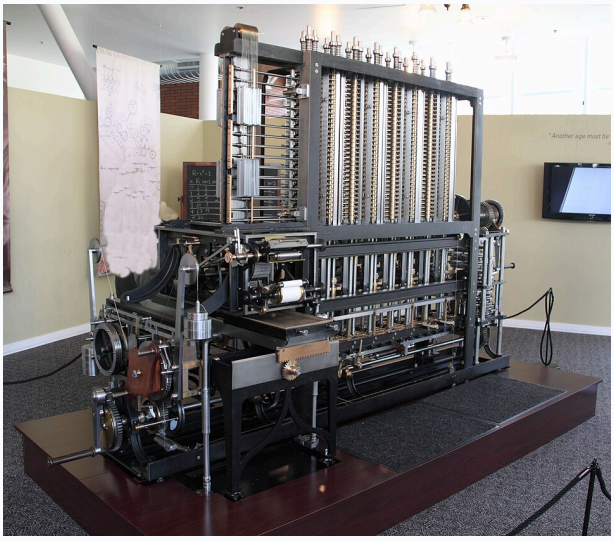


Figure 1.5. *Charles Babbage's Difference Engine No. 2, completed (posthumously) at the London Science Museum in 1991.* Difference engine



Figure 1.6. *Ada Lovelace, painted by Margaret Sarah Carpenter, 1836.* Ada Lovelace

In the 1830s, the English mathematician Charles Babbage designed a machine he called the *Analytical Engine*, a programmable, general-purpose mechanical computer driven by punched cards. He never finished it; the engineering tolerances of Victorian England were not equal to the task, and his money ran out. But the design was complete, and one of his collaborators, Ada Lovelace, the daughter of the poet Lord Byron, wrote a sequence of operations for the Engine to compute Bernoulli numbers. It was, by any reasonable definition, the first computer program, written for a machine that did not yet exist and would not for another century.

Lovelace also saw, more clearly than Babbage himself, what the device implied. Machines, she wrote, could in principle manipulate symbols, not just numbers, and might

one day compose music or assist in any field "in which the relations of things susceptible of being expressed by abstract signs are involved." Every program you will ever write rests on the truth of that paragraph.

→ **FURTHER READING: BEFORE ELECTRONICS**

- History of computing hardware *Wikipedia, comprehensive overview*
- The Computer History Museum timeline *computerhistory.org*
- Ada Lovelace's notes on the Analytical Engine *Finding Ada*
- Ada Lovelace: The First Computer Programmer *BBC podcast*

1.1 The Universal Machine

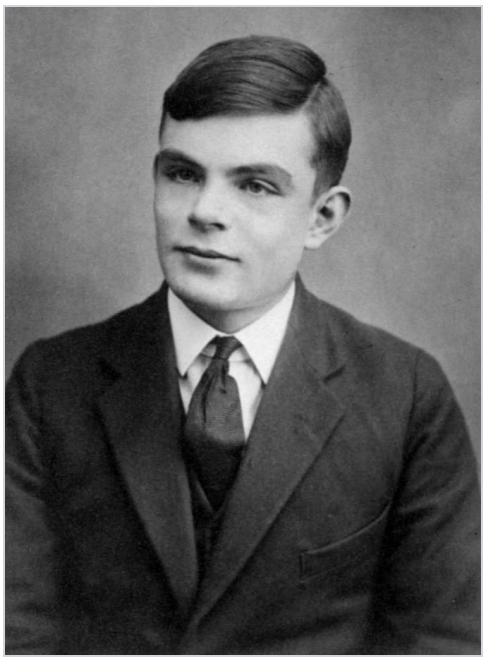


Figure 1.7. *Alan Turing, aged 16, in 1928.*

In 1936, a twenty-four-year-old English mathematician named Alan Turing published a paper with the unpromising title *On Computable Numbers, with an Application to the Entscheidungsproblem*. It is among the most consequential documents ever written. In it, Turing described a device that did not exist (a mathematical fiction, a thought experiment) that could, given the right instructions, simulate any other computing device imaginable. He called it an "a-machine"; we now call it a **Turing machine**.

The device was preposterously simple. It consisted of an infinite tape divided into cells, a head that could read or write one symbol at a time, and a small table of rules of the form: *if you see this symbol in this state, write that symbol, move left or right, and change state*. From this minimum, Turing proved, you could compute anything that was computable at all.

The point is not the tape. The point is what Turing's argument established: that **there exists a single kind of machine which, by being fed different instructions, can perform the work of any other kind of machine**. This is the idea of the *universal computer*, and every device you own (phone, laptop, server, watch) is one of them. They differ only in speed, capacity, and what is plugged in.

HISTORY The word *computer* originally referred to a person, usually a woman, whose job was to perform calculations by hand. NASA's "human computers", including Katherine Johnson, Dorothy Vaughan, and Mary Jackson, continued to work into the 1970s. The machine took the name from the role it replaced.

→ **FURTHER READING: TURING & COMPUTABILITY**

- Turing's original 1936 paper (PDF) *University of Virginia*
- Turing Machines *Stanford Encyclopedia of Philosophy*
- Computerphile: Turing Machines Explained *YouTube*
- Andrew Hodges, *Alan Turing: The Enigma* *the definitive biography*

1.2 The von Neumann Architecture

In June 1945, the Hungarian-American polymath John von Neumann circulated a draft document, the *First Draft of a Report on the EDVAC*, describing the architecture of a computer that would store its program in the same memory as its data. This was a departure from the practice of the time, in which programs were entered by physically rewiring the machine. Once a program lived in memory, it could be modified, copied, transmitted, and executed without altering a single wire. The idea spread quickly, and within five years almost every computer being built used the design. We still call it the **von Neumann architecture**.



Figure 1.8a. *John von Neumann (1903–1957).*

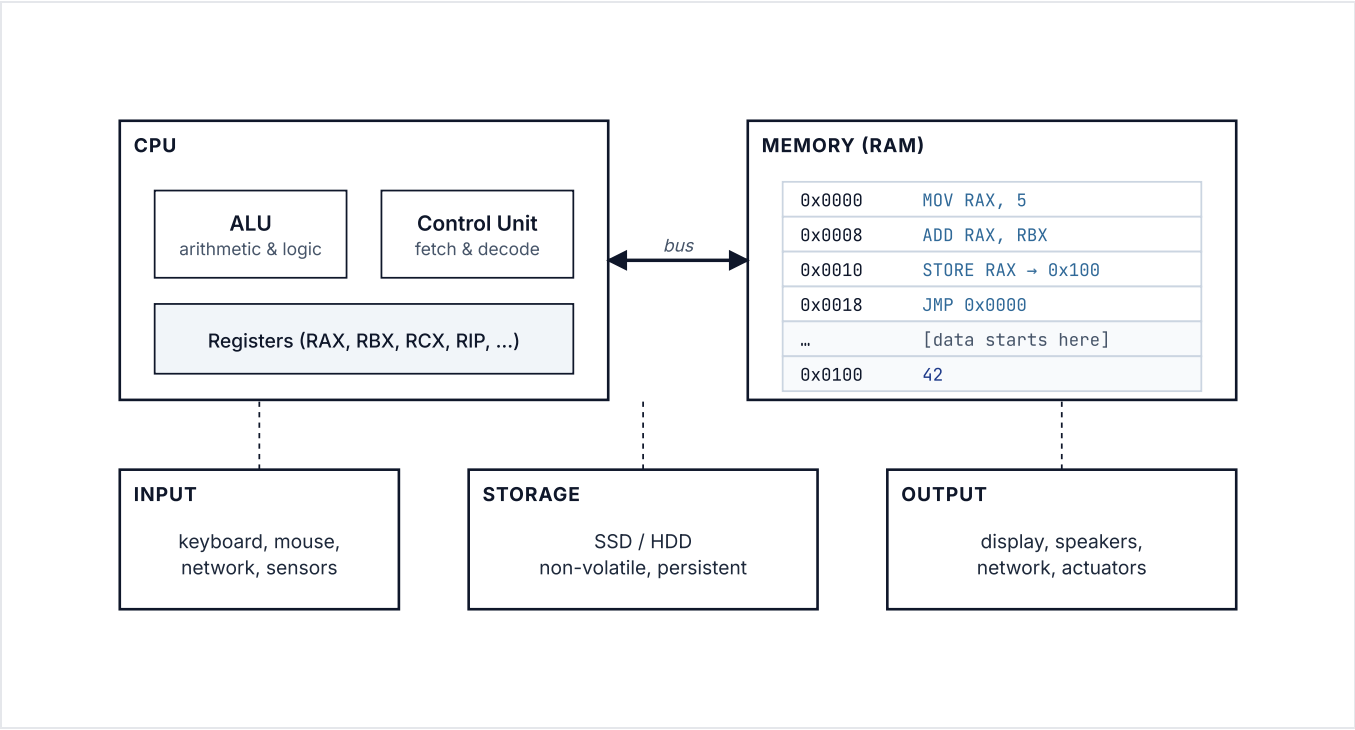


Figure 1.8. The von Neumann architecture. The CPU contains the arithmetic and control units and a small number of registers. Memory holds both instructions and data in the same address space. A bus carries values between the two. Input, storage, and output devices attach through controllers.

The architecture has four functional parts. The **CPU** is where computation happens. It contains an *arithmetic logic unit* for arithmetic and bitwise operations, a *control unit* for fetching and decoding instructions, and a small set of very fast working slots called *registers*. The **memory** (RAM, for random-access memory) holds programs and data; it is much larger than the registers but much slower. **Storage** (solid-state and rotating disks) holds data permanently across power cycles. **I/O devices** are the senses and limbs by which the machine reaches the world.

The CPU runs in a continuous loop, called the *fetch-decode-execute cycle*. It reads the instruction at the address held in a special register called the *instruction pointer*, decodes what it asks for, performs it, advances the pointer, and repeats. A modern processor performs this loop several billion times per second. The instruction at one address might be "add register A to register B"; the next might be "store the result at memory location 0x100"; the next might be "if register A is zero, jump to address 0x500". Out of these primitive moves, executed in sequence, every program runs.

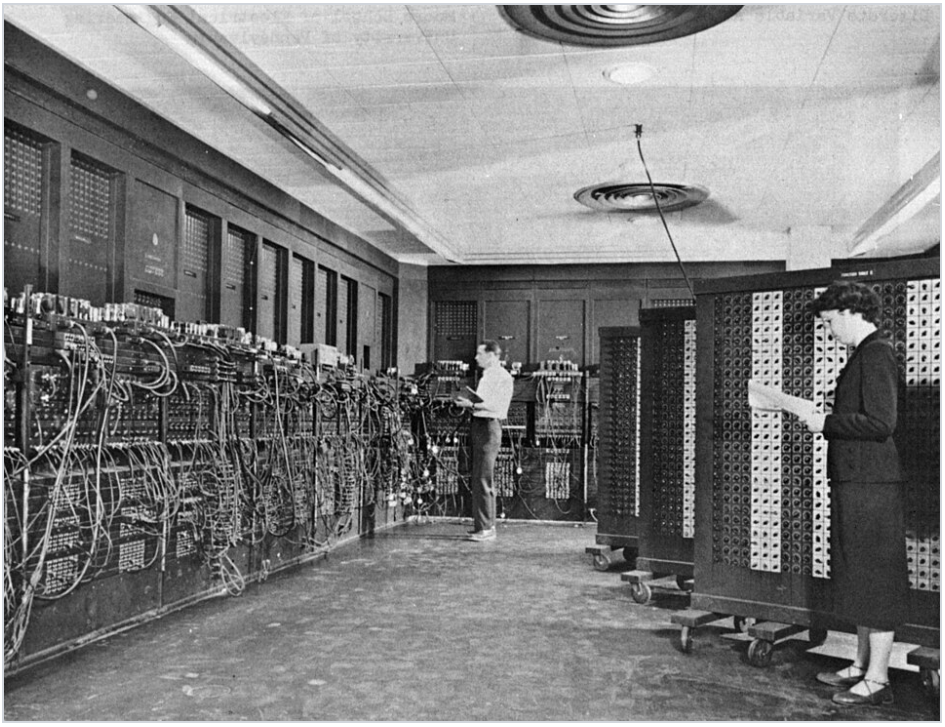


Figure 1.9. *ENIAC at the U.S. Army's Ballistic Research Laboratory, the first general-purpose electronic computer. 30 tonnes, 17,468 vacuum tubes, programmed by physically setting switches and re-routing cables. Shown are technicians Glen Beck and Betty Snyder; Snyder (later Betty Holberton) was one of ENIAC's original six women programmers.* ENIAC

DEF

Clock Cycle

The basic unit of time inside a CPU. A 3 GHz processor has three billion cycles per second; one cycle takes about 0.33 nanoseconds. A simple instruction may complete in one cycle; a complex one (a multiplication, a memory load) may take several. The number of cycles per instruction is called the CPI, and modern CPUs use pipelining and out-of-order execution to bring it below 1 in favourable code.

EXAMPLE 1.1 A single instruction, in three notations

Consider the operation: *add the integer in register RBX to the integer in register RAX*. Here it is, in three forms, descending toward what the silicon actually sees:

ASM

```
; assembly (what humans wrote in 1955)
ADD RAX, RBX
```

MACHINE

```
# machine code (what the CPU reads)
48 01 D8
```

BINARY

```
# the bits, in binary (what is on the wire)
01001000 00000001 11011000
```

The three lines describe the same operation. Every program in your computer's memory is, ultimately, a sequence of bytes like the second line. Assembly is a one-to-one human-readable transcription of it. Python is many, many layers above.

→ **FURTHER READING: COMPUTER ARCHITECTURE**

- *Computer Systems: A Programmer's Perspective* (Bryant & O'Hallaron) *the standard university text*
- Nand to Tetris *build a computer from logic gates, free course*
- Crash Course Computer Science *YouTube, 40 episodes*
- Von Neumann architecture *Wikipedia*
- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software* *superb introduction from first principles*

1.3 The Memory Hierarchy

Memory in a modern computer is not one thing. It is a hierarchy of several layers, each faster and smaller than the next. Understanding this hierarchy, and the orders of magnitude that separate its levels, is the difference between writing programs that are fast and programs that pretend to be.

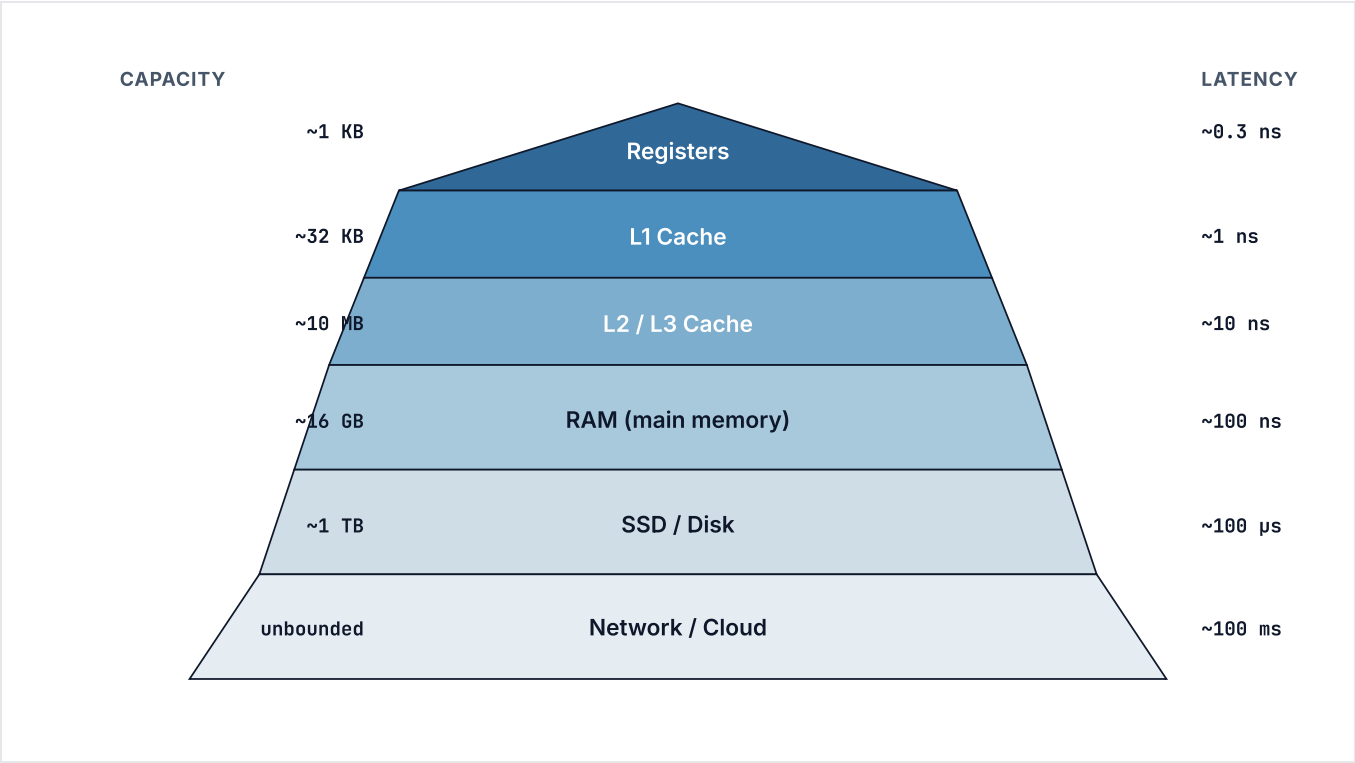


Figure 1.10. The memory hierarchy of a contemporary laptop. Each level is roughly an order of magnitude slower than the one above it. The numbers are approximate, but the ratios are not.

To grasp these numbers, multiply each by ten billion to scale the timings into human ones. A register access becomes about 3 seconds. An L1 cache access becomes 10 seconds. An L2 / L3 access becomes a minute and a half. A RAM access becomes about 15 minutes. An SSD read becomes a little over a day. A network round-trip becomes about a month. *This is the world your program lives in.* When people say a Python program is "slow", they almost always mean it is spending too much time at the lower levels of this pyramid, waiting for memory, for disk, for the network, and not enough time near the top.

PERFORMANCE INTUITION If a program runs a loop of arithmetic over values already in cache, it can manage billions of operations per second. If the same loop has to fetch each value from RAM, the rate drops by a factor of 100. If it has to read each value from SSD, the rate drops by another factor of 1,000. The size of these gaps is why so much modern software design is, in the end, about *locality*: keeping the data near the work.

→ FURTHER READING: MEMORY & CACHING

- Ulrich Drepper, "What Every Programmer Should Know About Memory" (PDF) *the definitive technical guide, 2007*
- Latency Numbers Every Programmer Should Know *Jeff Dean's famous list*
- Latency Numbers: Interactive Visualisation *how the numbers have changed over decades*

1.4 Numbers in Binary

The CPU only knows numbers, and even those it represents in binary, a positional notation with base 2 instead of base 10. Each digit in a binary number is called a *bit* and takes the value 0 or 1. Eight bits form a *byte*, which can represent $2^8 = 256$ distinct values.

The shift from decimal to binary is mostly a mechanical translation. The decimal number 13, written 1101 in binary, decomposes as:

WORKING

$$
\begin{aligned}
&1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\
=\ &8 + 4 + 0 + 1 \\
=\ &13
\end{aligned}
$$

Integers: Two's Complement

Storing negative numbers is the first complication. Almost every modern CPU uses a scheme called *two's complement*. In an 8-bit two's-complement representation, the leftmost bit signals sign (0 for positive, 1 for negative), but the negative values are not simply the positives with the sign flipped. Instead, to negate a number, you invert all the bits and add 1.

Decimal	8-bit Binary
0	`00000000`
1	`00000001`
127	`01111111`
-1	`11111111`

-2	`11111110`
-128	`10000000`

Why this strange-looking scheme? Because under two's complement, the CPU can use a single circuit, the addition circuit, to perform both addition and subtraction. Computing `5 - 3` is mechanically identical to computing `5 + (-3)`. The cost of this elegance is a small asymmetry: an 8-bit two's-complement integer ranges from -128 to +127, not -127 to +127. One negative number has no positive partner.

EXAMPLE 1.2 Integer overflow, in C and in Python

Most languages treat an integer as a fixed-size container. C's `int` is typically 32 bits, and what happens when you exceed its range is among the most quietly devastating bugs in programming history:

C

```c
// C
int a = 2147483647;     // 2^31 - 1, the maximum 32-bit signed int
int b = a + 1;
printf("%d", b);           // prints -2147483648  (wraps around!)
```

The 1996 explosion of the Ariane 5 rocket, which destroyed €500 million of European satellites, was caused by exactly this kind of overflow: a 64-bit floating-point value was converted into a 16-bit signed integer that could not hold it. Python's design rejects this trap entirely. Its integers grow as large as needed:

PYTHON

```python
# Python
a = 2 ** 1000
print(len(str(a)))          # 302: the number has 302 digits
print(a + 1 != a)           # True: no overflow, ever
```

OUTPUT

```
302
True
```

This is a trade. C's fixed-size integers are dramatically faster to manipulate; Python's are dramatically safer. You will, in your career, have reason to prefer each.

Real Numbers: IEEE 754

Storing non-integer numbers (3.14, 0.001, 6.022×10^{23}) requires a scheme more elaborate than counting in binary. The standard used by essentially every CPU built since the late 1980s is **IEEE 754**. It represents a real number as a triple: a sign bit, an exponent, and a fraction (also called the mantissa or significand).

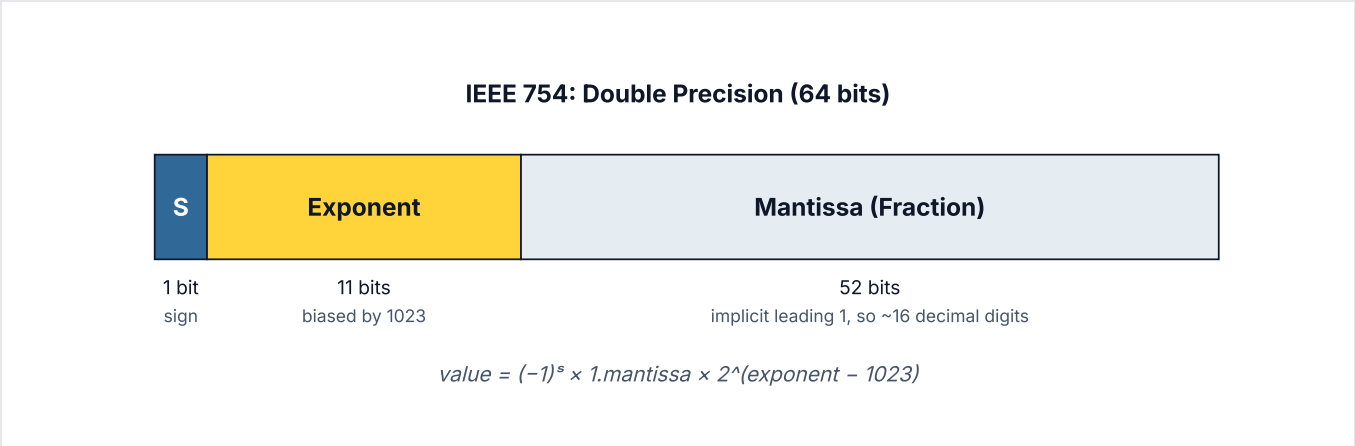


Figure 1.11. The 64-bit float layout. A 32-bit float is the same scheme with smaller fields (1 + 8 + 23). Python's `float` is always 64-bit.

The consequence of this representation is a fact every programmer must internalise: **floating-point numbers are approximations**. The decimal 0.1 cannot be expressed exactly in binary, much as 1/3 cannot be expressed exactly in decimal. The closest 64-bit IEEE 754 approximation of 0.1 is, in fact:

EXACT VALUE

```
0.1000000000000000055511151231257827021181583404541015625
```

This tiny error, harmless in isolation, accumulates over millions of operations and reveals itself in unexpected places. We will return to the consequences in Chapter 4. For now, internalise the diagram and the principle.

→ **FURTHER READING: BINARY & IEEE 754**

- The Floating-Point Guide *floating-point-gui.de, an excellent primer*
- Python docs: Floating Point Arithmetic, Issues and Limitations *official, concise*
- David Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic" *the canonical deep dive*
- Two's complement *Wikipedia, with worked examples*

1.5 Machine Code, Assembly, and the First Bug

A program in memory is a sequence of bytes interpreted as instructions. Each instruction tells the CPU to do one of a few hundred primitive operations: load a value, store a value, add, subtract, compare, jump. The encoding is determined by the CPU's *instruction set architecture*: x86-64 for most laptops and desktops, ARM64 for phones and recent Macs, RISC-V for an increasing slice of embedded and research hardware.

Direct programming in machine code was abandoned by the late 1950s. Programmers wrote in *assembly*, a near-perfect one-to-one human-readable transcription of machine code, until the high-level languages of the 1960s displaced it for most work. Today, assembly remains in use only for the innermost loops of operating-system kernels, drivers, and certain performance-critical libraries.

EXAMPLE 1.3 A C function and its assembly

This is the C function `add`:

```c
int add(int a, int b) {
    return a + b;
}
```

The x86-64 assembly it compiles to under optimisation looks like this:

```asm
add:
    lea     eax, [rdi + rsi]   ; eax = rdi + rsi
    ret                        ; return; result is in eax
```

Two instructions. The C function's arguments arrive in registers `rdi` and `rsi` (per the System V calling convention), the sum is placed in `eax`, and `ret` hands control back to the caller. The equivalent Python function, also a one-liner, runs through perhaps two hundred machine instructions on the interpreter side before the addition itself is performed.

The First Bug

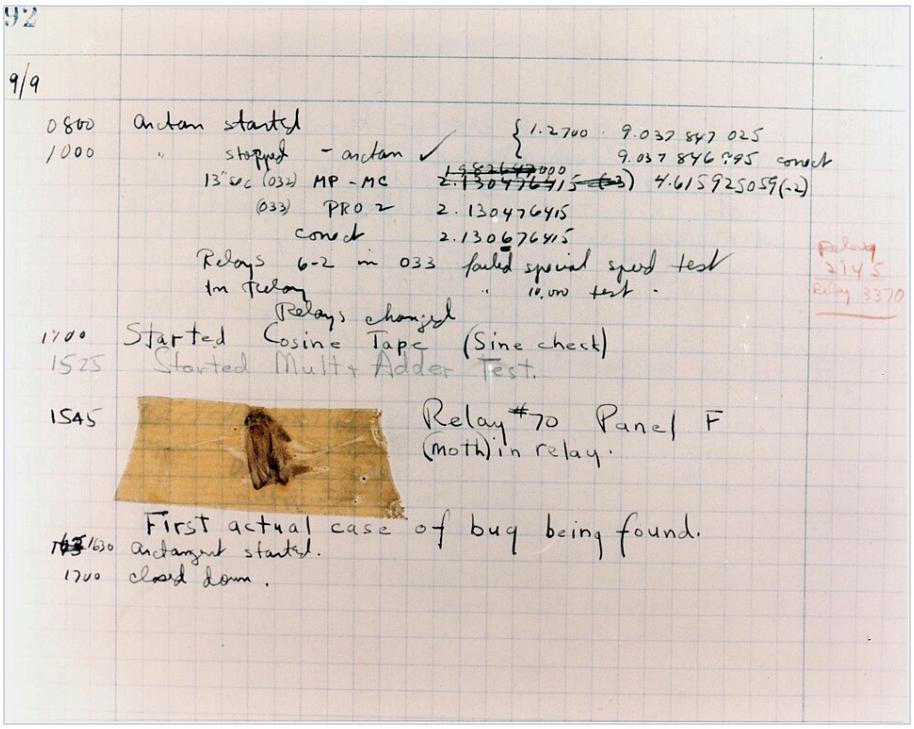


Figure 1.12. *The log entry from the Harvard Mark II computer team, 9 September 1947: "First actual case of bug being found." A moth had become trapped in a relay; Admiral Grace Hopper's team taped it into the logbook. The word "bug" had been used informally by engineers for decades, but this entry made it permanent.* Etymology of "bug"

The Call Stack

When a function calls another function, the CPU must remember where it was so it can come back. The mechanism is the *call stack*: a region of memory that grows downward as functions are entered and shrinks upward as they return. Each function's local variables, parameters, and return address live in a region of the stack called its *stack frame*.

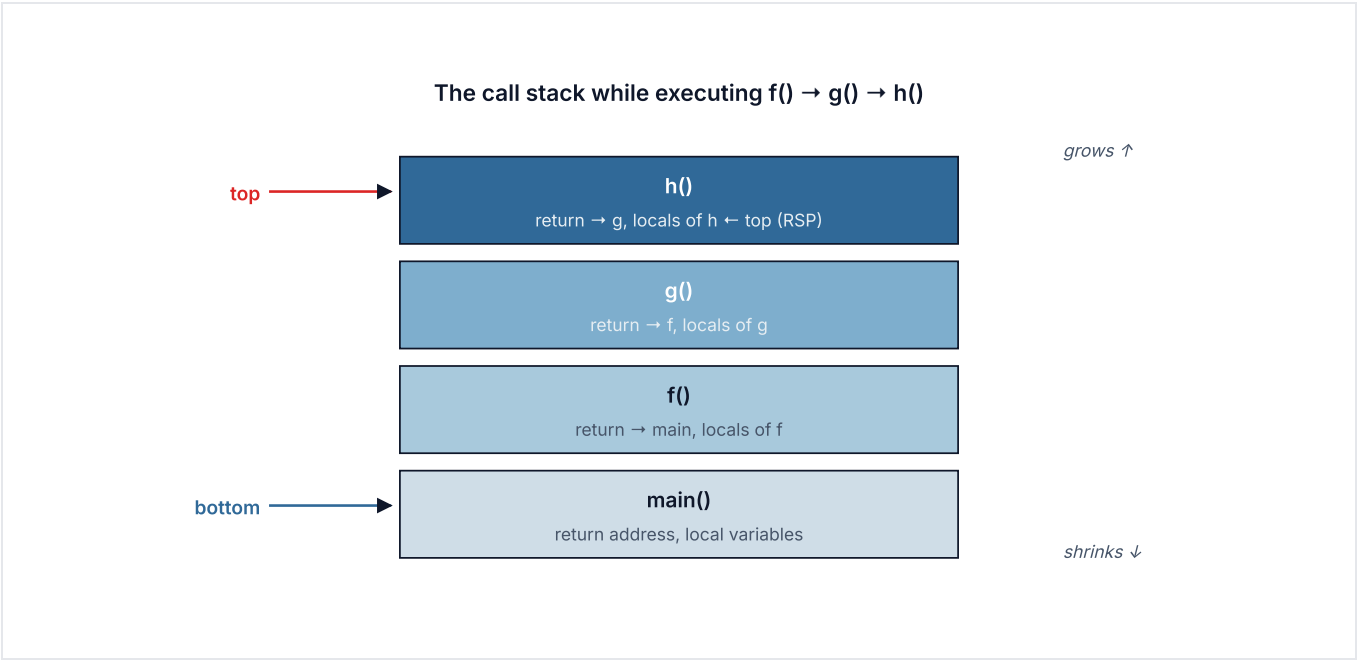


Figure 1.13. The call stack. Each function call pushes a new frame onto the top of the stack; each return pops one off. The stack is finite (typically a few megabytes), and a program that recurses without bound will eventually exhaust it (a stack overflow).

Python, too, has a call stack. When an exception is raised, the *traceback* it prints is, literally, a snapshot of this stack at the moment of the error. We will see this again in Chapter 21.

→ **FURTHER READING: ASSEMBLY & THE STACK**

- Compiler Explorer (godbolt.org) *type C, see the assembly; invaluable*
- Harvard CS61: Systems Programming and Machine Organization *free online course materials*
- Jonathan Bartlett, *Programming from the Ground Up* *assembly for beginners*

1.6 Processes, Threads, and the Operating System

Real computers do not run one program at a time. They run dozens, sometimes hundreds: your editor, your web browser, your music player, the daemons that keep your network alive, the system that updates your clock. The illusion that each program owns the machine is sustained by a piece of software called the *operating system*.

An operating system (Windows, macOS, Linux, iOS, Android) does three principal things. It *multiplexes* the CPU, slicing time into small intervals, running a different program in each, switching between them faster than any human perception can detect. It *isolates* programs from one another, giving each its own private view of memory, called a *virtual address space*, so that one program's bug cannot corrupt another's data. And it *mediates* access to hardware: your program does not write directly to the screen or read from the network card; it asks the operating system, which checks permissions and routes the request.

DEF **Process**

A running instance of a program, as seen by the operating system. A process has its own memory space, its own file descriptors, its own permissions, and at least one thread of execution. When you launch Python, the OS creates a Python process. When you launch a second Python interpreter, you get a second, completely independent process.

DEF **Thread**

A path of execution within a process. A single process may have many threads, sharing the same memory but each with its own call stack and its own current instruction pointer. Threads are how a program does several things in parallel. They are also a notorious source of bugs, because shared memory plus parallel execution is a difficult combination to reason about.

Python has a famously specific relationship with threads: the *Global Interpreter Lock* (GIL) ensures that only one thread inside a CPython interpreter executes Python bytecode at a time, even on a multi-core machine. This trade, simplicity of the interpreter's internals at the cost of CPU-parallelism, has been a subject of debate for thirty years. We will look at it properly in Chapter 21. For now, just know it is there.

→ **FURTHER READING: OPERATING SYSTEMS**

- *Operating Systems: Three Easy Pieces* *free book, the modern standard*
- MIT 6.828 lectures *YouTube, advanced*
- Operating system *Wikipedia overview*
- What Is the Python Global Interpreter Lock? *Real Python*

1.7 What Python Inherits From All This

The Python you write does not, in any literal sense, run on the machine described above. It runs on a virtual machine (a piece of C code called the CPython interpreter) which itself runs on the machine described above. But the constraints of the underlying hardware leak through. When you create a list of a million integers, those million PyObject pointers occupy real bytes in real memory. When you call a function, a real stack frame is pushed onto a real stack. When your program is slow, it is slow for reasons rooted in the hierarchy of Figure 1.10.

A programmer who knows none of this can still write Python. They will, however, be regularly puzzled. Why is appending to a list fast but inserting at the front slow? Why is dictionary lookup constant-time but list `in` linear? Why does `multiprocessing` exist when `threading` already does? Each answer involves the hardware. We will encounter all

three before this book is finished, and you will understand each immediately, because you read this chapter.

The remaining chapters of Part I cover the second half of the foundation: where Python itself came from, what CPython actually is, and how a Python statement becomes electricity. After that the language itself begins.

§ § §

Exercises

EXERCISE 1.1 Bit counting

How many distinct values can be represented by 1 bit? By 4 bits? By 16 bits? By 64 bits? Express each as a power of 2 and approximate it in decimal. Why does a 64-bit integer suffice for almost every practical counting problem?

EXERCISE 1.2 Two's complement by hand

Compute the 8-bit two's-complement representation of −37. Show your working: write +37 in binary, invert the bits, then add 1. Verify by adding your result to +37 in binary and observing that the answer is zero (with a discarded carry).

EXERCISE 1.3 Memory hierarchy scaling

If a register access takes 0.3 nanoseconds and a network round-trip takes 100 milliseconds, what is the ratio between them? If you scaled "register = 1 second", what would a network round-trip be, in human time?

EXERCISE 1.4 Why 0.1 is not 0.1

Explain in your own words, with reference to Figure 1.11, why a 64-bit float cannot represent the decimal number 0.1 exactly. What other simple decimal numbers *can* it represent exactly? (Hint: which fractions have terminating expansions in base 2?)

EXERCISE 1.5 Stack overflow

If a function call pushes a stack frame of 200 bytes onto a 1-megabyte call stack, roughly how deep can the recursion go before the stack overflows? Compare your answer to Python's default recursion limit (1000). Why might Python's limit be lower than the hardware allows?

EXERCISE 1.6 An instruction budget

A modern CPU executes around 3×10^9 instructions per second per core. A typical web page takes around 100 milliseconds to render. Estimate, very roughly, how many CPU instructions are spent rendering one web page. Does this match your intuition about how complex a web page is?

EXERCISE 1.7 Processes vs threads

Describe, in plain language, the difference between a process and a thread. Give one situation in which you would prefer threads (many tasks sharing data), and one in which you would prefer processes (many tasks that must not interfere with each other).

EXERCISE 1.8 A research question

The Ariane 5 rocket explosion of 4 June 1996 was caused by an integer overflow. Spend twenty minutes researching what happened. Write a one-paragraph technical summary in your own words. The point of this exercise is not the rocket; the point is that even very expensive teams sometimes write code without thinking about the hardware constraints we have discussed.

Solutions to Chapter 1 Exercises

Solution 1.1: Bit counting

1 bit gives $2^1 = 2$ values (0 or 1). 4 bits give $2^4 = 16$. 16 bits give $2^{16} = 65{,}536$. 64 bits give $2^{64} \approx 1.8 \times 10^{19}$, about 18 quintillion. A 64-bit integer can count every grain of sand on Earth and have room to spare; it can count the seconds since the universe began with about 22 orders of magnitude unused. The only domains that need more are mathematics (cryptography, combinatorics) and certain scientific simulations.

Solution 1.2: Two's complement of -37

WORKING

```
+37  =  00100101
invert: 11011010
add 1:  11011011

verify: 00100101    (+37)
      + 11011011    (-37)
      = 00000000    (carry discarded)
```

So −37 in 8-bit two's complement is `11011011` (decimal 219 if interpreted as unsigned).

Solution 1.3: Memory hierarchy scaling

The ratio is $0.1 / (0.3 \times 10^{-9}) \approx 3.3 \times 10^8$, about a third of a billion times slower. Scaled so a register access is 1 second, a network round-trip is roughly 333 million seconds, which is about 10.5 years. This is why caching matters so much: the cost of going to the network for data you already have nearby is, on this scale, geological.

Solution 1.4: Why 0.1 is not 0.1

A binary fraction terminates only if the denominator (when the fraction is in lowest terms) is a power of 2. The fraction 1/10 = 1/(2 × 5) has a factor of 5 in its denominator, so it does not terminate in binary; it produces a repeating expansion 0.000110011001100... that must be truncated to fit in 52 mantissa bits. Decimals that *do* terminate include 0.5 (= 1/2), 0.25 (= 1/4), 0.125 (= 1/8), and any sum of these. So 0.5 + 0.25 = 0.75 is exact in a float; 0.1 + 0.2 = 0.3 is not.

Solution 1.5: Stack overflow

1 MB / 200 B = 5,242 frames at most. Python's default limit of 1000 is set well below the hardware ceiling, deliberately, to give a clearer error message before the C stack itself overflows, which would crash the interpreter outright. The limit can be raised with

`sys.setrecursionlimit()`, but doing so without an architectural reason is usually a code smell. Most deep recursions are better rewritten as loops.

Solution 1.6: An instruction budget

$3 \times 10^9 \times 0.1 = 3 \times 10^8$ instructions per page render, three hundred million. This is for one core; modern machines have several. The number sounds enormous and is. The amount of work a modern browser does to lay out a page (parsing HTML, resolving CSS, computing layout, painting pixels, running JavaScript) comfortably fills it.

Solution 1.7: Processes vs threads

A *process* is a running program with its own private memory space; an operating system isolates each from the others. A *thread* is a path of execution within a single process; multiple threads in one process share memory directly. Threads are preferable when tasks need to communicate constantly (a web server handing requests off to worker threads that share a connection pool). Processes are preferable when tasks must not be able to corrupt each other (a browser's tab-per-process model, where a crash in one tab does not bring down the others).

Solution 1.8: Ariane 5

The Ariane 5 inherited inertial reference software from Ariane 4. In a routine pre-flight conversion, a 64-bit floating-point measurement of horizontal velocity was cast into a 16-bit signed integer. Ariane 5 was a faster rocket; the velocity exceeded 32,767 (the maximum value the 16-bit integer could hold); the conversion overflowed and produced a value the guidance system interpreted as garbage. The rocket veered, broke up under aerodynamic stress, and self-destructed 37 seconds after launch. The original Ariane 4 conversion had been deemed safe based on physical limits the older rocket could not exceed; nobody re-verified that assumption when the software was carried over. The fault was not a bug in the conversion code. The fault was the absence of a check on the assumption beneath it.

CHAPTER 2 · PART I · THE GROUND

The Lineage of Python

No language is invented from nothing. Python is the considered descendant of half a dozen earlier languages, the personal project of one programmer, and the collective work of thousands since. This chapter traces where it came from and explains what actually happens inside the program that runs your code.

> *I decided to try to design a language that I myself wanted to use. I had no illusions that this would change the world, but I knew I wanted to scratch my own itch.*
>
> — Guido van Rossum, creator of Python

In December 1989, a thirty-three-year-old Dutch programmer named Guido van Rossum found himself with a week of free time over the Christmas holiday and a problem he had been turning over for years. His office at the Centrum Wiskunde & Informatica in Amsterdam was closed. He had a home computer and an idea: a scripting language that would have the power of the systems languages he used at work, but the gentleness of a teaching language he had helped build in the 1980s. He started writing an interpreter. He named the project after a British comedy troupe whose scripts he had been reading, *Monty Python's Flying Circus*. The snake came later, and only as a logo.

The language that grew out of that holiday is now, by most measures, the most widely taught and most widely written programming language in the world. To understand why it has the shape it does, you have to understand the languages it descended from, the decisions its creator made deliberately, and the machinery that the standard implementation, CPython, uses to bring your code to life. That is the work of this chapter.

2.1 A Family of Languages

By the time Python was conceived, programming languages had been evolving for more than thirty years, and a recognisable family tree had formed. Each new language was, in large part, a reaction to the frustrations of the ones before it. Knowing the broad shape of that tree explains a surprising amount about why Python looks the way it does.

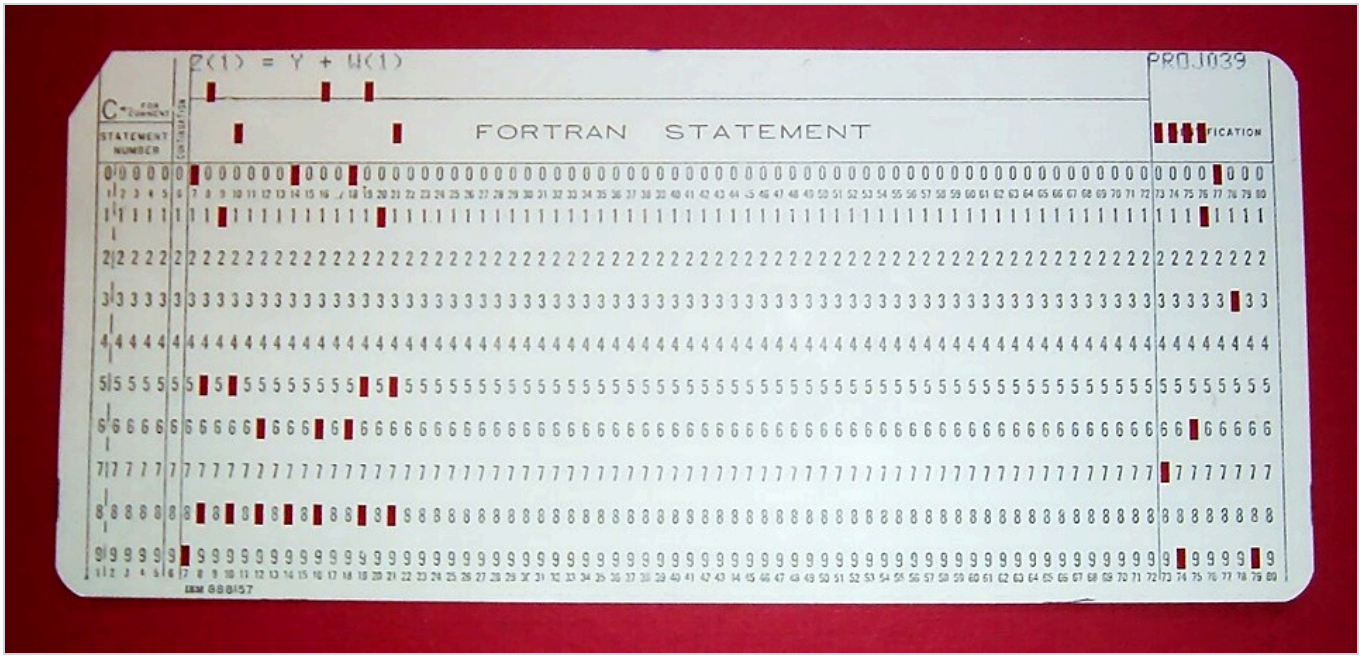


Figure 2.1. *A FORTRAN punched card. Into the 1970s, a program was a stack of these cards, one statement per card, fed into a reader. A dropped deck meant a re-sorted afternoon.* The punched-card era

The first high-level language to see wide use was **FORTRAN** (1957), built by a team under John Backus at IBM. Its great contribution was the *compiler*: a program whose job was to translate something resembling mathematical notation into machine code automatically, so that programmers no longer had to write the machine code themselves. Many doubted a compiler could ever produce code efficient enough to be worth using. Backus proved them wrong, and in doing so invented an entire category of software.

The languages that followed each added an idea. **LISP** (1958) introduced the notion that code and data could share the same form, and brought recursion and automatic memory management into the mainstream. **ALGOL** (1960) gave us block structure and the lexical scope rules that nearly every later language, including Python, would adopt. **Simula** (1967) invented the *object*: a bundle of data together with the procedures that act on it. **C** (1972) struck a balance between closeness to the hardware and human readability so successful that it became, and remains, the language in which operating systems are

written. **Smalltalk** (1972) took Simula's object idea to its logical conclusion, making everything in the language an object, an idea Python would later embrace wholeheartedly.

→ FURTHER READING: THE FAMILY OF LANGUAGES

- History of programming languages *Wikipedia overview*
- The History of Programming Languages *talk by various, computer history*
- HOPL: History of Programming Languages conference papers *the primary academic record*

2.2 The Three Parents of Python

Python did not descend evenly from the whole tree. Three languages shaped it directly, and each left a different mark.

ABC: the gift of readability

Through most of the 1980s, Guido van Rossum worked at CWI on a language called **ABC**, intended to teach programming to beginners and to be usable by scientists who were not professional programmers. ABC was where Python's most distinctive feature was born: the use of indentation, rather than braces or keywords, to mark which statements belong together. ABC also favoured a small number of powerful, high-level data types and an uncluttered syntax that read almost like English.

ABC ultimately failed to find an audience, and Guido came to understand why. It was a closed world. You could not easily add new features, talk to the operating system, or read and write ordinary files. It tried to be a complete environment rather than a tool that cooperated with the rest of the system. When Guido designed Python, he kept ABC's readability and threw out its isolation. Python would be extensible from the start, and it would talk to everything.

C: the language Python is written in

The standard Python interpreter is a program written in C, and this is no accident. C was the lingua franca of systems programming, the language in which the operating systems Python needed to run on were themselves built. Writing the interpreter in C meant Python could run anywhere C ran, which by 1990 meant almost everywhere. It also meant that

performance-critical pieces of Python, and later entire libraries such as NumPy, could be written in C and called from Python, giving the language a way to escape its own slowness when it mattered. We will return to this idea repeatedly; it is central to why a "slow" language powers so much fast software.



Figure 2.2. *Dennis Ritchie (1941–2011), creator of C.*

Modula-3: modules and exceptions

While at CWI, Guido encountered **Modula-3**, a language from the Digital Equipment Corporation research labs. From it Python borrowed its module system, the way code is organised into importable units, and the basic shape of its exception handling, the `try` and `except` machinery you will meet in Chapter 21. These were mature, well-considered designs, and rather than reinvent them Guido adapted what already worked.

> **DEF** **Scripting Language**
>
> A language designed for writing short programs that automate tasks, glue other programs together, or configure systems, rather than for building large standalone applications from scratch. The term has blurred over time: Python began as a scripting language and is now used to build systems of every size. The distinction was always more about intent than capability.

→ **FURTHER READING: PYTHON'S ORIGINS**

- Guido's Foreword for "Programming Python" (1996) *his own account of the early days*
- The History of Python *Guido's own blog on design decisions*
- ABC (programming language) *Wikipedia*

2.3 Guido's Design Decisions



Figure 2.3. *Guido van Rossum, who created Python over the winter of 1989 and led its development as "Benevolent Dictator For Life" until 2018.* Guido van Rossum

A language is the sum of thousands of small choices, and the choices that give Python its character were made early and held to with unusual consistency. Four of them matter most.

The first is **readability above cleverness**. Guido observed that code is read far more often than it is written: a line written once may be read a hundred times over the life of a program, by its author and by strangers. He optimised the language for the reader. This is why indentation is mandatory rather than decorative, why the language avoids cryptic punctuation, and why there is a cultural preference, captured in the Zen of Python, for there to be one obvious way to do a thing rather than many clever ones.

The second is that **everything is an object**. Numbers, strings, functions, classes, modules, even the types themselves: all are objects, with the same uniform interface. This decision, inherited from Smalltalk, gives the language a deep consistency. The whole of Chapter 3 is devoted to its consequences.

The third is **batteries included**. Python ships with a large standard library so that common tasks, reading files, parsing dates, fetching web pages, working with JSON, can be done out of the box without hunting for third-party code. This made Python immediately useful and helped it spread.

The fourth is **pragmatism over purity**. Where a purely object-oriented language insists you write `list.length()`, Python offers the blunter `len(list)` because it reads better. Where a purely functional language forbids changing data in place, Python allows it where it is convenient. Python borrows good ideas from every paradigm and refuses to be a slave to any one of them.

EXAMPLE 2.1 The Zen of Python

Hidden inside every Python installation is a short poem of design principles, written by Tim Peters in 1999. Type one line at the interpreter to read it:

PYTHON

```
import this
```

OUTPUT

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Readability counts.
...
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
```

The joke in the last line is about Guido, who is Dutch. The poem is not a specification, but when the community argues about whether a proposed feature belongs in the language, these nineteen aphorisms are quoted more often than any technical document.

→ **FURTHER READING: DESIGN PHILOSOPHY**

- PEP 20: The Zen of Python *the canonical text*
- PEP 8: Style Guide for Python Code *how Python is meant to be written*
- "Transforming Code into Beautiful, Idiomatic Python" by Raymond Hettinger *a classic talk on Pythonic style*

2.4 What CPython Actually Is

Here is a distinction that confuses many beginners and that, once clear, never confuses you again. **Python is a language specification.** It is a document describing what the language means: what `2 + 2` should produce, what a `for` loop does, how a function call behaves. **CPython is an implementation**: an actual program, written in C, that reads Python source code and executes it according to that specification. When you download "Python" from python.org, what you are installing is CPython.

CPython is by far the dominant implementation, so much so that its behaviour has, in places, become the specification by default. But it is not the only one, and the alternatives illuminate what an implementation is free to change.

Implementation	Written in	Distinctive trait
CPython	C	the reference; what almost everyone runs
PyPy	RPython	a JIT compiler, often several times faster
Jython	Java	runs on the JVM, interoperates with Java
IronPython	C#	runs on .NET
MicroPython	C	fits on a microcontroller with kilobytes of RAM
GraalPy	Java	runs on the GraalVM, high performance

The existence of PyPy is the most instructive. It runs the same Python language but, by compiling hot code to machine code as it runs (a technique called just-in-time compilation), it executes many programs several times faster than CPython. That CPython does not do this is a deliberate trade: a simpler interpreter that is easier to maintain, embed, and extend with C libraries, at the cost of raw speed. Throughout this book, unless stated otherwise, "Python" means CPython, because that is what you are almost certainly running.

→ **FURTHER READING: IMPLEMENTATIONS**

- The Python Language Reference *the specification itself*
- PyPy *the JIT-compiled implementation*
- CPython source code on GitHub *readable, well-organised, surprisingly approachable*

2.5 The Compilation Pipeline

Python is often called an interpreted language, and it is, but the description hides something. Before any interpretation happens, your source code passes through several stages of translation. Understanding these stages demystifies a great deal of the interpreter's behaviour, including why syntax errors are reported before your program runs at all, and what those `__pycache__` folders contain.

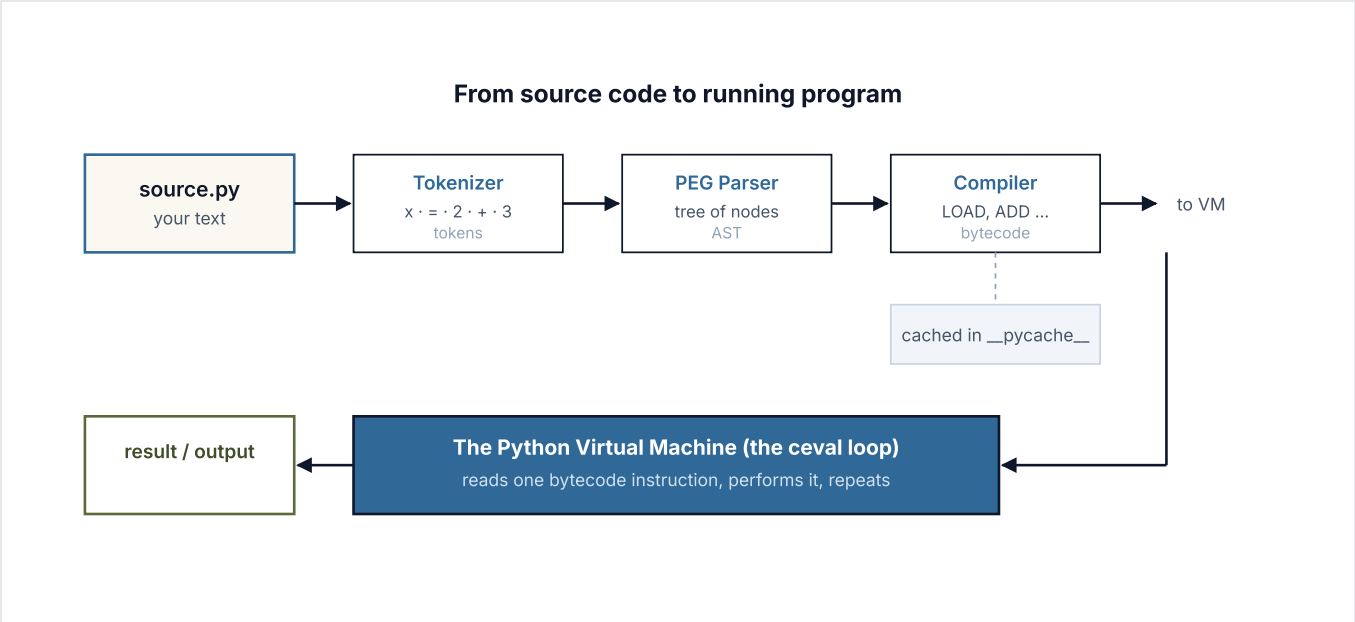


Figure 2.4. The CPython pipeline. The first three stages happen once, before execution; their result, the bytecode, is cached on disk so that unchanged modules need not be recompiled. The virtual machine then executes the bytecode instruction by instruction.

The **tokenizer** reads your source as a stream of characters and groups them into tokens: the name `x`, the operator `=`, the number `2`, and so on. The **parser** arranges those tokens into a tree that reflects the grammatical structure of the program, called the *Abstract Syntax Tree*. Since version 3.9, CPython uses a PEG parser, a more powerful design than the one it replaced. The **compiler** walks that tree and emits *bytecode*, a sequence of sim-

ple instructions for a virtual machine. Finally, the **virtual machine**, a loop in a C file historically named `ceval.c`, executes the bytecode one instruction at a time.

EXAMPLE 2.2 Looking at the bytecode

The `dis` module ("disassemble") lets you see the bytecode the compiler produced for any function. This is not a trick; it is the actual sequence the virtual machine will run.

PYTHON

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

OUTPUT

```
  2           RESUME                   0
  3           LOAD_FAST                a
              LOAD_FAST                b
              BINARY_OP                0 (+)
              RETURN_VALUE
```

Five instructions. `LOAD_FAST` pushes a local variable onto an internal stack, `BINARY_OP` pops two values and pushes their sum, and `RETURN_VALUE` hands the result back. The Python virtual machine is a *stack machine*: nearly every instruction works by pushing values onto, or popping them off, an operand stack.

DEF **Bytecode**

An intermediate, low-level instruction set, simpler than Python but higher-level than machine code, that the Python virtual machine executes. It is portable across operating systems and CPUs, which machine code is not. The `.pyc` files in `__pycache__` are compiled bytecode, cached so that importing an unchanged module skips recompilation. They are an optimisation, safe to delete; Python regenerates them as needed.

→ **FURTHER READING: THE PIPELINE**

- CPython Developer's Guide: The Compiler *how it really works, from the source*
- The dis module documentation *every bytecode instruction explained*
- PEP 617: New PEG parser for CPython *why the parser was replaced in 3.9*

2.6 Reference Counting and the Garbage Collector

In a language like C, the programmer must explicitly free every piece of memory they allocate. Forgetting to do so is a *memory leak*; freeing memory still in use is a *use-after-free*, the source of a great share of the security vulnerabilities of the last forty years. Python relieves you of this burden entirely. You never free memory by hand. Understanding how it does this for you is worth a few pages, because it explains some otherwise mysterious behaviour.

CPython's primary mechanism is **reference counting**. Every object carries a hidden counter recording how many names, list slots, or other references currently point at it. Each new reference increments the count; each reference that goes away decrements it. The instant the count reaches zero, the object is unreachable, and its memory is reclaimed immediately.

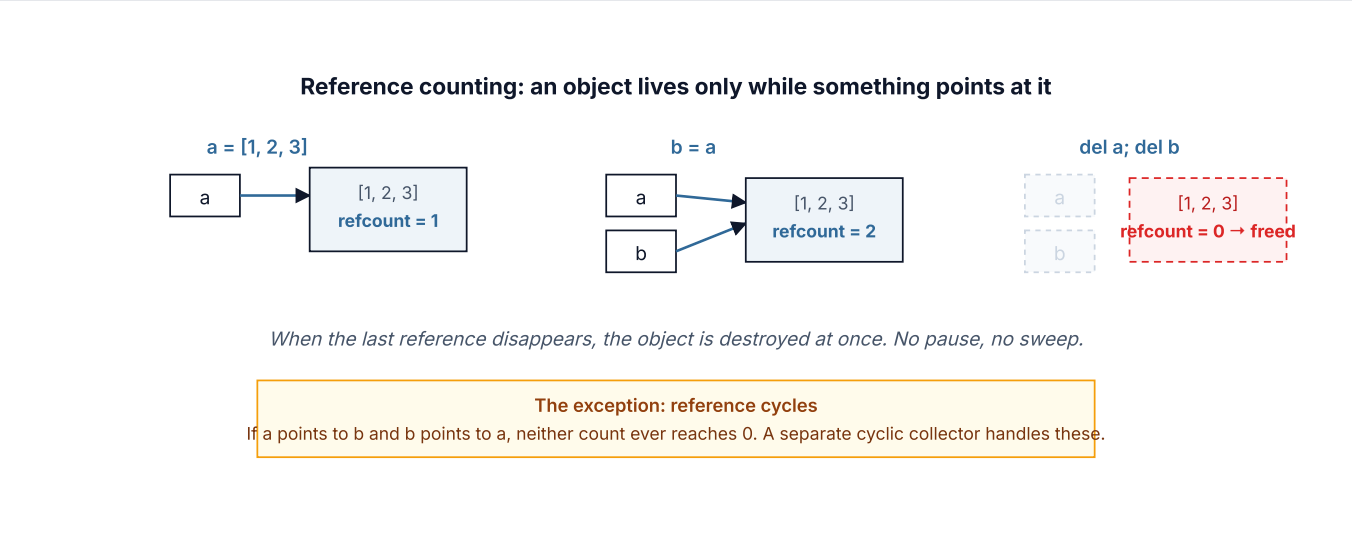


Figure 2.5. Reference counting in action. This is also the mechanism behind the aliasing behaviour of Chapter 3: when `b = a` makes two names point at one list, it is literally incrementing one shared counter.

Reference counting is simple and prompt, but it has one blind spot: *cycles*. If object A holds a reference to object B, and B holds a reference back to A, then even after every

outside name to them is gone, each still has a reference count of one, pointing at the other. Neither will ever be freed by counting alone. To catch these, CPython runs a second mechanism, a **generational cyclic garbage collector**, which periodically looks for groups of objects that refer only to each other and are reachable from nothing else, and reclaims them.

WHY TWO MECHANISMS Reference counting handles the overwhelming majority of objects instantly and predictably. The cyclic collector exists only to mop up the rare cycles that counting cannot. Splitting the work this way keeps the common case fast. You can inspect and even tune the cyclic collector through the `gc` module, though you will rarely need to.

→ **FURTHER READING: MEMORY MANAGEMENT**

- Python C-API: Reference Counting *Py_INCREF and Py_DECREF*
- CPython Developer's Guide: Garbage Collector *the cyclic collector in detail*
- Memory Management in Python *a clear, illustrated walkthrough*

2.7 The Global Interpreter Lock

No discussion of CPython is complete without the Global Interpreter Lock, universally abbreviated the GIL. It is the single most debated design decision in the language's history, and the one beginners most often hear about without understanding.

The GIL is a mutex, a lock, that ensures only one thread executes Python bytecode at any given moment, even on a processor with many cores. The reason it exists is reference counting. Recall that every object's reference count is constantly being incremented and decremented. If two threads tried to adjust the same count at the same time, the count could be corrupted, leading to objects freed too early or never freed at all. The GIL makes the interpreter's internals safe by the simplest possible means: it lets only one thread run Python code at a time, so the counts can never be touched concurrently.

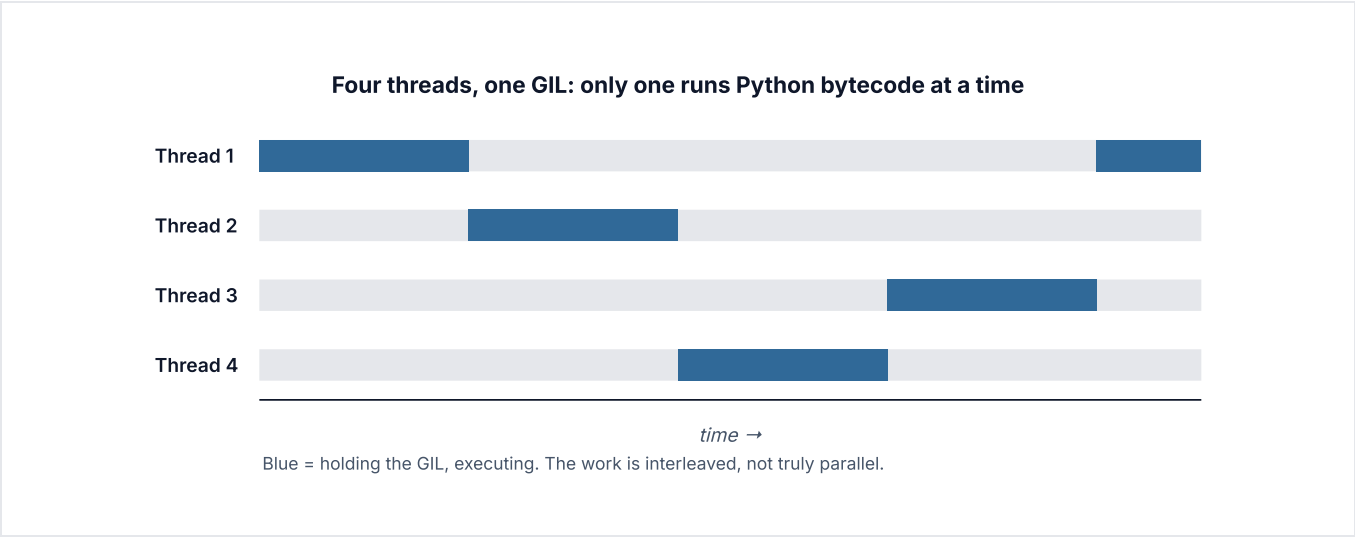


Figure 2.6. With the GIL, threads take turns. For work that spends its time waiting (network, disk), this is fine, because a waiting thread releases the GIL. For work that is pure computation, the threads cannot use multiple cores, and you reach instead for processes.

The consequence is precise and worth stating carefully. For work that is *input-output bound*, programs that spend most of their time waiting for the network, the disk, or a database, threads work well, because a thread that is waiting releases the GIL for others to use. But for work that is *CPU bound*, pure calculation, Python threads cannot run on more than one core at once, and adding threads buys you nothing. The traditional answer has been to use multiple *processes* instead, each with its own interpreter and its own GIL, communicating through channels the operating system provides. This is the subject of Chapter 21.

After three decades of debate, this is now changing. A 2023 proposal, PEP 703, laid out a way to make CPython work correctly without the GIL, and recent versions ship an experimental "free-threaded" build that removes it. Making reference counting safe without a single global lock is delicate work, and the free-threaded build is still maturing, but the long era in which the GIL was simply a fact of Python life is drawing toward its close.

→ **FURTHER READING: THE GIL**

- What Is the Python GIL? *Real Python, the clearest introduction*
- PEP 703: Making the GIL Optional *the free-threading proposal*
- "Understanding the Python GIL" by David Beazley *the famous deep-dive talk*

2.8 How Python Evolves

A living language must change without fracturing. Python's mechanism for this is the **Python Enhancement Proposal**, or PEP. A PEP is a design document proposing a new feature, a change to the language, or a process. It is written up, discussed in public, revised, and either accepted or rejected. Every significant feature of modern Python, from f-strings to type hints to the removal of the GIL, began as a PEP and can be read in full today. They are numbered, and a handful are worth knowing by heart.

PEP	Subject
PEP 1	How the PEP process itself works
PEP 8	The style guide: how to format Python code
PEP 20	The Zen of Python
PEP 484	Type hints
PEP 572	The walrus operator, `:=`
PEP 703	Making the GIL optional

For its first twenty-seven years, final decisions rested with Guido van Rossum, who held the half-joking title of *Benevolent Dictator For Life*. In 2018, after an unusually bitter argument over PEP 572, the walrus operator, Guido stepped down from that role. Governance now rests with an elected **Steering Council** of five core developers, chosen by the people who maintain the language. The arrangement, defined in PEP 13, has held steady ever since, and Python's evolution has continued without a single benevolent dictator to settle disputes.

> **A HABIT WORTH FORMING** When you encounter a Python feature you do not understand, search for "PEP" and the feature's name. The PEP will usually tell you not just what the feature does but *why* it was added, what alternatives were rejected, and what problems it was meant to solve. Few languages document their own reasoning so openly.

→ FURTHER READING: GOVERNANCE & EVOLUTION

- The PEP Index *every proposal, searchable*
- PEP 13: Python's Governance *how the Steering Council works*
- discuss.python.org *where the language's future is debated in the open*

§ § §

Exercises

EXERCISE 2.1 Specification versus implementation

In your own words, explain the difference between "Python the language" and "CPython the implementation". Name one thing an alternative implementation such as PyPy is free to change, and one thing it must keep the same.

EXERCISE 2.2 Read the Zen

Run `import this` in a Python interpreter. Pick the three aphorisms that mean the least to you right now. Write one sentence for each guessing what it might mean. Keep your answers; revisit them after Chapter 19.

EXERCISE 2.3 Disassemble a function

Write a function that takes two numbers and returns their product plus one. Use `dis.dis` to print its bytecode. Identify which instruction performs the multiplication and which performs the addition. How many instructions does the whole function compile to?

EXERCISE 2.4 Watch a reference count

Using `sys.getrefcount(obj)`, observe how the reference count of a list changes as you bind it to a second name and then delete that name. Note: `getrefcount` itself temporarily adds one reference, so the number is always one higher than you might expect. Explain why.

EXERCISE 2.5 Reason about the GIL

You have two programs. Program A downloads 100 web pages. Program B computes the prime factors of 100 large numbers. For each, decide whether Python threads would speed it up on an 8-core machine, and explain your reasoning in terms of the GIL.

EXERCISE 2.6 Trace the pipeline

Describe what happens, stage by stage, when you run `python hello.py`, from the moment you press Enter to the moment output appears. Use the vocabulary of Section 2.5: tokenizer, parser, AST, compiler, bytecode, virtual machine.

EXERCISE 2.7 Find a PEP

The walrus operator `:=` was added by PEP 572 and was controversial enough to prompt Guido's resignation as BDFL. Read the "Rationale" section of PEP 572. Summarise in three sentences what problem it was meant to solve and one objection people raised against it.

EXERCISE 2.8 A research question

PyPy runs the same Python code as CPython but is often several times faster. Spend fifteen minutes researching how a just-in-time compiler achieves this. Write a paragraph explaining, at a high level, why running code can sometimes be faster than interpreting it line by line, and what CPython gives up by not doing so.

Solutions to Chapter 2 Exercises

Solution 2.1: Specification versus implementation

"Python the language" is the set of rules describing what programs mean, written in the Language Reference. "CPython the implementation" is an actual program, written in C, that executes those rules. An implementation like PyPy is free to change *how* it runs code: it may compile hot loops to machine code, use a different garbage collector, or lay out objects in memory differently. It must keep the same *observable behaviour*: `2 + 2` must still be `4`, a `for` loop must still iterate, and a correct Python program must produce the same results. Implementations compete on speed, memory, and platform, not on meaning.

Solution 2.2: Read the Zen

Answers will vary. Two that often puzzle beginners: "There should be one obvious way to do it" is a reaction against languages that offer five interchangeable ways to write the same loop, which fragments how code is read across a team. "Although that way may not be obvious at first unless you're Dutch" is simply a joke about Guido. The value of the exercise is in returning to your guesses later and seeing how your understanding has grown.

Solution 2.3: Disassemble a function

PYTHON

```
import dis
def f(a, b):
    return a * b + 1
dis.dis(f)
```

OUTPUT

```
  2     RESUME              0
  3     LOAD_FAST           a
        LOAD_FAST           b
        BINARY_OP           5 (*)
        LOAD_CONST          1 (1)
        BINARY_OP           0 (+)
        RETURN_VALUE
```

The `BINARY_OP 5 (*)` performs the multiplication; `BINARY_OP 0 (+)` performs the addition. The function compiles to seven instructions (the exact count and form vary slightly between Python versions, which is itself a useful lesson: bytecode is an implementation detail, not part of the language specification).

Solution 2.4: Watch a reference count

```python
import sys
a = [1, 2, 3]
print(sys.getrefcount(a))   # 2: the name a, plus getrefcount's own argument
b = a
print(sys.getrefcount(a))   # 3: a, b, and the argument
del b
print(sys.getrefcount(a))   # 2 again
```

When you call `getrefcount(a)`, the value `a` is passed as an argument to that function, which means that during the call there is one extra reference to the object: the function's parameter. So the reported count is always one higher than the number of references in your own code. This is a small but perfect illustration of the principle that, in Python, passing a value to a function creates another reference to it.

Solution 2.5: Reason about the GIL

Program A (downloading 100 web pages) is input-output bound: most of its time is spent waiting for the network. A waiting thread releases the GIL, so other threads can run while it waits. Threads *would* speed this up significantly. Program B (factoring large numbers) is CPU bound: it is pure computation with no waiting. Because the GIL allows only one thread to execute Python bytecode at a time, the threads would take turns on a single core and the 8 cores would sit mostly idle. Threads would *not* help; you would use multiple processes (or a free-threaded build) instead.

Solution 2.6: Trace the pipeline

When you press Enter, the operating system starts a CPython process and hands it `hello.py`. The *tokenizer* reads the file's characters and groups them into tokens. The *parser* arranges those tokens into an Abstract Syntax Tree representing the program's structure; if the syntax is invalid, you get a `SyntaxError` here, before anything runs. The *compiler* walks the tree and emits bytecode, which may be cached as a `.pyc` file. The *virtual ma-*

chine then executes the bytecode one instruction at a time, and when it reaches a `print` call, output appears on your screen through the operating system's input-output services.

Solution 2.7: Find a PEP

PEP 572 added assignment expressions, written `:=`, which let you bind a value to a name in the middle of a larger expression, for instance inside a `while` condition or a comprehension, so the value need not be computed twice or assigned on a separate line. The chief objection was that it introduced a second way to assign (alongside `=`) and could be used to write dense, hard-to-read one-liners, in tension with the Zen's preference for one obvious way and for readability. The argument grew heated enough that Guido, exhausted by it, resigned as BDFL shortly afterward.

Solution 2.8: A research question

A just-in-time (JIT) compiler watches a program as it runs, identifies the "hot" code that executes most often (typically loops), and compiles that code to native machine instructions on the fly, so subsequent runs of that code skip interpretation entirely and run at near-C speed. It can also specialise the machine code to the actual types it observes at runtime. CPython interprets each bytecode instruction every time, paying the interpretation cost on every pass through a loop. By not having a JIT, CPython stays far simpler: easier to maintain, easier to embed in other programs, and far easier to extend with C libraries, which is the trade its designers chose. (Recent CPython versions have begun adding a limited JIT of their own, narrowing the gap.)

CHAPTER 3 · PART I · THE GROUND

The Object Model

Here Python truly begins. Before the first number or string, there is one idea from which the entire language unfolds: everything you can name is an object, and every object has the same three properties. Grasp this chapter and the rest of the book becomes a series of corollaries.

> *Everything is an object. The big idea is messaging. The key in making great and growable systems is much more to design how its modules communicate rather than what their internal properties and behaviors should be.*
>
> — Alan Kay, who coined the term "object-oriented programming"

Most introductions to Python begin with numbers and strings, treating them as different kinds of thing with different rules, and only much later, if at all, reveal that they were the same kind of thing all along. We are going to do the opposite. We will spend this chapter on the single idea that unifies the whole language, and only then, in the chapters that follow, look at the particular objects it gives you. The order is unusual, but it is the order that makes everything afterward simple. When you understand that a number, a string, a list, a function, and a class are all objects obeying one set of rules, you stop memorising special cases and start reasoning from principle.

The promise of this chapter is concrete. By its end you will understand exactly what a variable is (not what most beginners think), why two equal-looking values are sometimes the same object and sometimes not, why changing a list inside a function changes it outside but changing a number does not, why a list cannot be a dictionary key, and how the `+` sign knows how to add two numbers and also join two strings. These are not five facts to memorise. They are five consequences of one idea.

3.1 Everything Is an Object

In some languages, the world is divided in two. There are *primitives*, the small fast things like integers and characters that the hardware understands directly, and there are *objects*, the larger structured things you build yourself. The two obey different rules. A primitive cannot have methods; an object can. This division is a constant low tax on the programmer's attention, who must always remember which kind of thing they are holding.



Figure 3.1. *Alan Kay, who coined "object-oriented programming" at Xerox PARC in the 1970s.* Alan Kay

Python abolishes the division. In Python, **everything is an object**: the integer `42`, the string `"hello"`, the list `[1, 2, 3]`, the function you define, the class you write, the module you import, and even the types themselves. Every one of them is an object with the same uniform interface. This is why you can ask an integer for its methods, store a function in a list, pass a class as an argument, and treat all of them with the same handful of built-in functions.

EXAMPLE 3.1 Asking different things the same questions

The built-in `type()` reports any object's type, and `type()` works on everything, because everything is an object:

```python
type(42)            # <class 'int'>
type("hello")       # <class 'str'>
type([1, 2, 3])     # <class 'list'>
type(print)         # <class 'builtin_function_or_method'>
type(int)           # <class 'type'>, even a type is an object
type(type)          # <class 'type'>, and its type is type itself
```

The last two lines deserve a pause. The type of `int` is `type`. The type of `type` is `type`. The hierarchy does not regress forever; it closes on itself. `type` is the object whose job is to make other types, and it is its own type. This is the still point at the centre of the whole model.

A practical consequence is that functions and classes are *first-class*: they can be assigned to names, stored in containers, passed as arguments, and returned from other

functions, exactly like any other value. Much of Python's expressive power, and the whole of Chapter 16 on decorators, rests on this.

→ **FURTHER READING: OBJECTS EVERYWHERE**

- The Python Data Model *the reference chapter this one is based on*
- "Python's Class Development Toolkit" by Raymond Hettinger *objects from the ground up*
- First-class functions *Wikipedia*

3.2 Identity, Type, and Value

Every object in Python, without exception, has exactly three intrinsic properties. Understanding them, and which of them can change, resolves most of the confusions that follow.

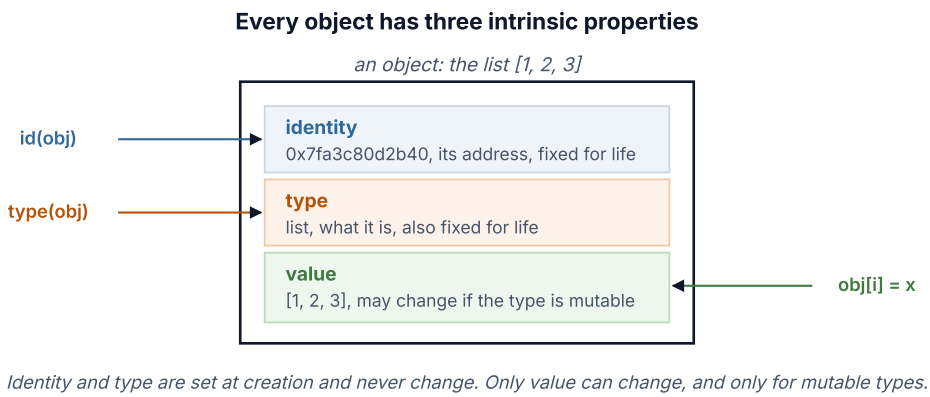


Figure 3.2. The anatomy of any Python object. In CPython, the identity is literally the object's address in memory, which is why it never changes: the object does not move.

The **identity** is what distinguishes an object from every other object that exists at the same time. It is fixed the moment the object is created and never changes for as long as the object lives. You can read it with the built-in `id()`. In CPython, the identity is simply the object's address in memory. The operator `is` compares identities: `a is b` asks whether `a` and `b` are the very same object, not merely equal ones.

The **type** determines what an object is and therefore what it can do: what operations it supports, what methods it has, how it behaves. Like identity, an object's type is fixed at

creation. You read it with `type()`. The type is itself an object, as we saw in Section 3.1.

The **value** is the data the object holds. Here is the one property that may change: whether it can depends entirely on the object's type. Types whose values can change after creation are called *mutable*; types whose values cannot are called *immutable*. This single distinction, the subject of Section 3.4, is the most consequential fact about any Python object.

> **DEF** **Identity**
>
> The property that uniquely distinguishes an object during its lifetime, returned by `id(obj)` and compared by the `is` operator. Two objects with identical values may still have different identities. In CPython, identity is the object's memory address. Do not rely on the specific number; rely only on whether two identities are equal.

→ **FURTHER READING: IDENTITY, TYPE, VALUE**

- Objects, values and types *the Python reference, Section 3.1*
- Python "is" vs "==": Identity vs Equality *Real Python*

3.3 Names Are Not Boxes

We now reach the single most important correction this book can offer a beginner. In school mathematics, and in many programming languages, a variable is a box: `x = 5` opens a box labelled `x` and puts a `5` inside it. Assigning `x = 7` replaces the contents of the box. This mental model is wrong for Python, and the difference is not academic; it is the source of more genuine confusion than any other feature of the language.

In Python, a variable is not a box. It is a **name**, and a name is a label that is *bound* to an object that exists elsewhere. The statement `x = 5` does three things, in order: it creates (or finds) an integer object with the value 5, it creates the name `x`, and it binds the name to the object, like tying a luggage tag to a suitcase. The name is not the object. The name is a way to reach the object.

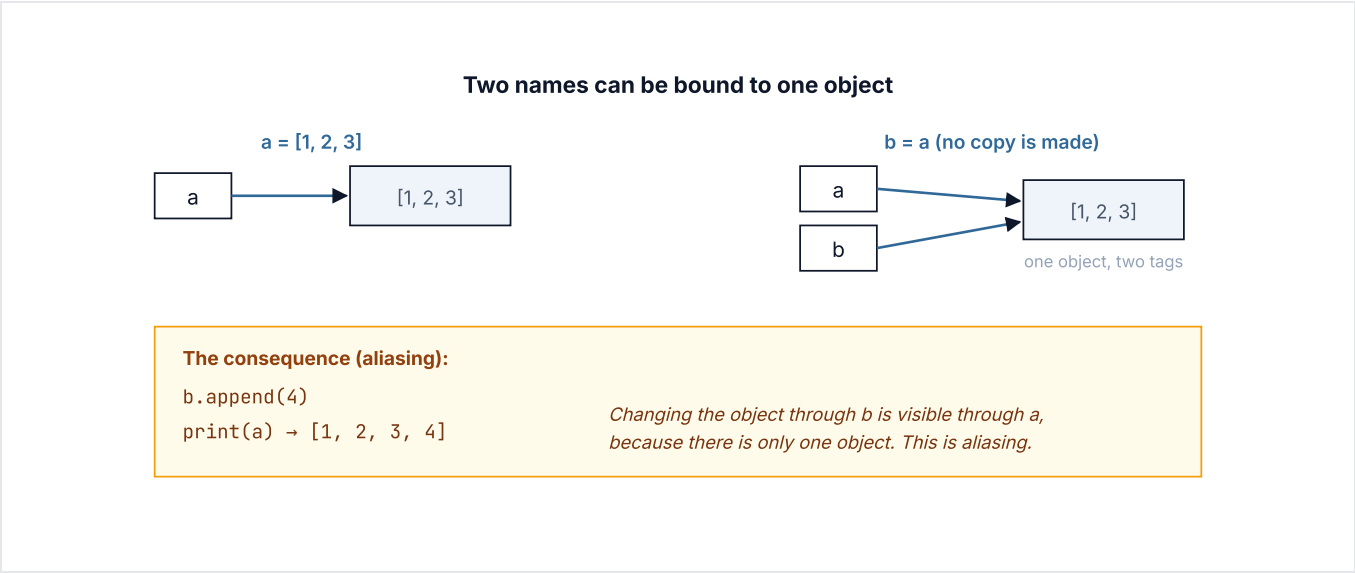


Figure 3.3. Binding, not copying. The statement `b = a` binds a second name to the same object; it does not duplicate it. This is the same shared object whose reference count we watched in Figure 2.5.

Because a name is only a label, several names can be bound to one object at once. This is called **aliasing**, and it is the source of a beginner's first truly baffling bug. When you write `b = a`, you do not copy the object; you bind a second name to it. If the object is mutable and you change it through one name, the change is visible through the other, because there was only ever one object.

EXAMPLE 3.2 Aliasing, and how to avoid it

The shared-object behaviour, and the three usual ways to ask for a genuine copy:

```python
a = [1, 2, 3]
b = a                # b is bound to the SAME list
b.append(4)
print(a)             # [1, 2, 3, 4], a sees it too
print(a is b)        # True, one object, two names

# To get an independent copy, ask for one explicitly:
c = a.copy()         # or  a[:]  or  list(a)
c.append(5)
print(a)             # [1, 2, 3, 4], unchanged
print(a is c)        # False, different objects
```

A shallow copy duplicates the outer list but not the objects inside it. If your list contains other mutable objects (lists of lists), you need `copy.deepcopy()` to copy all the way down. We return to this in Chapter 8.

Reassignment fits the same model. When you write `x = 7` after `x = 5`, you do not change the integer 5 into a 7. You bind the name `x` to a different object, the integer 7. The 5 is untouched; if nothing else refers to it, its reference count drops to zero and it is reclaimed, exactly as Chapter 2 described.

→ **FURTHER READING: NAMES AND BINDING**

- "Facts and Myths about Python names and values" by Ned Batchelder *the definitive explanation, with diagrams*
- Ned Batchelder's PyCon talk on the same *video version*

3.4 Mutability

An object is **mutable** if its value can change after it is created, and **immutable** if it cannot. This property belongs to the type, not the individual object: every list is mutable, every string is immutable, and no exceptions exist. The table below sorts the built-in types.

Immutable	Mutable
`int`, `float`, `complex`, `bool`	`list`
`str`	`dict`
`tuple`	`set`
`frozenset`	`bytearray`
`bytes`	most objects you define yourself
`None`	

The crucial subtlety is what it means to "change" an immutable object: you cannot, and any operation that appears to is in fact creating a new object and rebinding your name to it. When you write `s = s.upper()`, the string method `upper()` does not alter the original string; it builds and returns a new one, and the assignment binds `s` to that new string. The original is unchanged, and if no other name holds it, it is reclaimed.

EXAMPLE 3.3 Immutable means a new object every time

PYTHON

```python
s = "hello"
print(id(s))        # e.g. 140234881234   (some address)
s = s.upper()
print(s)            # HELLO
print(id(s))        # DIFFERENT address, a new object

# Strings reject in-place change outright:
s[0] = "J"          # TypeError: 'str' object does not support item assignmen

# A list, being mutable, accepts it:
nums = [1, 2, 3]
before = id(nums)
nums[0] = 99
print(id(nums) == before)   # True, same object, changed in place
```

THE CLASSIC TRAP Because mutability is a property of the type, a mutable object used as a function's default argument is created *once*, when the function is defined, and then shared across every call. This produces one of the most surprising bugs in the language. We cover it in full in Chapter 15, but it is, at root, simply this section's principle applied to a default value.

→ **FURTHER READING: MUTABILITY**

- Mutable and immutable objects *Python reference*
- Python's Mutable vs Immutable Types *Real Python, with memory diagrams*

3.5 What Happens When You Pass an Object to a Function

Programmers arriving from other languages often ask whether Python passes arguments "by value" or "by reference". The honest answer is that it does neither, in the sense those terms usually carry, and the confusion dissolves the moment you apply the model of names and objects you already have. Python passes arguments by *binding the parameter name to the same object the caller passed*. The name inside the function and the name outside it are two labels on one object. Some call this "call by object reference" or "call by sharing".

From this single rule, the behaviour that surprises people follows necessarily. If the object is mutable and the function mutates it in place, the caller sees the change, because there is one object. If the function instead rebinds its parameter to a new object, the caller sees nothing, because rebinding a local name has no effect on the caller's name.

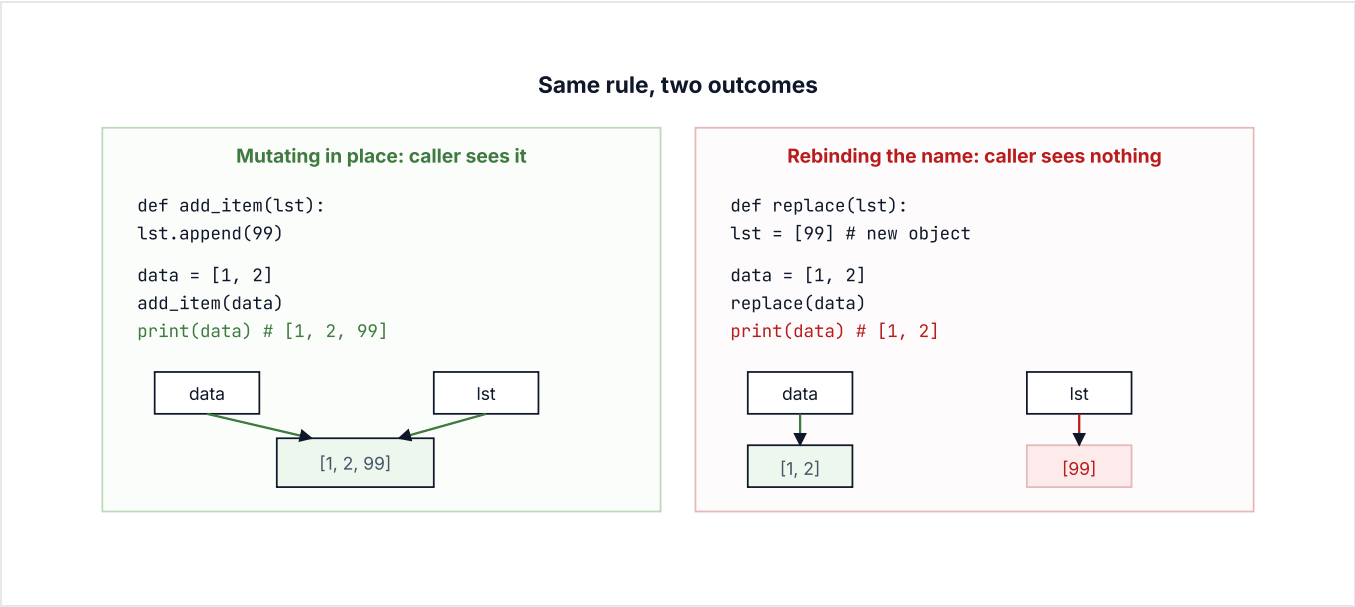


Figure 3.4. Both functions receive the same object. The left mutates it, so the caller's list changes. The right rebinds its local name to a brand-new list, leaving the caller's binding untouched.

There is nothing special about functions here. The parameter is simply a new name, bound by the call to the argument object, in exactly the way `b = a` binds a new name in ordinary code. Once you see function calls as binding, you never again have to memorise a separate rule for them.

→ **FURTHER READING: ARGUMENT PASSING**

- Python FAQ: call by reference? *the official answer*
- "Python's pass-by-object-reference" *a memorable explanation*

3.6 Equality versus Identity

Two questions can be asked of a pair of objects, and beginners frequently confuse them. *Are these the same object?* is a question of identity, answered by `is`. *Do these objects have the same value?* is a question of equality, answered by `==`. They are different questions with sometimes different answers, and using the wrong operator is a common and subtle bug.

PYTHON

```
a = [1, 2, 3]
b = [1, 2, 3]    # a separate list with equal contents
c = a
```

```
a == b      # True, same value
a is b      # False, different objects
a is c      # True, same object
```

The rule of thumb is short. Use `==` to compare values, which is what you want almost all the time. Use `is` only to ask about identity, and in practice that means almost exclusively comparing against `None`, which is a unique object: write `if x is None`, never `if x == None`. The reason `is None` is preferred is both speed and correctness, since a perverse class could define `==` to lie, but nothing can fake identity with the one true `None`.

One quirk of CPython tempts beginners into misusing `is`, and it is worth seeing so that you are never fooled by it. For efficiency, CPython keeps a single cached copy of every small integer, from -5 to 256, and reuses it. So two names set to `100` happen to point at the same cached object, and `is` returns `True`. But two names set to `1000` get separate objects, and `is` returns `False`. The values are equal in both cases; the identities are an implementation detail you must never depend on.

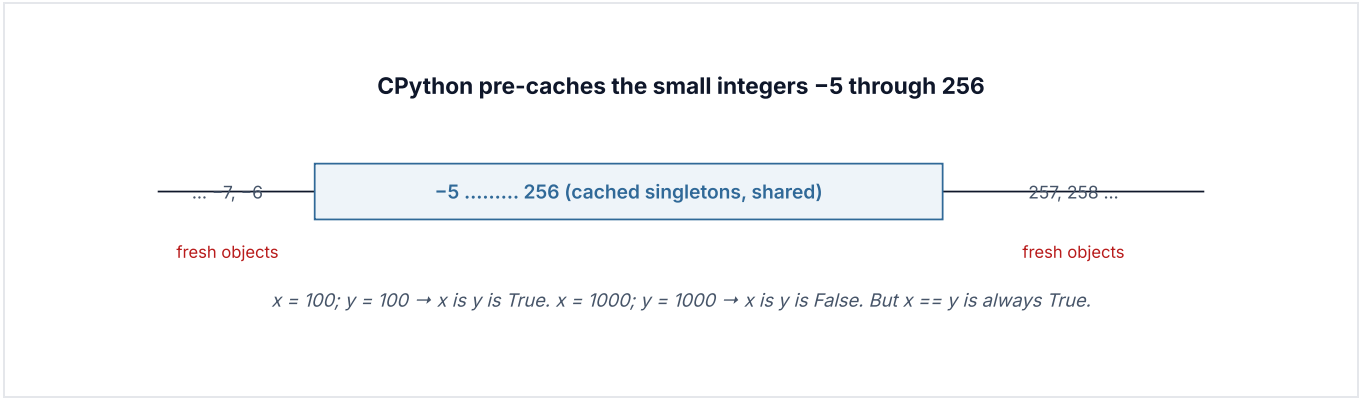


Figure 3.5. The small-integer cache. This is the canonical example of why `is` must never be used to compare values: the answer depends on an optimisation, not on the program's meaning.

→ **FURTHER READING: == VS IS**

- Comparisons *Python reference*
- "Is there a difference between == and is?" *the canonical Stack Overflow answer*

3.7 Hashability

Some objects can serve as dictionary keys or set members, and some cannot. The dividing property is called **hashability**. An object is hashable if it has a hash value that never

changes during its lifetime, provided by a method called `__hash__`, and if it can be compared for equality with other objects. Dictionaries and sets use this hash value to find things quickly, a mechanism we explain fully in Chapter 10.

The practical rule connects directly to mutability. Immutable built-in objects, numbers, strings, tuples (of hashable things), and frozensets, are hashable and may be used as keys. Mutable built-in objects, lists, dictionaries, and sets, are not hashable and may not. The reason is a matter of necessity, not arbitrary rule: if you could change an object after using it as a key, its hash would change, and the dictionary would lose track of where it stored the entry.

```python
d = {}
d[(1, 2)] = "a point"      # tuple key: fine, tuples are hashable
d["name"] = "Ada"          # str key: fine

d[[1, 2]] = "oops"         # TypeError: unhashable type: 'list'

hash((1, 2))               # a fixed integer, e.g. -3550055125485641917
hash([1, 2])               # TypeError: unhashable type: 'list'
```

THE CONTRACT Hashability ties together equality and hashing by one rule: if two objects are equal, they must have the same hash. Objects you design yourself inherit a sensible default, but if you override `__eq__` to define your own notion of equality, you must override `__hash__` to match, or your objects will misbehave as dictionary keys. Chapter 19 covers how to do this correctly.

→ **FURTHER READING: HASHING**

- Glossary: hashable *Python docs*
- object.__hash__ *the hash/eq contract in full*

3.8 The Data Model: How Objects Speak a Common Language

We can now answer the question posed at the start of the chapter: how does the `+` sign know how to add two numbers and also join two strings and also concatenate two lists?

The answer reveals the machinery beneath the whole language. Python defines a set of **special methods**, recognisable by their double-underscore names and universally nicknamed *dunder* methods, that objects implement to participate in the language's syntax. When you write `a + b`, Python does not have special knowledge of addition. It calls `a.__add__(b)`, and the object decides what addition means for its type.

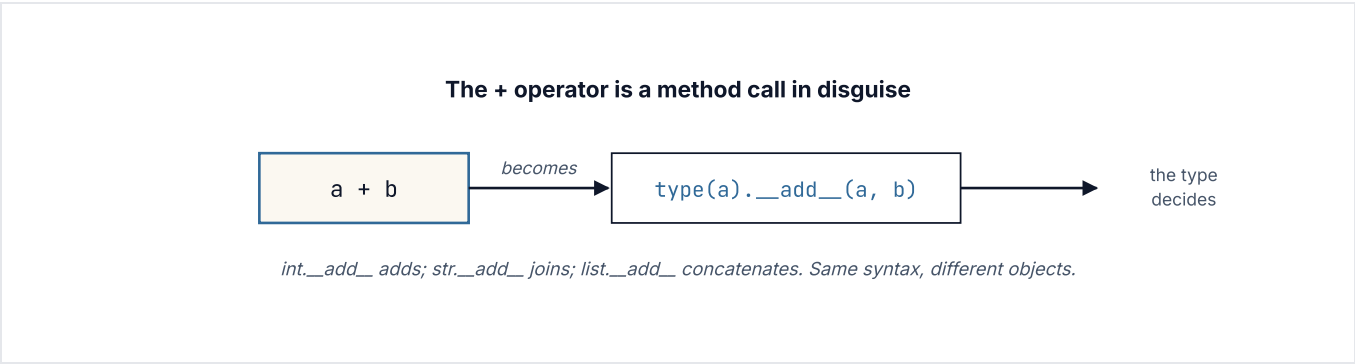


Figure 3.6. Operator dispatch. Nearly every piece of Python syntax, arithmetic, indexing, iteration, calling, comparison, is a disguised call to a dunder method, which is why your own classes can participate in all of it.

This is the deepest reason "everything is an object" is more than a slogan. The syntax of the language is uniform precisely because it is all implemented as method calls on objects, and an object of any type, including one you write yourself, can implement those methods and thereby behave like a built-in. A table of the most common protocols:

You write	Python calls	Protocol
`a + b`	`a.__add__(b)`	arithmetic
`a == b`	`a.__eq__(b)`	comparison
`len(a)`	`a.__len__()`	sizing
`a[i]`	`a.__getitem__(i)`	indexing
`for x in a`	`a.__iter__()`	iteration
`a(x)`	`a.__call__(x)`	calling
`print(a)`	`a.__str__()`	display
`repr(a)`	`a.__repr__()`	debugging display
`with a:`	`a.__enter__()` / `__exit__()`	context management

EXAMPLE 3.4 A type that behaves like a built-in

By implementing the right dunder methods, a class you write supports the same syntax as any built-in. Here a two-dimensional vector that can be added with `+` and printed legibly:

PYTHON

```python
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

v = Vector(1, 2) + Vector(3, 4)
print(v)            # Vector(4, 6)
```

The `+` worked on a type that did not exist until you wrote it, because `+` was never about numbers. It was always about `__add__`. This is the whole of Chapter 19 in miniature.

→ **FURTHER READING: THE DATA MODEL**

- Special method names *the complete list*
- Luciano Ramalho, *Fluent Python* *the definitive book on the data model*
- A Guide to Python's Magic Methods *free, well organised*

3.9 Truthiness

One more protocol deserves its own section, because it shapes how you write conditions throughout the language. Every object in Python can be tested for truth, in an `if` or a `while` or with the `and`, `or`, and `not` operators, not only the values `True` and `False`. When asked, an object reports its own truthiness through the `__bool__` method, or failing that, through `__len__` (an object with length zero is false). The values that count as false are few and worth memorising.

Falsy (count as False)	Truthy (count as True)
`False`, `None`	everything not in the falsy column
`0`, `0.0`, `0j`, `Decimal(0)`	any non-zero number
`""` (empty string)	any non-empty string
`[]`, `()`, `{}`, `set()`	any non-empty container

The idiomatic consequence is that you test a container's emptiness directly, rather than comparing its length to zero. Write `if not names:` rather than `if len(names) == 0:`; the first is shorter, reads more naturally, and works for any object that defines truthiness. This habit, small in itself, is one of the clearest marks of code written by someone who understands the object model rather than merely uses it.

EXAMPLE 3.5 Truthiness in practice

PYTHON

```python
names = []
if not names:
    print("the list is empty")      # this runs

# A custom object decides its own truth:
class ShoppingCart:
    def __init__(self):
        self.items = []
    def __bool__(self):
        return len(self.items) > 0

cart = ShoppingCart()
if not cart:
    print("your cart is empty")     # this runs
```

→ **FURTHER READING: TRUTHINESS**

- Truth Value Testing *Python docs*
- object.__bool__ *how to define your own*

§ § §

Exercises

EXERCISE 3.1 Everything has a type

Predict the output of `type(type([]))` before running it. Then run it. Explain in one sentence why the answer is what it is, referring to Section 3.1.

EXERCISE 3.2 Predict the aliasing

Without running it, predict what each `print` shows. Then run it to check.

```python
a = [1, 2, 3]
b = a
c = a[:]
b.append(4)
c.append(5)
print(a)
print(b)
print(c)
```

EXERCISE 3.3 Immutable rebinding

Write code that demonstrates, using `id()`, that `x += 1` on an integer produces a new object, while `lst.append(1)` on a list keeps the same object. Explain the difference in terms of mutability.

EXERCISE 3.4 The function-call puzzle

Write two functions, each taking one list argument. The first should mutate the list so the change is visible to the caller. The second should attempt to replace the list but leave the caller's list unchanged. Demonstrate both, and explain why they differ using the model from Section 3.5.

EXERCISE 3.5 is versus ==

Find a pair of values for which `==` is `True` but `is` is `False`, and a pair for which both are `True`. Then explain why `x = 256; y = 256; x is y` behaves differently from `x = 257; y = 257; x is y`.

EXERCISE 3.6 Why can't a list be a key?

Try to use a list as a dictionary key and observe the error. Then explain, in terms of the relationship between mutability and hashing, why allowing it would break the dictionary. What hashable object would you use instead to represent the same data?

EXERCISE 3.7 Implement a dunder

Extend the `Vector` class from Example 3.4 with a `__eq__` method so that `Vector(1, 2) == Vector(1, 2)` is `True`. Then check whether your vectors can be added to a set. What happens, and what additional method would you need to fix it?

EXERCISE 3.8 Truthiness

For each of these values, predict whether `bool(value)` is `True` or `False`: `0`, `0.0`, `""`, `" "`, `[]`, `[0]`, `None`, `{}`, `{0: 0}`. Run `bool()` on each to confirm. Which one most often surprises beginners, and why?

Solutions to Chapter 3 Exercises

Solution 3.1: Everything has a type

`type([])` is `<class 'list'>`, and `type(list)` is `<class 'type'>`, so `type(type([]))` is `<class 'type'>`. The answer is `type` because every type is itself an object whose type is `type`, the object responsible for creating types. The chain stops here because `type` is its own type.

Solution 3.2: Predict the aliasing

`a` prints `[1, 2, 3, 4]`, `b` prints `[1, 2, 3, 4]`, and `c` prints `[1, 2, 3, 5]`. Because `b = a` binds `b` to the same object as `a`, appending through `b` is visible through `a` (they are one list). But `c = a[:]` makes an independent copy, so appending 5 to `c` affects only `c`. The lesson: `=` shares, slicing copies.

Solution 3.3: Immutable rebinding

```python
x = 10
print(id(x)); x += 1; print(id(x))   # two DIFFERENT ids

lst = [1]
print(id(lst)); lst.append(2); print(id(lst))   # SAME id
```

An `int` is immutable, so `x += 1` cannot change the existing integer; it computes a new integer and rebinds `x` to it, giving a new `id`. A `list` is mutable, so `append` changes the existing object in place, and its `id` is unchanged.

Solution 3.4: The function-call puzzle

```python
def mutate(lst):
    lst.append(99)        # changes the shared object

def rebind(lst):
    lst = [99]            # binds the LOCAL name to a new object

data = [1, 2]
mutate(data); print(data)   # [1, 2, 99]
rebind(data); print(data)   # [1, 2, 99], rebind did nothing visible
```

`mutate` changes the one object both names point at, so the caller sees it. `rebind` only re-points its local parameter at a new list; the caller's name still points at the original, so nothing changes for the caller. The parameter is just another name bound by the call.

Solution 3.5: `is` versus `==`

Two distinct lists with equal contents (`[1] == [1]` is `True`, `[1] is [1]` is `False`) satisfy the first case; any object compared with itself, or `None is None`, satisfies the second. The integers `256` are the same object because CPython caches every small integer from -5 to 256 and reuses it, so `is` is `True`. The integer `257` is outside that cache, so

each occurrence is a fresh object and `is` is `False`. In both cases `==` is `True`, which is why you must use `==` for value comparison.

Solution 3.6: Why can't a list be a key?

Using a list as a key raises `TypeError: unhashable type: 'list'`. A dictionary stores each entry in a location derived from its key's hash. If the key were mutable, you could change it after insertion, its hash would change, and the dictionary would look in the wrong place and fail to find the entry. Forbidding mutable keys prevents this corruption. To represent the same ordered data as a key, use a `tuple`, which is immutable and therefore hashable: `d[(1, 2)] = ...`.

Solution 3.7: Implement a dunder

```python
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)
```

With `__eq__` defined, `Vector(1, 2) == Vector(1, 2)` is `True`. But adding the vectors to a set raises `TypeError: unhashable type: 'Vector'`. This is by design: when you define `__eq__`, Python sets `__hash__` to `None`, because the default hash (based on identity) would no longer agree with your new equality. To make the vectors usable in a set or as keys, add a matching `__hash__`, for example `def __hash__(self): return hash((self.x, self.y))`, honouring the contract that equal objects share a hash.

Solution 3.8: Truthiness

False: `0`, `0.0`, `""`, `[]`, `None`, `{}`. True: `" "` (a space is a non-empty string), `[0]` (a non-empty list, even though its only element is falsy), `{0: 0}` (a non-empty dict). The one that most often surprises beginners is `[0]`: a list containing a single zero is truthy, because truthiness of a container depends on whether it is *empty*, not on the truthiness of its contents. Likewise `" "` catches people who forget a space is a real character.

CHAPTER 4 · PART II · THE PRIMITIVES

Numbers in Five Forms

Most languages give you two kinds of number and leave you to suffer their limitations. Python gives you five, each a deliberate trade between range, exactness, and speed. Choosing the right one is a real skill, and choosing the wrong one has, more than once, destroyed rockets and bankrupted rounding errors into fortunes.

> *The fast drives out the slow even if the fast is wrong.*
>
> — William Kahan, principal architect of the IEEE 754 floating-point standard

Numbers feel like the simplest thing a computer could possibly handle, and that feeling is a trap. We learned numbers as children and stopped questioning them, but the machine does not have the luxury of the infinite paper on which we were taught to do long division. Every numeric type a computer offers is a compromise: how large a value may grow, how precisely it may be held, how quickly it may be calculated. A language's choice of numeric types is therefore a choice about which compromises to expose and which to hide, and Python's choice is unusually generous. Where C offers a handful of fixed-size integers and two sizes of float, Python offers five distinct numeric types, and understanding when each is the right tool is part of what separates a fluent programmer from a fluent typist.

This chapter covers all five: the unbounded integer, the floating-point real, the exact decimal, the exact rational, and the complex number. It explains how each is stored, where each is exact and where each lies, and how Python decides what happens when you mix them. Everything here is a direct application of Chapter 3: numbers are objects, arithmetic is method dispatch, and the differences between the types are differences in what those objects promise.

4.1 The Numeric Tower

Before the individual types, the structure that relates them. Python organises numbers into an abstract hierarchy, defined formally in the `numbers` module and known informally as the **numeric tower**. Each level is a more specialised kind of number than the one above it, and each concrete type you actually use slots in at the appropriate level.

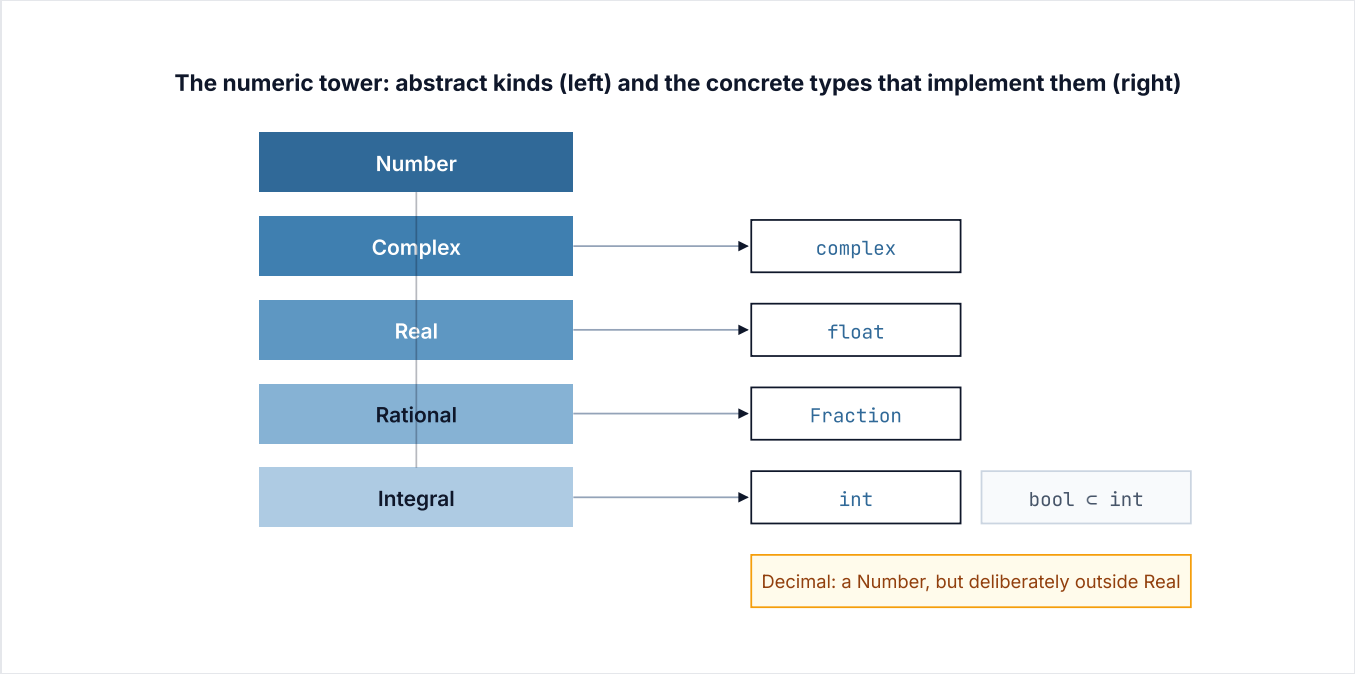


Figure 4.1. The numeric tower. The abstract kinds let code ask general questions (`isinstance(x, numbers.Integral)` means "is this any whole number?"). `Decimal` sits apart on purpose, so it will not silently mix with binary floats.

The tower exists so that code can reason about numbers in general without caring about their exact type. A function that needs any whole number can test `isinstance(n, numbers.Integral)` and accept both `int` and `bool`; a function that needs any real value can test against `numbers.Real`. You will not use the `numbers` module often, but knowing the tower is there explains why `int` and `float` cooperate so smoothly, and why `Decimal` sometimes refuses to.

→ **FURTHER READING: THE NUMERIC TOWER**

- PEP 3141: A Type Hierarchy for Numbers *the design rationale*
- The numbers module *Python docs*

4.2 Integers: Unbounded by Design

Python's `int` has no maximum value. It grows as large as the memory of your machine allows, and arithmetic on it never overflows. This is unlike C, Java, and most other languages, where an integer is a fixed-size box (commonly 32 or 64 bits) that silently wraps around when it overflows, the flaw that, as Chapter 1 described, destroyed the Ariane 5 rocket. Python pays for safety with speed: a Python integer is a full object, not a raw machine word, and arithmetic on it is slower. For the vast majority of programs the safety is worth far more than the speed.

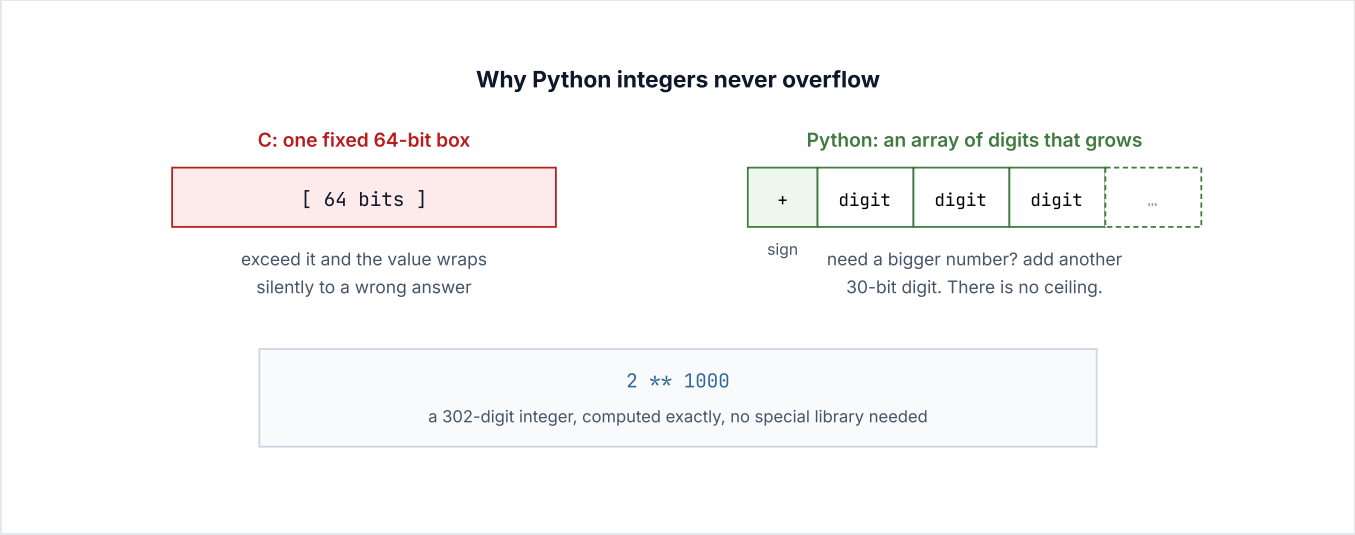


Figure 4.2. CPython stores an integer as a sign plus an array of 30-bit "digits". When a value outgrows the current array, Python allocates a longer one. This is why `2 ** 1000` simply works.

EXAMPLE 4.1 Integers that no fixed-size type could hold

```python
factorial = 1
for i in range(1, 51):
    factorial *= i
print(factorial)
# 30414093201713378043612608166064768844377641568960512000000000000

print((2 ** 1000).bit_length())   # 1001, the number needs 1001 bits
print((255).bit_count())          # 8, number of 1-bits set
```

Literals, bases, and bit operations

Integer literals can be written in four bases, may use underscores as digit separators for readability, and support the full set of bitwise operators inherited from C.

```python
255          # decimal
0b1111_1111  # binary, with an underscore separator
0o377        # octal
0xff         # hexadecimal
int("ff", 16)  # 255, parse a string in a given base

5 & 3     # 1   bitwise AND
5 | 3     # 7   bitwise OR
5 ^ 3     # 6   bitwise XOR
1 << 4    # 16  left shift (multiply by 2**4)
256 >> 2  # 64  right shift (floor-divide by 2**2)
```

→ **FURTHER READING: INTEGERS**

- Numeric types *Python docs*
- How Python implements integers *the digit-array layout in detail*

4.3 Floats and the IEEE 754 Reality

Python's `float` is a 64-bit IEEE 754 double-precision number, the exact layout we dissected in Figure 1.11: one sign bit, eleven exponent bits, fifty-two fraction bits. It is the same floating-point type used by C, Java, JavaScript, and essentially every modern processor, and it carries the same famous hazard. A float is an *approximation* of a real number, not an exact value, and the approximation occasionally surprises you.



Figure 4.3. *William Kahan, who led the IEEE 754 standard and won the 1989 Turing Award for it.* William Kahan

```python
0.1 + 0.2          # 0.30000000000000004
0.1 + 0.2 == 0.3   # False
```

This is not a bug in Python; it is the nature of representing decimal fractions in binary, and Chapter 1 explained why 0.1 has no exact binary form. What is worth adding here is *why*

the interpreter usually hides the problem. Since Python 3.1, the `repr` of a float prints the shortest decimal string that round-trips back to the same underlying value, so `repr(0.1)` shows the tidy `0.1` even though the stored value is the long expansion. The display is honest about the value's identity but merciful about its ugliness. The ugliness only surfaces when arithmetic makes two approximations disagree, as in `0.1 + 0.2`.

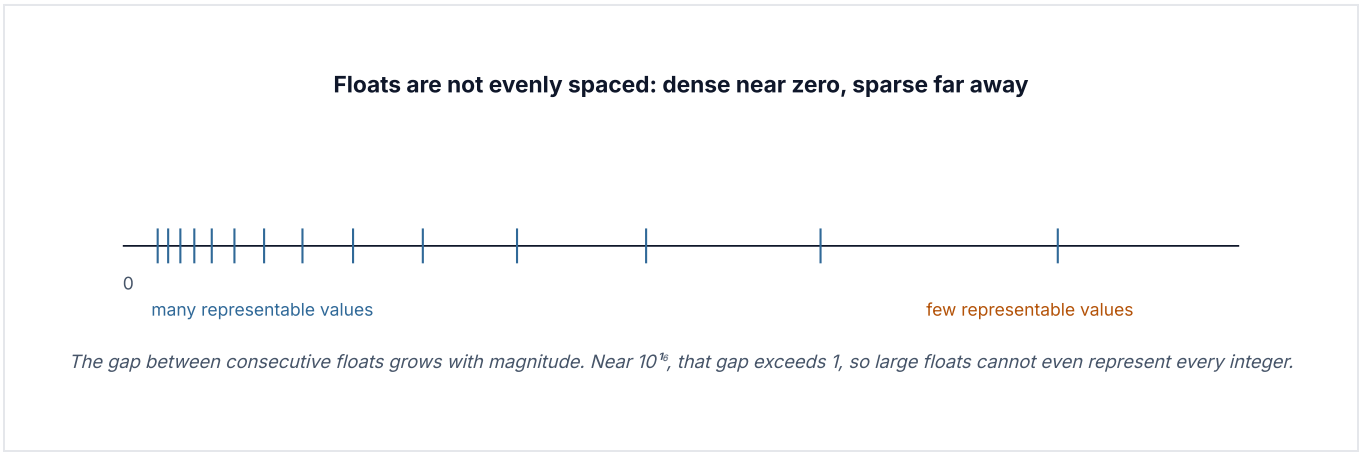


Figure 4.4. Floating-point values are not spread evenly along the number line. This is why precision is lost on large magnitudes, and why `2.0 ** 53 + 1 == 2.0 ** 53` is `True`.

The correct way to compare two floats is therefore not `==` but a tolerance check. The `math` module provides one built for the purpose, `math.isclose`, which compares with both a relative and an absolute tolerance.

EXAMPLE 4.2 Comparing floats safely, and the special values

```python
import math
math.isclose(0.1 + 0.2, 0.3)        # True

# IEEE 754 also defines three values that are not ordinary numbers:
math.inf                             # positive infinity
-math.inf                            # negative infinity
math.nan                             # "not a number", the result of 0/0, i

# nan is the one value not equal to itself:
math.nan == math.nan                 # False!
math.isnan(math.nan)                 # True, the correct way to test

import sys
sys.float_info.max                   # 1.7976931348623157e+308
sys.float_info.epsilon               # 2.220446049250313e-16
```

The behaviour of `nan` is not a Python quirk; the IEEE standard mandates that `nan` compare unequal to everything, including itself. Code that sorts or deduplicates floating-point data must handle it deliberately, or it will behave strangely in ways that are maddening to debug.

HISTORY In 1994, a flaw in the floating-point division unit of Intel's new Pentium processor produced wrong answers for certain divisions. Intel initially dismissed it as rare, then, under public pressure, recalled the chips at a cost of around 475 million dollars. The episode established the modern expectation that arithmetic hardware must be provably correct, and it remains the textbook example of why floating-point behaviour is treated with such seriousness.



Figure 4.5. *An original Intel Pentium. A subtle error in its division lookup table, the "FDIV bug" of 1994, forced a recall costing roughly 475 million dollars and became a lasting lesson in the cost of floating-point mistakes.* Pentium FDIV bug

→ **FURTHER READING: FLOATING POINT**

- The Floating-Point Guide *the friendliest introduction*
- Goldberg, "What Every Computer Scientist Should Know About Floating-Point" *the canonical paper*
- Python tutorial: Floating-Point Arithmetic *official, concise*

4.4 Decimal: When the Cents Must Add Up

When a bank totals an account or a tax authority computes what is owed, floating-point error is not a curiosity but a liability. For these uses Python provides `decimal.Decimal`, a type that performs arithmetic in base ten, exactly as a careful human would with pencil and paper, so that `0.1 + 0.2` is precisely `0.3` and not a hair more.

EXAMPLE 4.3 Decimal arithmetic, and the construction trap

```python
from decimal import Decimal, getcontext, ROUND_HALF_UP

Decimal("0.1") + Decimal("0.2")        # Decimal('0.3'), exact

# THE TRAP: build Decimals from strings, never from floats.
Decimal(0.1)      # Decimal('0.1000000000000000055511151231257...')
Decimal("0.1")    # Decimal('0.1'), clean

# A currency calculation, rounded to cents:
price = Decimal("19.99")
tax   = Decimal("0.0825")
total = (price * (1 + tax)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP
print(total)        # 21.64
```

The trap is worth stating loudly: `Decimal(0.1)` captures the float's error, because by the time the literal `0.1` reaches the `Decimal` constructor it is already a flawed binary float. `Decimal("0.1")`, from a string, is exact. Always build money values from strings.

A `Decimal` carries a *context* that controls how many significant digits it keeps (the default is 28) and how it rounds (the default is banker's rounding, which we meet again in Section 4.9). You can adjust the context globally with `getcontext()` when a calculation needs more or fewer digits, and `quantize` rounds a value to a fixed number of decimal places, which is exactly what currency requires.

→ **FURTHER READING: DECIMAL**

- The decimal module *Python docs, with a FAQ on money*
- The General Decimal Arithmetic Specification *the standard Python's Decimal implements*

4.5 Fraction: Exact Rationals

Where `Decimal` is exact in base ten, `fractions.Fraction` is exact in every base, because it stores a number as a true ratio of two integers, a numerator and a denominator,

always reduced to lowest terms. One third plus one sixth is exactly one half, with no rounding anywhere, because the arithmetic is done on the integers themselves.

```python
from fractions import Fraction

Fraction(1, 3) + Fraction(1, 6)     # Fraction(1, 2), exact
Fraction("0.25") * 4                 # Fraction(1, 1)
Fraction(6, 8)                       # Fraction(3, 4), auto-reduced

# Recover a clean fraction from a messy float:
Fraction(0.5)                        # Fraction(1, 2)
Fraction(0.1).limit_denominator(1000)  # Fraction(1, 10), the intended value
```

Fractions are the right tool when exactness matters and the values are genuinely rational: probability calculations, symbolic mathematics, and anywhere a result like "two thirds" must stay two thirds rather than becoming `0.6666666666666666`. The `limit_denominator` method is a small gem, recovering the simple fraction a human most likely intended from a float that approximates it.

→ **FURTHER READING: FRACTIONS**

- The fractions module *Python docs*

4.6 Complex Numbers

Python is one of the few mainstream languages with complex numbers built into the core, no import required. A complex literal is written with a `j` suffix on its imaginary part, following the engineering convention rather than the mathematician's `i`. Complex numbers are indispensable in signal processing, electrical engineering, and physics, and having them in the language proper rather than in a library makes that work noticeably cleaner.

```python
z = 2 + 3j
z.real            # 2.0
z.imag            # 3.0
abs(z)            # 3.605551275463989, the magnitude (hypotenuse)
z.conjugate()     # (2-3j)
z * (1 - 2j)      # (8-1j)
```

```
import cmath
cmath.sqrt(-1)      # 1j, the square root math.sqrt cannot give you
cmath.phase(z)    # 0.982..., the angle in radians
```

The companion module `cmath` provides the complex versions of the trigonometric and exponential functions that `math` offers only for reals. Note that `math.sqrt(-1)` raises an error, while `cmath.sqrt(-1)` happily returns `1j`: the two modules embody different promises about what a number is allowed to be.

→ **FURTHER READING: COMPLEX NUMBERS**

- The cmath module *Python docs*

4.7 A Surprise: `bool` Is an `int`

Here is a fact that startles most newcomers and that is, once seen, a small delight. In Python, `bool` is not a separate type alongside `int`; it is a *subclass* of `int`. `True` is, for every arithmetic purpose, equal to `1`, and `False` is equal to `0`. This was done in 2003 for backward compatibility, when booleans were added to a language that had long used 1 and 0 in their place, and it has proven quietly useful ever since.

PYTHON

```
True == 1                 # True
False == 0                # True
isinstance(True, int)     # True
True + True               # 2

# The practical payoff: counting with sum()
votes = [True, False, True, True, False]
sum(votes)                # 3, counts the True values

scores = [82, 45, 91, 67, 30]
sum(s >= 60 for s in scores)  # 3, how many passed
```

That last idiom, summing a sequence of comparisons to count how many are true, is genuinely Pythonic and reads cleanly once you know that a boolean is a number. It is a direct

consequence of the object model: `True` is an object whose type happens to be a subclass of `int`, so it inherits every numeric behaviour.

→ **FURTHER READING: BOOLEANS AS INTEGERS**

○ PEP 285: Adding a bool type *why bool subclasses int*

4.8 Mixing Types: Operator Dispatch and Division

What happens when you add an `int` to a `float`? The answer applies the data model from Chapter 3. The expression `1 + 2.0` first asks the integer, `(1).__add__(2.0)`. The integer does not know how to add a float to itself and returns the special value `NotImplemented`. Python then asks the float, from the other side, `(2.0).__radd__(1)`, and the float, which does know how to absorb an integer, converts the `1` to `1.0` and returns `3.0`. The result climbs the numeric tower to the more general type.

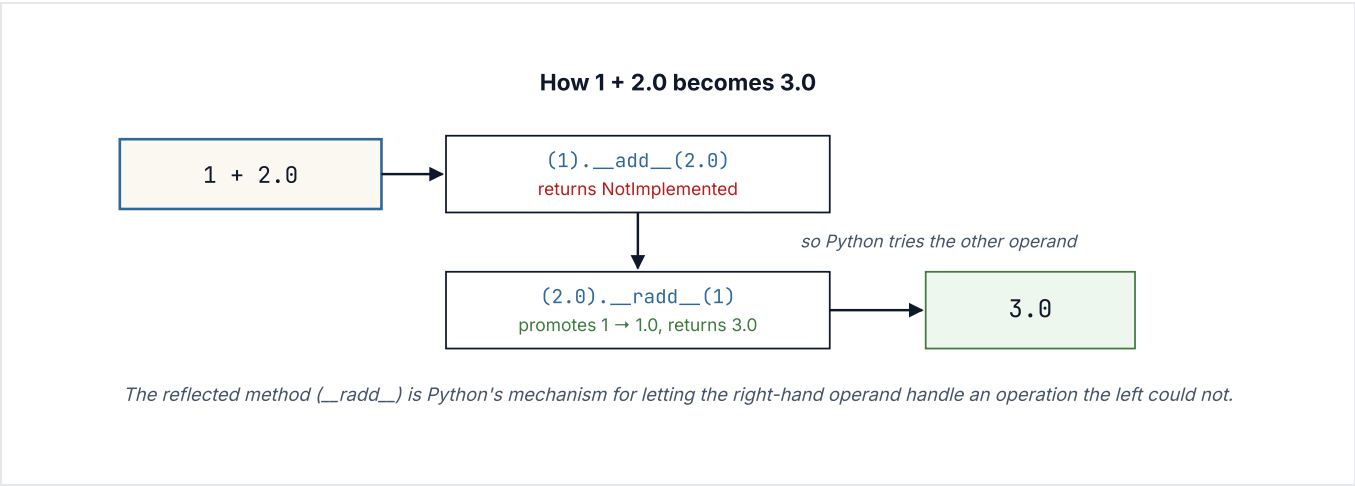


Figure 4.6. Mixed-type arithmetic resolved by reflected operators. This is the same dispatch shown in Figure 3.6, now with the fallback step that lets the second operand take over.

The two divisions, and the sign of the remainder

Python has two division operators, and the difference trips up nearly everyone once. True division, `/`, always produces a float, even when the operands divide evenly: `6 / 2` is `3.0`, not `3`. Floor division, `//`, rounds the result down toward negative infinity, and the modulo operator, `%`, returns a remainder whose sign follows the *divisor*, not the dividend. This last rule differs from C and surprises people, but it has the elegant property

that `a % n` is always in the range from 0 up to `n` for a positive `n`, which is usually what you want.

```python
7 / 2        # 3.5 , always a float
7 // 2       # 3   , floor division
-7 // 2      # -4  , rounds toward negative infinity, not toward zero
-7 % 2       # 1   , sign follows the divisor (positive here)
divmod(17, 5)  # (3, 2), quotient and remainder together
```

→ **FURTHER READING: OPERATORS**

- Emulating numeric types *__add__, __radd__, and the rest*
- Guido: "Why Python's integer division floors" *the rationale, first-hand*

4.9 The math Toolkit, and One Last Trap

Python's numeric work is supported by a family of standard-library modules. You will reach for these constantly, and knowing which one owns which job saves a great deal of searching.

Module	Use it for
`math`	real-valued functions: `sqrt`, `log`, `sin`, `floor`, `gcd`, `isclose`, `factorial`
`cmath`	the same, for complex numbers
`decimal`	exact base-ten arithmetic (money)
`fractions`	exact rational arithmetic
`statistics`	`mean`, `median`, `stdev`, `quantiles`
`random`	random numbers, choices, shuffling, sampling

A few `math` functions are worth knowing by name because they solve problems people often solve worse by hand: `math.isqrt(n)` gives the exact integer square root with no float involved; `math.fsum(values)` adds a sequence of floats with far less accumulated

error than the built-in `sum`; `math.comb(n, k)` and `math.perm(n, k)` give combinations and permutations; and `math.prod(values)` multiplies a sequence as `sum` adds one.

THE LAST TRAP: ROUND() ROUNDS TO EVEN

Python's built-in `round` does not round halves up the way most of us learned in school. It uses *banker's rounding*, rounding a value that sits exactly halfway to the nearest *even* digit. So `round(0.5)` is `0`, `round(1.5)` is `2`, and `round(2.5)` is also `2`. This is mandated by IEEE 754 and is statistically less biased than always rounding up, but it astonishes people who do not expect it. When you need school-style rounding, use `Decimal` with an explicit `ROUND_HALF_UP`.

EXAMPLE 4.4 The toolkit in use

PYTHON

```python
import math, statistics

math.isqrt(10**18)             # 1000000000, exact, no float error
math.fsum([0.1] * 10)          # 1.0 exactly; sum([0.1]*10) gives 0.999999999
math.comb(52, 5)               # 2598960, five-card poker hands

statistics.mean([82, 91, 67])   # 80
statistics.stdev([82, 91, 67])  # 12.165...

round(2.5)                     # 2, banker's rounding, not 3
round(3.5)                     # 4
```

→ **FURTHER READING: THE NUMERIC TOOLKIT**

- The math module *Python docs*
- The statistics module *surprisingly capable for everyday stats*
- "How to Round Numbers in Python" *Real Python, on banker's rounding*

§ § §

Exercises

EXERCISE 4.1 Beyond 64 bits

Compute `2 ** 200` and confirm with `.bit_length()` that the result needs 201 bits. Explain in one sentence why this calculation, which would silently overflow a 64-bit integer in C, simply works in Python.

EXERCISE 4.2 The 0.1 problem

Evaluate `0.1 + 0.1 + 0.1 == 0.3` and explain the result. Then rewrite the comparison using `math.isclose` so that it gives the answer a human would expect.

EXERCISE 4.3 Money done right

A shop sells an item for 19.99 with 8.25 percent tax, and a customer buys three. Compute the total two ways: once with ordinary floats, once with `Decimal` built from strings and rounded to the nearest cent. Show that the float version can leave a fraction of a cent, and explain why the `Decimal` version is the one an accountant would accept.

EXERCISE 4.4 Recover the fraction

The float `0.333333333333` is meant to represent one third. Use `Fraction` and `limit_denominator` to recover `Fraction(1, 3)` from it. Then show that `Fraction(1, 3) + Fraction(1, 3) + Fraction(1, 3)` equals exactly `1`, something the float version cannot promise.

EXERCISE 4.5 Count with booleans

Given `temps = [12, 5, 22, 18, 3, 27, 9]`, use the fact that `bool` is an `int` to count, in a single expression with `sum`, how many temperatures are above 15. Do not use a loop or a list comprehension that builds a list.

EXERCISE 4.6 The sign of the remainder

Predict the values of `-7 // 2`, `-7 % 2`, `7 // -2`, and `7 % -2` before running them. Then run them and reconcile any surprises with the rule that the remainder's sign follows the divisor.

EXERCISE 4.7 nan behaves strangely

Create a list `[1.0, float("nan"), 2.0]` and try to check whether `float("nan")` is "in" it using the `in` operator. Then try sorting the list. Explain both results in terms of the rule that `nan` is not equal to itself.

EXERCISE 4.8 A research question

Pick the right numeric type for each of these and justify your choice in one sentence: (a) the number of users registered on a website; (b) the balance of a bank account; (c) the position of a particle in a physics simulation; (d) the exact probability of drawing a specific poker hand; (e) the impedance of an electrical circuit.

Solutions to Chapter 4 Exercises

Solution 4.1: Beyond 64 bits

```python
n = 2 ** 200
print(n.bit_length())   # 201
```

It works because a Python `int` is not a fixed-size machine word but an object holding a growable array of digits, so when a value needs more bits Python allocates a longer array instead of overflowing.

Solution 4.2: The 0.1 problem

`0.1 + 0.1 + 0.1 == 0.3` is `False`, because each `0.1` is a tiny bit off from the true one-tenth, the three errors accumulate, and the sum lands just beside the (also slightly

off) stored value of `0.3`. The correct comparison is `math.isclose(0.1 + 0.1 + 0.1, 0.3)`, which is `True` because it allows for a small tolerance rather than demanding bit-for-bit equality.

Solution 4.3: Money done right

```python
from decimal import Decimal, ROUND_HALF_UP
# float version
print(19.99 * 1.0825 * 3)            # 64.927425000000005
# Decimal version
price, tax = Decimal("19.99"), Decimal("0.0825")
total = (price * (1 + tax) * 3).quantize(Decimal("0.01"), ROUND_HALF_UP)
print(total)                                 # 64.93
```

The float result carries a trailing `...005`, a fraction of a cent that has no meaning in money and that, summed across millions of transactions, becomes real error. The `Decimal` version computes in base ten and rounds explicitly to the cent, so it is exact and auditable, which is what an accountant requires.

Solution 4.4: Recover the fraction

```python
from fractions import Fraction
f = Fraction(0.333333333333).limit_denominator(1000)
print(f)                             # 1/3
print(f + f + f)                     # 1
```

`limit_denominator` finds the simplest fraction within a small tolerance of the float, recovering the one third the number was meant to be. Because `Fraction` stores an exact ratio of integers, adding it to itself three times gives exactly `1`, whereas `0.333333333333 * 3` falls slightly short.

Solution 4.5: Count with booleans

```python
temps = [12, 5, 22, 18, 3, 27, 9]
sum(t > 15 for t in temps)           # 3
```

Each comparison `t > 15` yields `True` or `False`, which are `1` and `0` as integers, so summing the generator counts how many comparisons were true. The values 22, 18, and 27 exceed 15, giving 3.

Solution 4.6: The sign of the remainder

`-7 // 2` is `-4` (floor division rounds down, toward negative infinity, not toward zero). `-7 % 2` is `1` (the remainder's sign follows the positive divisor). `7 // -2` is `-4` (again rounding down). `7 % -2` is `-1` (the divisor is negative, so the remainder is too). The invariant that ties them together is `(a // b) * b + (a % b) == a`, which holds in every case.

Solution 4.7: nan behaves strangely

Sorting raises no error but produces a meaningless order, because the comparisons used by the sort all involve `nan`, which returns `False` for every ordering question, so `nan` lands wherever it happened to start. The membership test is subtler: `float("nan") in [1.0, float("nan"), 2.0]` may return `True`, because `in` first checks identity (`is`) and only then equality, and if it is the very same `nan` object the identity check succeeds even though `nan == nan` is `False`. Both behaviours trace to the IEEE rule that `nan` is unequal to everything, itself included.

Solution 4.8: A research question

(a) `int`: a count is a whole number with no upper bound worth fearing. (b) `Decimal`: money must be exact in base ten and auditable. (c) `float`: physical positions are approximate anyway, and speed matters in a simulation with millions of steps. (d) `Fraction`: a probability like "combinations over total" is exactly rational, and keeping it exact avoids rounding. (e) `complex`: impedance has a real (resistance) and imaginary (reactance) part by its very definition.

CHAPTER 5 · PART II · THE PRIMITIVES

Strings & Text

Most real programs spend most of their time moving text around: reading it, cleaning it, searching it, assembling it into reports. Python's handling of text is among the finest in any language, and it rests on one decision that took the industry fifty years to get right: a string is a sequence of characters, not a sequence of bytes.

> *We were eating dinner in a New Jersey diner. Ken wrote it on the back of a placemat. By the time we got back to the office, UTF-8 was essentially done.*
>
> — Rob Pike, recalling the 1992 design of the UTF-8 encoding with Ken Thompson

There is a quiet revolution buried in the difference between Python 2 and Python 3, and it is almost entirely about text. In the older language, a string was a sequence of bytes, and the unspoken assumption that one byte equalled one character worked beautifully for English and collapsed for everyone else. Python 3 made the harder, correct choice: a string is a sequence of *characters*, abstract symbols drawn from every writing system humanity has ever devised, and bytes are a separate thing you convert to and from deliberately. This single change broke a great deal of old code and saved a generation of new programmers from a category of bug so common it has its own name. Understanding it is the foundation on which everything else in this chapter rests.

We will start with what a string actually is, take a necessary detour through Unicode and encodings (the part most books skip and most programmers later wish they had not), then cover the practical craft: writing string literals, slicing them, the methods that do the daily work, the three generations of formatting, how to build strings without quadratic slowdown, the binary counterpart in `bytes`, and a first proper look at regular expressions.

5.1 What a String Really Is

A Python string is an immutable sequence of **Unicode code points**. A code point is simply a number assigned to a character by the Unicode standard: the capital letter A is code point 65, the Greek alpha is 945, the Devanagari letter क is 2325, and the snake emoji is 128013. A string is a sequence of these numbers, and the characters you see on screen are how they are drawn. Crucially, one element of a Python string is one code point, regardless of how many bytes that code point will take when it is eventually stored or transmitted.

Two built-in functions convert between a character and its code point, and they are the cleanest way to see the principle in action.

```python
ord("A")        # 65  , the code point of a character
chr(65)         # 'A' , the character for a code point
ord("🐍")       # 128013
chr(128013)     # '🐍'

len("🐍")       # 1   , ONE code point, even though it is an emoji
```

That final line is worth dwelling on. In Python 3, the length of the snake emoji is one, because it is one character. In JavaScript and Java, whose strings are sequences of sixteen-bit units, the same emoji has a length of two, because it does not fit in sixteen bits and must be stored as a pair. Python spares you that complication entirely: a character is a character, and its length is one.

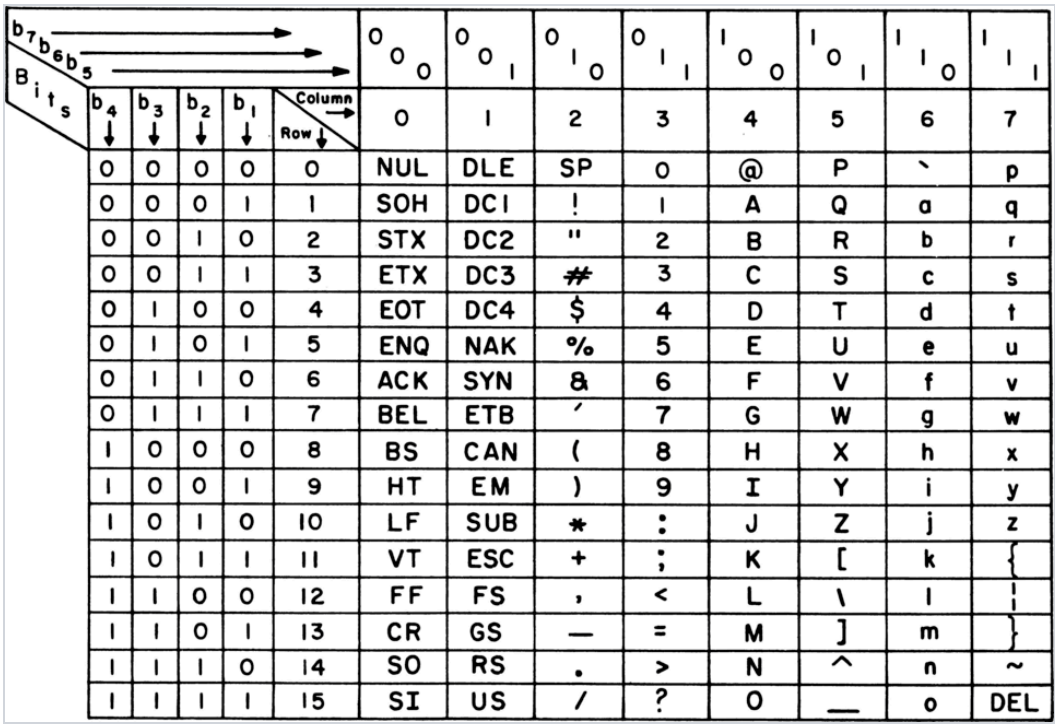
DEF **Code Point**

An integer between 0 and 1,114,111 (0x10FFFF) that the Unicode standard assigns to a single character, written in the form U+0041 for the letter A. A Python string is a sequence of code points. How those code points are turned into actual bytes for storage or transmission is a separate matter, called encoding, and is the subject of the next section.

→ **FURTHER READING: STRINGS AS CODE POINTS**

- Python Unicode HOWTO *the official guide, excellent*
- The Unicode Consortium *the standard itself*

5.2 Unicode and Encodings



Bits b_4	b_3	b_2	b_1	Row ↓ / Column →	0 (000)	1 (001)	2 (010)	3 (011)	4 (100)	5 (101)	6 (110)	7 (111)
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	¦
1	1	0	1	13	CR	GS	—	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

Figure 5.1. *The US-ASCII code chart of 1968. With seven bits it could encode 128 symbols, enough for English but for nothing else. The inadequacy of this chart for the rest of the world's writing is the whole reason Unicode had to be invented.* ASCII

For the first decades of computing, the dominant character set was ASCII, which used seven bits to encode 128 symbols: the English alphabet in two cases, the digits, punctuation, and some control codes. It was an American standard for an American industry, and it had no room for the accented letters of French, the umlauts of German, the Cyrillic of Russian, or the thousands of characters of Chinese. A patchwork of incompatible eight-bit and multi-byte extensions grew up to fill the gap, and text written under one scheme turned to garbage when read under another. The garbage was so common that the Japanese gave it a name, *mojibake*, "character transformation".

Unicode, begun in 1987, was the fix: a single universal catalogue assigning a unique code point to every character in every script ever used by humans, with room for more than a million. But Unicode only assigns the numbers. It does not say how to store them as bytes, and that is the job of an *encoding*. The same code point can be turned into bytes in several ways, and the dominant way, by an overwhelming margin, is UTF-8.

> **HISTORY** UTF-8 was designed in September 1992 by Ken Thompson and Rob Pike, the creators of Unix and later Go, over dinner at a diner in New Jersey, with the first draft

written on a placemat. Its genius is that the 128 ASCII characters keep their original single-byte values, so every ASCII file is already a valid UTF-8 file, while other characters expand to two, three, or four bytes. This backward compatibility is why UTF-8 conquered the web: adopting it cost nothing for existing English text.

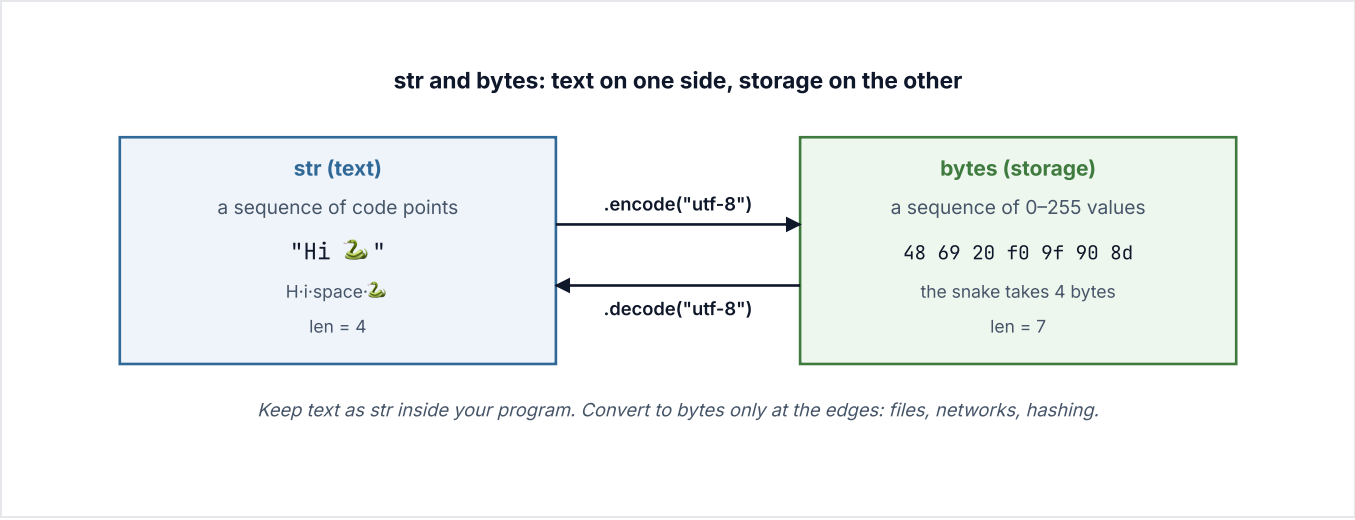


Figure 5.2. The bridge between text and storage. `encode` turns a string into bytes; `decode` turns bytes back into a string. Reading a file or a network message gives you bytes, which you decode; writing means encoding.

EXAMPLE 5.1 Encoding, decoding, and how the bytes differ

PYTHON

```python
s = "café 🐍"
len(s)                          # 6 characters

raw = s.encode("utf-8")         # turn text into bytes
type(raw)                       # <class 'bytes'>
len(raw)                        # 9 bytes: 'é' takes 2, '🐍' takes 4

raw.decode("utf-8")             # 'café 🐍', back to text

# Decoding with the WRONG encoding produces mojibake or an error:
raw.decode("latin-1")           # 'cafÃ© ð\x9f\x90\x8d', garbage
```

The last line is the bug that haunted text processing for decades: the bytes were correct, but they were interpreted under the wrong encoding, and the result is nonsense. The rule that prevents it is simple and absolute: always know the encoding of the bytes you are decoding, and when in doubt, assume UTF-8, because on the modern web it is almost always right.

→ FURTHER READING: UNICODE AND ENCODINGS

- Joel Spolsky: "The Absolute Minimum Every Developer Must Know About Unicode" *the classic essay*
- "The Absolute Minimum Every Software Developer Must Know About Unicode in 2023" *a modern update*
- UTF-8 *Wikipedia, with the encoding scheme*

5.3 How CPython Stores Strings

If a string is a sequence of code points, and code points range up to a million, does every character cost four bytes of memory? For most strings that would be wasteful, since a string of pure English needs only one byte per character. CPython solves this elegantly with a scheme introduced in version 3.3 and described in PEP 393, the *flexible string representation*. The interpreter looks at the largest code point in a string and picks the narrowest storage width that can hold it: one byte per character if every code point fits in 255, two bytes if every code point fits in 65,535, and four bytes only if some character truly needs it.

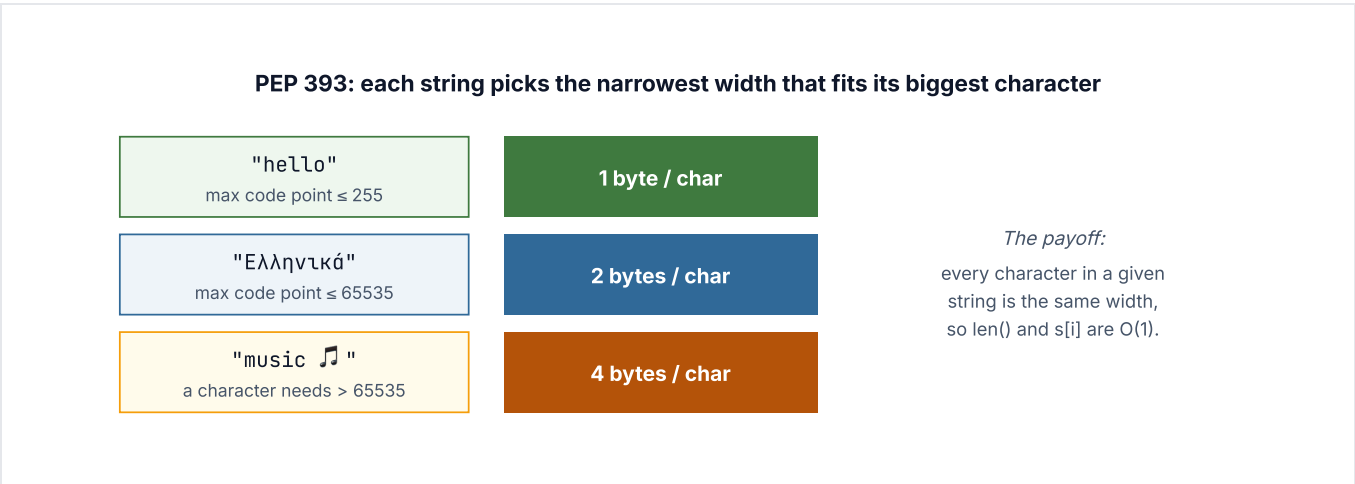


Figure 5.3. The flexible string representation. Because every character within one string occupies the same number of bytes, Python can find the i-th character by simple arithmetic, making indexing and length instant.

This design buys two things at once. It keeps the common case, ASCII and Latin text, as compact as the old byte strings were, so upgrading to proper Unicode cost almost no memory. And because every character in a given string is the same width, the interpreter can jump straight to the i-th character by multiplying, which means indexing a string and asking its length are both instant, no matter how long the string or how exotic its charac-

ters. Languages that store text as UTF-8 internally cannot promise that; they must walk the string to find a character by position. Python's choice trades a little memory for guaranteed fast indexing, and for most programs it is the right trade.

→ **FURTHER READING: STRING STORAGE**

- PEP 393: Flexible String Representation *the design, in full*

5.4 Writing String Literals

Python offers several ways to write a string in source code, each suited to a different need. The choice among them is mostly about avoiding the need to escape characters, which keeps text readable.

PYTHON

```python
a = 'single quotes'
b = "double quotes"            # identical; pick whichever avoids escaping
c = "she said \"hi\""           # escape the inner quotes
d = 'she said "hi"'             # cleaner: use the other quote

e = """A triple-quoted string
spans multiple lines
and keeps the line breaks."""

f = r"C:\Users\Ada\new"          # raw: backslashes are literal, not escapes

g = ("Python automatically joins "   # adjacent string literals
     "two literals with no operator "  # become one string
     "between them.")
```

The raw string, prefixed with `r`, deserves emphasis because it prevents a common class of error. Inside an ordinary string, a backslash begins an escape sequence: `\n` is a newline, `\t` a tab, `\\` a single backslash. This is a problem for Windows file paths and for regular expressions, both of which are full of backslashes. A raw string turns the escaping off, so `r"\n"` is a backslash followed by an `n`, two characters, not one newline. You will use raw strings constantly once you reach regular expressions in Section 5.10.

→ FURTHER READING: STRING LITERALS

○ String and bytes literals *the lexical rules, including every escape sequence*

5.5 Indexing, Slicing, and Immutability

A string is a sequence, so it supports the same indexing and slicing as any sequence, which we will meet again with lists in Chapter 8. Each character has an index from zero, negative indices count from the end, and a slice extracts a range using the notation `[start:stop:step]`, where `stop` is exclusive.

```python
s = "Pythonic"
s[0]        # 'P'
s[-1]       # 'c', last character
s[0:6]      # 'Python', indices 0 through 5
s[:6]       # 'Python', start defaults to 0
s[6:]       # 'ic'    , stop defaults to the end
s[::2]      # 'Pton'  , every second character
s[::-1]     # 'cinohtyP', reversed
```

The fact you must never forget is that strings are **immutable**, exactly as Chapter 3 described. You cannot change a character in place; an attempt raises a `TypeError`. Every operation that appears to modify a string in fact creates a new one. This is not a limitation to resent but a guarantee to rely on: because a string can never change underneath you, you can pass it to any function, use it as a dictionary key, and share it freely, knowing no one can alter it. The one place immutability bites is when you build a long string piece by piece in a loop, and Section 5.8 shows the right way to do that.

5.6 The String Methods

The string type carries roughly forty-seven methods. You do not need to memorise them; you need to know the two dozen that do the daily work and to recognise the shape of the rest so you can look them up. The table below collects the working set.

Method	Does	Example result

`.upper() .lower()`	change case	`"Hi".lower()` → `'hi'`
`.title() .capitalize()`	title / sentence case	`"war".capitalize()` → `'War'`
`.casefold()`	aggressive case fold for matching	`"ß".casefold()` → `'ss'`
`.strip() .lstrip() .rstrip()`	remove surrounding whitespace	`" hi ".strip()` → `'hi'`
`.split(sep)`	break into a list	`"a,b".split(",")` → `['a','b']`
`.join(seq)`	combine a list into a string	`",".join(['a','b'])` → `'a,b'`
`.replace(a, b)`	substitute text	`"cat".replace("c","b")` → `'bat'`
`.find(x) .index(x)`	locate a substring	`"hello".find("ll")` → `2`
`.count(x)`	count occurrences	`"banana".count("a")` → `3`
`.startswith() .endswith()`	test a prefix or suffix	`"a.csv".endswith(".csv")` → `True`
`.isdigit() .isalpha()`	test character classes	`"42".isdigit()` → `True`
`.zfill(n) .center(n)`	pad to a width	`"7".zfill(3)` → `'007'`

Two of these carry more weight than the rest and deserve a closer look, because together they handle a huge fraction of real text work: `split` takes a string apart into a list, and `join` puts a list back together into a string. They are inverses, and the asymmetry in how `join` is called (you call it on the separator, not on the list) trips up every beginner exactly once.

EXAMPLE 5.2 split and join, the workhorses

```python
line = "Ada,Lovelace,1815,London"
fields = line.split(",")
# ['Ada', 'Lovelace', '1815', 'London']

# join is called on the SEPARATOR, with the list as its argument:
" | ".join(fields)
# 'Ada | Lovelace | 1815 | London'

# split() with no argument splits on any run of whitespace, which is
# usually what you want for free-form text:
"  the   quick brown\tfox  ".split()
# ['the', 'quick', 'brown', 'fox']
```

Calling `join` on the separator reads strangely at first but is logical: the separator is the thing that knows how to glue, and it can glue any sequence handed to it. Write it wrong once, see the error, and you will remember it forever.

→ **FURTHER READING: STRING METHODS**

- String methods *the complete reference*

5.7 Formatting: Three Generations

Assembling a string from values, a name, a number, a date, is so common that Python has accumulated three ways to do it over its history. You should write all new code in the third, but you will read the other two in older code and must recognise them.

```python
name, score = "Ada", 91.5

# 1. Percent formatting (oldest, inherited from C). Still seen in logging.
"%s scored %.1f" % (name, score)        # 'Ada scored 91.5'

# 2. str.format (Python 2.6+). Verbose but still common.
"{} scored {:.1f}".format(name, score)   # 'Ada scored 91.5'
```

```
# 3. f-strings (Python 3.6+). The modern default. Use these.
f"{name} scored {score:.1f}"              # 'Ada scored 91.5'
```

The f-string, marked by an `f` before the opening quote, is the clearest of the three because the values appear inline, exactly where they will be in the output. Anything inside the braces is a full Python expression, evaluated on the spot, and after a colon you may add a *format specification* controlling width, alignment, precision, sign, and grouping. The specification is its own compact mini-language, and the most useful pieces of it fit in one example.

EXAMPLE 5.3 The f-string format specification

PYTHON

```
n = 1234567
pi = 3.14159265
name = "Ada"

f"{pi:.2f}"          # '3.14'      , two decimal places
f"{n:,}"             # '1,234,567' , thousands separators
f"{n:_}"             # '1_234_567' , underscore grouping
f"{0.875:.1%}"       # '87.5%'     , as a percentage
f"{255:#x}"          # '0xff'      , hexadecimal with prefix
f"{name:>10}"        # '       Ada', right-align in width 10
f"{name:^10}"        # '   Ada    ', centre
f"{name:*<10}"       # 'Ada*******', left-align, pad with *

# The self-documenting form (Python 3.8+), invaluable for debugging:
f"{score=}"          # 'score=91.5', prints the name and the value
```

The self-documenting form, an equals sign just before the closing brace, prints both the expression and its value. It is the fastest way to scatter informative debugging output through a program, and it is worth committing to memory now.

→ **FURTHER READING: FORMATTING**

- Formatted string literals *the f-string reference*
- fstring.help *a cheat sheet for the format spec*
- pyformat.info *format() and % side by side*

5.8 Building Strings Without Quadratic Slowdown

Because strings are immutable, a deceptively innocent loop can become catastrophically slow, and this is one of the few performance traps worth learning before you ever hit it. Consider building one long string by repeatedly adding to it with `+=`. Each `+=` cannot change the existing string, since strings are immutable, so it must create a *new* string by copying every character built so far and then appending the next piece. Building a string of n pieces this way copies roughly n-squared characters in total, which crawls to a halt as n grows.

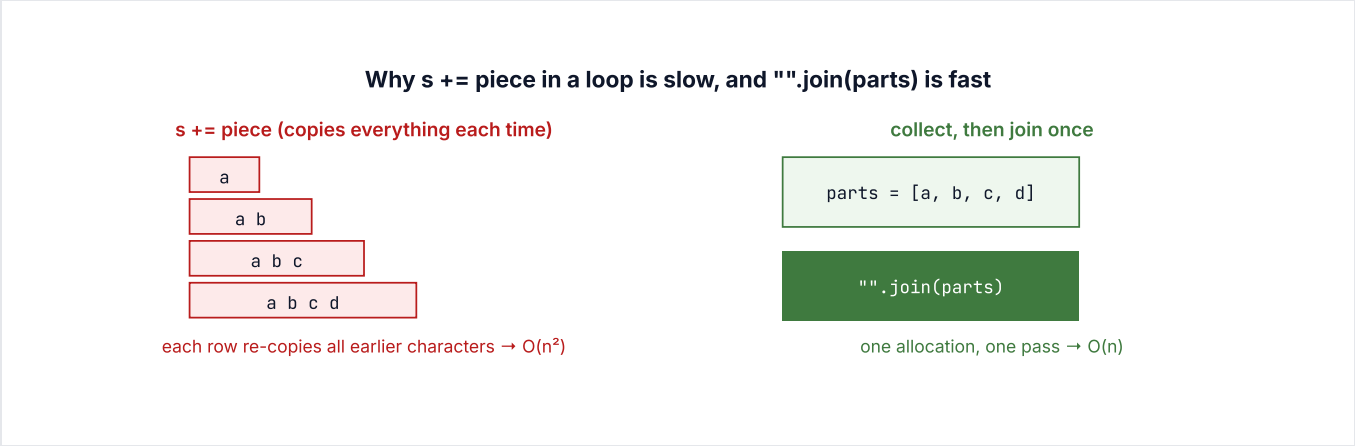


Figure 5.4. The string-building trap. Appending to an immutable string in a loop repeatedly copies the whole accumulated result. Collecting the pieces in a list and joining once at the end does the work in a single pass.

The idiomatic remedy is to collect the pieces in a list, which is mutable and appends cheaply, and to join them in a single operation at the end. The `join` method computes the total length once, allocates the result once, and copies each piece once, turning quadratic work into linear.

PYTHON

```
# Slow: O(n²), re-copies on every iteration
result = ""
for piece in pieces:
    result += piece

# Fast and idiomatic: O(n)
result = "".join(pieces)

# When the pieces are produced incrementally, io.StringIO is the tool:
import io
buffer = io.StringIO()
for piece in pieces:
```

```
    buffer.write(piece)
result = buffer.getvalue()
```

AN HONEST CAVEAT CPython contains a special optimisation that sometimes makes `s += piece` run in linear time when `s` has no other references. It is real but fragile: it does not apply on other implementations like PyPy, and it breaks the moment a second name points at the string. Do not rely on it. The `join` idiom is always fast and always clear, so simply make it a habit.

→ **FURTHER READING: BUILDING STRINGS**

- io.StringIO *Python docs*
- "Efficient String Concatenation in Python" *benchmarks of the approaches*

5.9 Bytes and Bytearray

Text is what your program reasons about; bytes are what crosses its boundaries. When you read a file, receive a network message, compute a cryptographic hash, or talk to hardware, you are handling raw bytes, and Python gives you the `bytes` type for exactly this. A `bytes` object is an immutable sequence of integers from 0 to 255, written with a `b` prefix, and its mutable sibling is `bytearray`.

PYTHON

```python
data = b"hello"
type(data)             # <class 'bytes'>
data[0]               # 104, an integer, not a character
data.decode("utf-8")   # 'hello', turn bytes back into text

# Hashing operates on bytes, not text, so you must encode first:
import hashlib
hashlib.sha256("password".encode("utf-8")).hexdigest()
# '5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8'
```

The discipline that keeps text handling sane is to convert at the boundaries and reason in text within. Decode bytes to `str` as soon as they enter your program, do all your work on strings, and encode back to bytes only when you write out. This is sometimes called

the "Unicode sandwich": bytes on the outside, text in the middle. Programs that mix the two carelessly are exactly the programs that produce mojibake.

→ FURTHER READING: BYTES

- Binary sequence types *bytes, bytearray, memoryview*
- Ned Batchelder: "Pragmatic Unicode" *the Unicode sandwich, explained*

5.10 A First Look at Regular Expressions

When the text you are searching has a *shape* rather than a fixed form, a phone number, an email address, a date, a log line, plain string methods are not enough, and you reach for the `re` module and its regular expressions. A regular expression is a small, dense pattern language: `\d` matches any digit, `\w` any word character, `.` any character at all, while `*`, `+`, and `?` control repetition. Patterns are almost always written as raw strings, because they are full of backslashes.

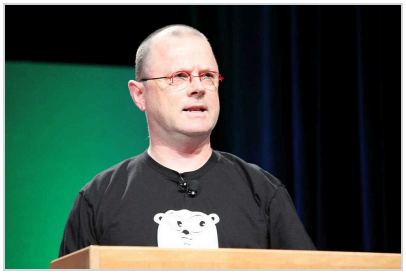


Figure 5.5. *Rob Pike, co-designer of UTF-8, photographed at a conference.*

EXAMPLE 5.4 The four operations you will use most

PYTHON

```python
import re
text = "Reach me at ada@example.com or bob@work.org by Friday."

# findall: every match, as a list
re.findall(r"[\w.]+@[\w.]+", text)
# ['ada@example.com', 'bob@work.org']

# search: the first match anywhere, or None
m = re.search(r"(\w+)@(\w+)", text)
m.group(0)        # 'ada@example', the whole match
m.group(1)        # 'ada'       , first parenthesised group

# sub: replace every match
re.sub(r"[\w.]+@[\w.]+", "[email hidden]", text)
# 'Reach me at [email hidden] or [email hidden] by Friday.'

# fullmatch: does the WHOLE string fit the pattern? (validation)
bool(re.fullmatch(r"\d{4}-\d{2}-\d{2}", "2026-05-27"))   # True
```

USE THEM SPARINGLY

Regular expressions are powerful and easy to abuse. If you are searching for fixed text, use `str` methods like `in`, `find`, or `replace`, which are clearer and faster. If your data is structured, such as JSON, HTML, or CSV, use a real parser, never a regular expression, because those formats are not regular and a regex will eventually meet input it mishandles. Reach for regular expressions when the shape is genuinely regular but variable: identifiers, phone numbers, log formats. A famous saying warns: faced with a problem, some people think "I will use regular expressions", and now they have two problems.

→ **FURTHER READING: REGULAR EXPRESSIONS**

- Python Regular Expression HOWTO *the official tutorial*
- regex101.com *an interactive tester that explains each part*
- regular-expressions.info *the deepest reference on the web*

§ § §

Exercises

EXERCISE 5.1 Characters versus bytes

For the string `s = "naïve 🦦"`, predict `len(s)` and `len(s.encode("utf-8"))` before running them. Explain why the two numbers differ, referring to the idea of code points and encoding.

EXERCISE 5.2 Round trip

Take the string `"Zürich"`, encode it to UTF-8 bytes, then decode it back. Then deliberately decode the same bytes with `"latin-1"` and observe the mojibake. Explain in one sentence what went wrong in the second case.

EXERCISE 5.3 Parse a record

Given the line `"Lovelace, Ada, 1815"`, use `split` and `strip` to produce the list `['Lovelace', 'Ada', '1815']` with no surrounding spaces on any field. Then reassemble it into `"Ada Lovelace (1815)"` using an f-string.

EXERCISE 5.4 The join puzzle

A beginner writes `["a", "b", "c"].join("-")` and gets an `AttributeError`. Explain why, and write the correct version. Why did Python's designers attach `join` to the string rather than to the list?

EXERCISE 5.5 Format a table

Given `rows = [("Ada", 91.5), ("Bo", 88.25), ("Caroline", 100.0)]`, print each row so the names are left-aligned in a width of 10 and the scores are right-aligned with one decimal place in a width of 6. Use f-strings with format specifications.

EXERCISE 5.6 Measure the slowdown

Using the `time` module, build a string of 100,000 characters two ways: with `+=` in a loop, and with `"".join(...)`. Compare the times. Explain the difference in terms of how many characters each approach copies in total.

EXERCISE 5.7 A palindrome, properly

Write a function that decides whether a string is a palindrome, ignoring case, spaces, and punctuation, so that `"A man, a plan, a canal: Panama"` returns `True`. Use string methods and slicing, not a loop. Which method removes the non-letters most cleanly?

EXERCISE 5.8 Regex versus method

You need to check whether a string ends in `.csv`, and separately to extract all numbers from a sentence. One of these should use a string method and one should use a regular expression. Decide which is which, write both, and justify your choice using the guidance in Section 5.10.

Solutions to Chapter 5 Exercises

Solution 5.1: Characters versus bytes

`len(s)` is 7 (n, a, ï, v, e, space, 🎲: seven code points). `len(s.encode("utf-8"))` is 11. Counting the bytes: n, a, v, e, and the space take one each (5 bytes), `ï` takes two, and the die emoji takes four, giving 5 + 2 + 4 = 11. The numbers differ because `len` on a string counts characters (code points), while `len` on the encoded bytes counts the storage those characters require, and non-ASCII characters require more than one byte each.

Solution 5.2: Round trip

```python
raw = "Zürich".encode("utf-8")
raw.decode("utf-8")     # 'Zürich', correct
```

```
raw.decode("latin-1")   # 'ZÃ¼rich', mojibake
```

The `ü` is encoded by UTF-8 as two bytes. Latin-1 interprets each byte as its own character, so those two bytes become two separate, wrong characters (`Ã¼`). The bytes were never corrupted; they were decoded under an encoding different from the one used to create them.

Solution 5.3: Parse a record

```python
line = "Lovelace, Ada, 1815"
parts = [p.strip() for p in line.split(",")]
# ['Lovelace', 'Ada', '1815']
last, first, year = parts
f"{first} {last} ({year})"   # 'Ada Lovelace (1815)'
```

Splitting on the comma leaves a leading space on the second and third fields, so each piece is passed through `strip` to remove it.

Solution 5.4: The join puzzle

Lists have no `join` method, so `["a","b","c"].join("-")` raises `AttributeError`. The correct form is `"-".join(["a", "b", "c"])`, which gives `'a-b-c'`. `join` belongs to the string because the separator is the thing doing the joining and because `join` can then glue *any* iterable of strings, not only lists; attaching it to the separator made it general.

Solution 5.5: Format a table

```python
rows = [("Ada", 91.5), ("Bo", 88.25), ("Caroline", 100.0)]
for name, score in rows:
    print(f"{name:<10}{score:>6.1f}")
```

```
Ada         91.5
Bo          88.2
Caroline   100.0
```

`{name:<10}` left-aligns the name in ten columns; `{score:>6.1f}` right-aligns the score in six columns with one decimal place. Note that 88.25 displays as 88.2 because of banker's rounding from Chapter 4.

Solution 5.6: Measure the slowdown

```python
import time
n = 100_000

t = time.perf_counter()
s = ""
for _ in range(n): s += "x"
print("+=", time.perf_counter() - t)

t = time.perf_counter()
s = "".join("x" for _ in range(n))
print("join", time.perf_counter() - t)
```

The `join` version is dramatically faster (often by a factor of hundreds, though CPython's special-case optimisation can narrow the gap on this exact pattern). In principle, `+=` copies all characters accumulated so far on every one of the n iterations, doing work proportional to n-squared, while `join` allocates the result once and copies each character once, doing work proportional to n.

Solution 5.7: A palindrome, properly

```python
def is_palindrome(s):
    cleaned = "".join(c.lower() for c in s if c.isalnum())
    return cleaned == cleaned[::-1]

is_palindrome("A man, a plan, a canal: Panama")   # True
```

The comprehension keeps only alphanumeric characters (using `isalnum`) and lowercases them, joining the survivors into a clean string. The palindrome test is then the elegant `cleaned == cleaned[::-1]`, comparing the string to its reverse via slicing.

Solution 5.8: Regex versus method

```python
# Fixed suffix: a string method is clearer and faster.
"data.csv".endswith(".csv")        # True

# Variable shape (numbers anywhere): a regular expression fits.
import re
re.findall(r"\d+", "order 17 shipped 2026, qty 3")   # ['17', '2026', '3']
```

Checking for a fixed ending is exactly what `endswith` is for, so a regular expression there would be needless complexity. Extracting numbers that may appear anywhere and have varying length is a genuine pattern-matching task, which is precisely where regular expressions earn their keep.

CHAPTER 6 · PART II · THE PRIMITIVES

Booleans & Truthiness

Two values, three operators, one protocol. The whole machinery of decision in Python rests on a remarkably small idea, formalised by a self-taught English schoolmaster in 1854 and quietly built into every modern computer ever since.

> *The respective interpretation of the symbols 0 and 1 in the system of logic are Nothing and Universe.*
>
> — George Boole, *An Investigation of the Laws of Thought*, 1854

There is a beautiful symmetry in the fact that the language of computation, born of vacuum tubes and silicon, rests on a mathematics that was published a hundred and fifty years before any computer ran. Before Babbage's Engine, before electronic switching, before the very word *algorithm* meant what we mean by it today, George Boole worked out, in a small house in Cork, the algebra by which true and false could be manipulated like numbers. Two states, three operations, a handful of laws. That was enough, as it turned out, to organise every decision a computer would ever make.

This chapter is short because the topic is small, but the topic is small in the way the alphabet is small: the rest of the language sits on top of it. We will look at Python's two boolean values and how they behave; the comparison operators that produce them; the logical operators that combine them, with a wrinkle no other chapter has prepared you for; the rule of *truthiness* that decides whether *any* value, not just a boolean, counts as true; the one-line conditional expression; the recent `match` statement that brings structural pattern matching to Python; and De Morgan's laws, which let you rewrite tangled negations into readable code.

6.1 The Two Values

Python has exactly two boolean values: `True` and `False`. Both are written capitalised, both are global names that refer to single, shared objects (no matter how many times you write `True` in a program, you are pointing at the very same object), and both belong to the type `bool`.

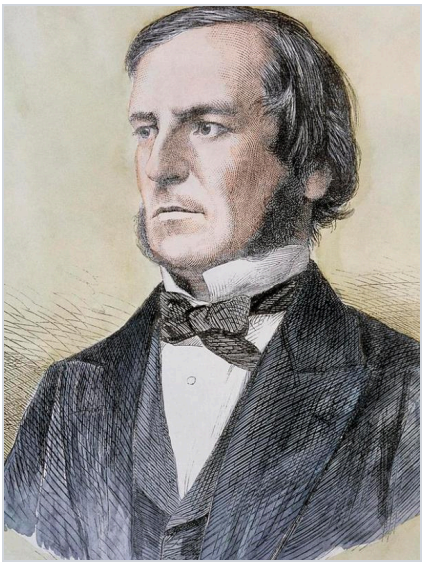


Figure 6.1. *George Boole (1815–1864), the self-taught English mathematician whose 1854 Laws of Thought founded the algebra of true and false.* George Boole

Now a fact that we met briefly in Chapter 4 and that bears restating, because the rest of the chapter quietly depends on it: `bool` is a subclass of `int`. `True` equals `1` in every arithmetic context, `False` equals `0`, and `isinstance(True, int)` is true. This is not a strange accident; it was decided deliberately when booleans were added to the language in 2003 by PEP 285, so that all the code that had been using 0 and 1 as truth values for the previous twelve years would keep working unchanged. The consequence is small but useful: you can `sum` a sequence of comparisons to count how many are true.

```python
scores = [82, 45, 91, 67, 30, 99]
sum(s ⩾ 60 for s in scores)   # 4, how many passed
```

DEF bool

The type of the two values `True` and `False`. It is a subclass of `int`: every `True` behaves as `1` and every `False` as `0` in arithmetic. Both values are singletons, so `True is True` and `False is False` are always trivially true, but you almost never need `is` with booleans; comparing for value is the same as comparing for identity, and the value test reads better.

→ FURTHER READING: BOOLEANS IN PYTHON

- PEP 285: Adding a Bool Type *why it subclasses int*
- Python docs: Boolean Type *the official reference*
- Boolean algebra *the mathematics behind it*

6.2 Comparison, and the Surprising Chain

Booleans are mostly produced by *comparing* two values. Python has six basic comparison operators, and they are the obvious ones with the obvious meanings.

Operator	Reads	Example	Result
`==`	equal to	`3 == 3`	True
`≠`	not equal to	`3 ≠ 4`	True
`<`	less than	`3 < 4`	True
`>`	greater than	`4 > 3`	True
`⩽`	less than or equal	`3 ⩽ 3`	True
`⩾`	greater than or equal	`3 ⩾ 3`	True

Under the hood, `a == b` calls `a.__eq__(b)`, which is what Chapter 3.8 promised: comparison is method dispatch on the type. The class of `a` decides what equality means, which is why `[1, 2] == [1, 2]` is true (lists compare their contents) and `Decimal("0.1") == 0.1` is false (a Decimal refuses to be equal to a binary-float approximation). The other operators reach for `__lt__`, `__gt__`, `__le__`, and `__ge__`; Python automatically tries the reflected method if the first operand returns `NotImplemented`.

Chained comparisons

Python lets you write comparisons in chains, and the meaning is exactly the mathematical one: `1 ⩽ x ⩽ 10` reads as "x is between 1 and 10 inclusive". The language is doing something subtle for you here. Under the hood, that single expression is the logical AND of two comparisons, with `x` evaluated *only once*: `(1 ⩽ x) and (x ⩽ 10)`. The short-

circuit behaviour of `and` (Section 6.3) means the second comparison is skipped entirely if the first is false.

EXAMPLE 6.1 A chain you would not write that way in C

```python
x = 5
1 ⩽ x ⩽ 10            # True, the natural way
1 ⩽ x and x ⩽ 10      # True, same meaning, more typing

# Chains do not need to be monotonic; they just AND each adjacent pair:
a = 2
1 < a > 0               # True, (1 < a) and (a > 0)
1 < a == 2              # True, (1 < a) and (a == 2)
```

In C, `1 < x < 10` means something quite different: `(1 < x) < 10`, which reduces to comparing a 0-or-1 result against 10, and is silently wrong. Python's chains are the kind of small detail that makes the language pleasant to read.

WORTH NOTING `≠` is normally just the inverse of `==`, but a class that defines `__eq__` without also defining `__ne__` gets the negation for free in Python 3. If you override one, you may safely leave the other; the dunder protocol fills in the missing piece. (This is a change from Python 2, which required both.)

→ **FURTHER READING: COMPARISONS**

- Comparisons *Python Language Reference, with the chained-comparison rule in full*
- Rich comparison methods *how `__lt__`, `__eq__`, et al. work*

6.3 `and`, `or`, `not`: Two Wrinkles

The three logical operators in Python read as English and behave, for boolean inputs, exactly as you would expect: `a and b` is true when both are, `a or b` is true when at least one is, and `not a` flips the value. Two things make them interesting, and both are worth knowing well because they shape idiomatic Python.

Short-circuit evaluation

The first wrinkle is that `and` and `or` are *short-circuit*. For `a and b`, Python evaluates `a` first; if it is falsy, the whole expression cannot be true, so `b` is not evaluated at all. For `a or b`, Python evaluates `a`; if it is truthy, the whole expression must be true, so `b` is skipped. This is more than a performance trick. It lets you guard one expression with another: `x is not None and len(x) > 0` never calls `len` when `x` is `None`, because the first half short-circuits.

They return one of the operands, not a boolean

The second wrinkle is more surprising, and once you see it, a whole shelf of Python idioms suddenly makes sense. `a and b` does *not* return `True` or `False`; it returns one of the actual operands. Specifically, `and` returns the first falsy value it finds, or the last value if all are truthy; `or` returns the first truthy value, or the last value if all are falsy.

```python
0 or "fallback"      # 'fallback' , 0 is falsy, so or returns the second operand
"Ada" or "Anon"     # 'Ada'      , first truthy value short-circuits
"Ada" and "Lovelace"  # 'Lovelace' , both truthy, and returns the last
0 and "never reached"   # 0         , first falsy short-circuits

# The classic default-value idiom:
display = user_input or "Guest"    # falls back when user_input is empty
```

`not` is the exception: it always returns a true boolean, `True` or `False`, never an operand. This asymmetry is occasionally a small frustration but more often a relief, because `not` is the operator you tend to use when you actually want a boolean.

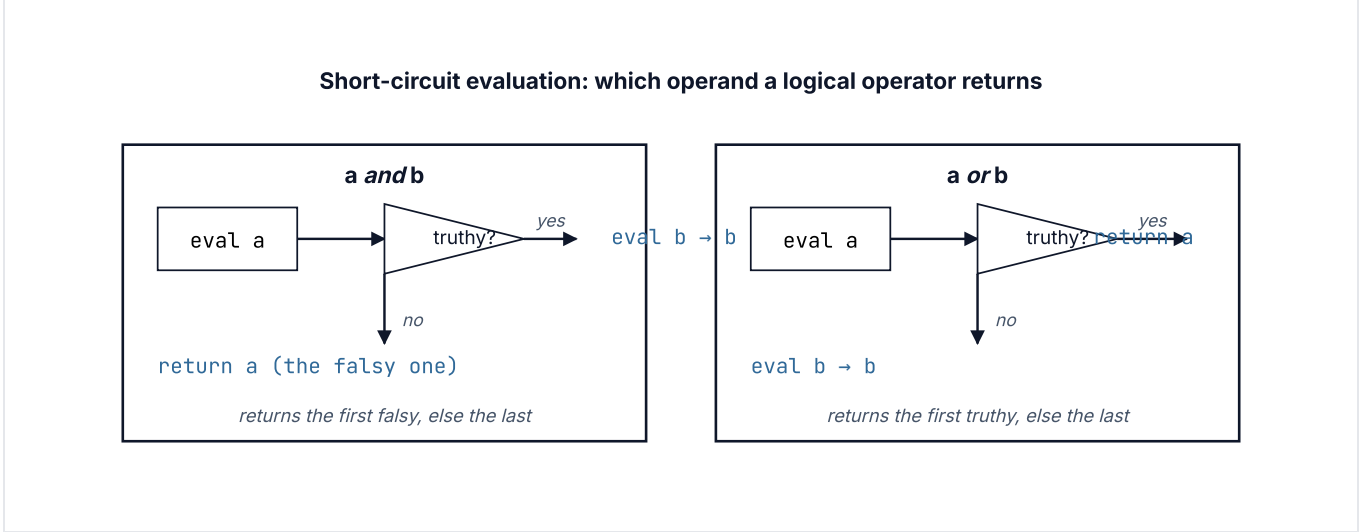


Figure 6.2. The two short-circuit operators, drawn as decision trees. The labels `a` and `b` are the operands themselves, not booleans derived from them, which is why the operators can *return* them.

EXAMPLE 6.2 A peek at the bytecode

The short-circuit is not a runtime decision; the compiler emits a conditional jump that skips the second operand entirely.

PYTHON

```python
import dis
def f(a, b): return a and b
dis.dis(f)
```

OUTPUT

```
  1   RESUME              0
      LOAD_FAST           a
      COPY                1
      POP_JUMP_IF_FALSE to RETURN_VALUE
      POP_TOP
      LOAD_FAST           b
      RETURN_VALUE
```

Read it: load `a`, keep a copy on the stack, jump straight to `RETURN_VALUE` if it is falsy (skipping `b` entirely), otherwise drop the copy and load `b` in its place. The compiler is doing the short-circuit; the interpreter never sees a choice.

→ **FURTHER READING: LOGICAL OPERATORS**

- Boolean operations *the precise short-circuit rules*
- When to use `and` / `or` as default selectors *a discussion of the idiom*

6.4 Truthiness, Revisited

Chapter 3.9 introduced the idea that every Python value can stand in for a boolean. This chapter is the right place to look at it again, because the rule shapes most of the conditions you will write. When Python tests a value for truth, it calls `__bool__()` on the value; if that is not defined, it falls back to `__len__()`, treating zero length as false; if neither is

defined, the value is truthy. The set of values that come out falsy is small and worth memorising:

- `False` and `None`
- `0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0)`
- `""` (the empty string)
- `[]`, `()`, `{}`, `set()`, `frozenset()` (empty containers)
- `range(0)` and any other empty sequence

Everything else is truthy. This rule is what makes `if names:` a complete way to say "if the list is not empty", and why `while line := file.readline():` works as the loop's natural termination. It is also where some of the most pernicious silent bugs live, because *truthiness asks about emptiness, not meaning*. The string `"False"` is a non-empty string, so it is truthy; a configuration switch read from a file as text needs an explicit comparison, not a truth test.

A SILENT BUG A program reads a setting as text and uses `if user_setting:` to decide whether it is on. The user writes `False` in the config. The string `"False"` is non-empty, so the branch runs and the setting is treated as *on*, the opposite of what was meant. Truth tests ask "is there anything here?", not "is this thing meaningful?". When you mean the second, compare explicitly: `if user_setting.lower() == "true":`.

EXAMPLE 6.3 A class that decides its own truth

PYTHON

```python
class Cart:
    def __init__(self):
        self.items = []
    def __bool__(self):
        return len(self.items) > 0

cart = Cart()
if not cart:
    print("Cart is empty")    # this runs
cart.items.append("apple")
if cart:
    print("Cart has items")   # this runs
```

OUTPUT

```
Cart is empty
Cart has items
```

By defining `__bool__`, a class participates in the same `if` grammar as the built-in containers. If we had defined `__len__` instead, the effect would be identical, because Python falls back to length when no `__bool__` exists.

→ **FURTHER READING: TRUTHINESS**

- Truth value testing *the canonical list of falsy values*
- `__bool__` and the length fallback *how the protocol works*

6.5 The Conditional Expression

Python has one in-line form of `if`, called the *conditional expression*, that picks between two values:

PYTHON

```python
status = "adult" if age ⩾ 18 else "minor"
```

The shape is `X if C else Y`. Python evaluates the condition `C` first; if true, the whole expression yields `X`, otherwise `Y`. Like `and` and `or`, only one of the two value-expressions is evaluated, so the unused side cannot raise an error. The order is sometimes surprising on first reading: the value comes before the condition, not after. This is deliberate; it reads more naturally for the common case of "this value, when this is so, otherwise this other one".

Use it sparingly. The form is a small joy when it expresses a binary choice on one line. Nest it, or stretch it across two lines, and it becomes harder to read than the ordinary `if` block it tried to replace.

> **STYLE** If the entire expression does not fit comfortably on one line, write the ordinary block instead. Two levels of nested conditional expressions (`a if c1 else b if c2 else d`) are legal but discouraged; the linear ordering does not match how people parse English, and the reader has to mentally re-bracket them.

→ **FURTHER READING: CONDITIONAL EXPRESSIONS**

- PEP 308: Conditional Expressions *why the unusual order*

6.6 Structural Pattern Matching: `match`

Released with Python 3.10 in October 2021, the `match` statement is the language's newest piece of control flow, and one of its most powerful. Pattern matching is not the same thing as a `switch` statement from C or Java, although a `switch` is roughly the simplest thing you can do with it. The full feature is closer to the destructuring of Haskell, Scala, or Rust: a `match` case is not just a value to compare against, it is a *shape* to compare against, and successfully matching a shape binds names to the parts.

The minimum example looks like a switch:

PYTHON

```
def classify(status):
    match status:
        case 200:
            return "OK"
        case 301 | 302:
```

```
            return "redirect"
        case 404:
            return "not found"
        case _:
            return "unknown"
```

The interesting part begins when the patterns describe shape:

EXAMPLE 6.4 Destructuring with `match`

```python
def describe(point):
    match point:
        case (0, 0):
            return "origin"
        case (0, y):                        # captures the second component a
            return f"on the y-axis at {y}"
        case (x, 0):
            return f"on the x-axis at {x}"
        case (x, y) if x == y:            # a guard: extra condition
            return f"diagonal at {x}"
        case (x, y):                        # anything else, with both compon
            return f"point ({x}, {y})"
        case _:
            return "not a point"

print(describe((0, 5)))   # on the y-axis at 5
print(describe((3, 3)))   # diagonal at 3
print(describe((7, 2)))   # point (7, 2)
```

Inside a `case`, a bare name (`x`, `y`) is a *capture pattern*: it matches anything and binds the matched value to that name. A literal (`0`) matches only equality. A guard, the `if` clause, adds an extra condition that the pattern alone cannot express. And the leading underscore is the *wildcard*, matching anything without binding a name.

The same patterns work for lists, dictionaries, and instances of your own classes. A dictionary pattern `{"name": n, "age": a}` succeeds when the subject is a mapping with at least those keys, binding their values. A class pattern `Point(x=0, y=y)` succeeds when the subject is a `Point` instance with `x` equal to zero, binding the `y` attribute. The feature is large enough that PEP 636, the official tutorial, is the best place to learn it properly.

Used well, it replaces long chains of `isinstance` tests with code that reads as the data does.

ONE TRAP A name that is just a bare lowercase identifier always means *capture*, not *compare*. To match against the value of a constant, the constant has to be *dotted*: `case Color.RED:`, not `case RED:`, the latter would simply rebind `RED` to whatever the subject was. The rule is documented in PEP 634 and surprises everyone exactly once.

→ **FURTHER READING: PATTERN MATCHING**

- PEP 636: Structural Pattern Matching Tutorial *the most readable introduction*
- PEP 634: Specification *the formal grammar*
- PEP 635: Motivation and Rationale *why it is structural, not a switch*

6.7 De Morgan's Laws and the Readable Negative

The last topic in this short chapter is not a feature of Python but a fact of logic, valuable because it lets you rewrite cluttered conditions into clean ones. Augustus De Morgan, who corresponded with Boole and shared his interests, observed in the 1840s that negation distributes over `and` and `or` by swapping them. In Python's notation:

```python
not (a and b)   # is the same as   (not a) or  (not
not (a or  b)   # is the same as   (not a) and (not
```

The rule rarely matters in arithmetic; it matters in code, because negated conditions are harder to read than positive ones, and a needlessly negated *combination* of conditions is harder still. When you find yourself writing `if not (x > 5 and x < 10):`, De Morgan tells you it is the same as `if x ⩽ 5 or x ⩾ 10:`, which says what it means.



Figure 6.3. *Augustus De Morgan (1806–1871), Boole's contemporary and friend, who codified the duality between conjunction and disjunction now named for him.* Augustus De Morgan

EXAMPLE 6.5 A condition rewritten three times

```python
# Original, awkward: a double negative
if not (user.is_admin or user.has_token):
    refuse()

# De Morgan: distribute the not over the or, flipping it to and
if not user.is_admin and not user.has_token:
    refuse()

# Better still: flip the test and the action together
if user.is_admin or user.has_token:
    allow()
else:
    refuse()
```

All three forms are logically identical. The third reads in the positive, with the negation pushed onto a single keyword (`else`) rather than smeared across several. This is the kind of rewriting that earns its place in code review.

→ **FURTHER READING: DE MORGAN AND BEYOND**

- De Morgan's laws *the broader mathematical context*
- Replace nested conditional with guard clauses *a refactoring that benefits from De Morgan*

§ § §

Exercises

EXERCISE 6.1 Counting with booleans

Given `scores = [82, 45, 91, 67, 30, 99, 58]`, write a single expression using `sum` that counts how many scores are at least 60. Explain why this works in terms of the `bool` / `int` relationship.

EXERCISE 6.2 A chained comparison

Rewrite each of these into a single chained comparison, then explain what each one is testing: (a) `a ⩾ 1 and a ⩽ 100`; (b) `a > b and b > c`; (c) `x == y and y == z`.

EXERCISE 6.3 Predict the operand

Without running them, predict the value *and the type* of each expression: (a) `0 or "fallback"`; (b) `"Ada" and "Lovelace"`; (c) `[] or [1, 2]`; (d) `not "Ada"`; (e) `None or 0 or False or "default"`.

EXERCISE 6.4 A safe length check

Write a single condition that is true if `x` is a non-empty string. Why does relying on `len(x)` alone risk an error here, and how does `and`'s short-circuit help?

EXERCISE 6.5 The "False" trap

Explain in one sentence why `if "False":` runs its body, and write the correct test for a configuration string whose value should be treated as "off" when it equals `"false"` (in any case).

EXERCISE 6.6 Pattern matching

Write a function `describe(shape)` that uses `match` to recognise four kinds of value: a 2-element tuple it calls a "point", a 3-element tuple it calls a "3D point", a dict with a `"name"` key it greets by name, and anything else it labels `"unknown"`.

EXERCISE 6.7 De Morgan rewrite

Rewrite each into a positive, easier-to-read form: (a) `not (x > 0 and x < 10)`; (b) `not (user.is_admin or user.is_owner)`; (c) `not (a == b)`.

EXERCISE 6.8 A history question

Python did not always have a `bool` type. Until Python 2.3 (released in 2003), the values 0 and 1 were used in its place. Read the abstract of PEP 285 in three minutes and write one paragraph in your own words explaining why a separate `bool` type was added *and* why it was made a subclass of `int`.

Solutions to Chapter 6 Exercises

Solution 6.1: Counting with booleans

```python
scores = [82, 45, 91, 67, 30, 99, 58]
sum(s >= 60 for s in scores)   # 4
```

Each comparison `s >= 60` produces a `True` or `False`. Because `bool` is a subclass of `int`, those values are exactly `1` and `0` when used in arithmetic, so summing the generator counts the number of true comparisons. The values 82, 91, 67, and 99 pass, giving 4.

Solution 6.2: Chained comparison

(a) `1 <= a <= 100`, "a is between 1 and 100 inclusive". (b) `a > b > c`, "the three are in strictly decreasing order". (c) `x == y == z`, "all three are equal". In each case Python evaluates the middle expression once and AND-s the adjacent comparisons, short-circuiting if any fails.

Solution 6.3: Predict the operand

(a) `'fallback'`, a `str`: 0 is falsy, so `or` returns the second operand. (b) `'Lovelace'`, a `str`: both are truthy, so `and` returns the last. (c) `[1, 2]`, a `list`: `[]` is falsy. (d) `False`, a `bool`: `not` always returns a boolean, never an operand. (e) `'default'`, a `str`: `None`, `0`, and `False` are all falsy, so `or` walks past them and lands on the first truthy value.

Solution 6.4: A safe length check

```python
isinstance(x, str) and len(x) > 0
# or simply: isinstance(x, str) and x       (truthiness gives len > 0 for free)
```

Calling `len` on something that has no length (such as `None` or an integer) raises `TypeError`. The `and` short-circuits: if the first operand is false, the second is never evaluated, so the type check protects the length check from blowing up on the wrong sort of input.

Solution 6.5: The "False" trap

The literal `"False"` is a non-empty string, and any non-empty string is truthy, so `if "False":` runs its body. The correct test reads the value explicitly: `if setting.lower() == "false":` or, more robustly, a small parser that accepts `"false"`, `"0"`, and `"off"`. Truthiness is a question about emptiness, never about content.

Solution 6.6: Pattern matching

```python
def describe(shape):
    match shape:
        case (x, y):
            return f"point ({x}, {y})"
        case (x, y, z):
            return f"3D point ({x}, {y}, {z})"
        case {"name": name}:
            return f"hello, {name}"
        case _:
            return "unknown"
```

The tuple patterns match by length; the mapping pattern matches any dict that has at least a `"name"` key and binds its value; the wildcard catches everything else. Note that the mapping pattern would still match if extra keys were present, which is usually what you want.

Solution 6.7: De Morgan rewrite

(a) `x ⩽ 0 or x ⩾ 10`, "x is outside the open interval (0, 10)". (b) `not user.is_admin and not user.is_owner`, or even better the positive form `user.is_admin or user.is_owner` with the branches swapped. (c) `a ≠ b`. The general rule: a not over an

and becomes an or of the nots; a not over an or becomes an and of the nots; and the cleanest result usually swaps the if and else rather than carrying the negation along.

Solution 6.8: A history question

PEP 285, by Guido van Rossum in 2002, argued that Python should have a real boolean type to make the language's intent clearer and to give the values `True` and `False` proper names that `repr` could print. But Python had been using `0` and `1` for twelve years; any code that did so had to keep working. Making `bool` a subclass of `int` resolved the tension at a stroke: every `True` still equals `1` in arithmetic, every old test that summed truth values still works, but the new type prints distinctively, can be checked with `isinstance`, and signals to a reader that the value is logical rather than numeric. It is one of the cleaner compromises in the language's history.

CHAPTER 7 · PART II · THE PRIMITIVES

None & the Sentinel

A whole chapter for one value, because that one value carries surprising weight: it stands for absence, it powers the most common default-argument idiom, it is the source of more silent bugs than any other single feature, and its history reaches back to a billion-dollar regret.

> *I call it my billion-dollar mistake. It was the invention of the null reference in 1965. At that time, I was designing the first comprehensive type system for references in an object-oriented language. My goal was to ensure that all use of references should be absolutely safe, with checking performed automatically by the compiler. But I could not resist the temptation to put in a null reference, simply because it was so easy to implement.*
>
> — Tony Hoare, QCon London 2009

Every language that lets you say "there is no value here" has to decide how. Some pick a special pointer that the programmer must everywhere remember to check, and pay the cost in null-pointer crashes for decades after. Some refuse the idea entirely and force you to wrap every absent thing in an explicit container. Python takes a middle path: it has a single, well-known object called `None` that stands for the absence of a value, hands it to you wherever a function has nothing useful to return, and gives you a small set of idioms for handling it well. Used correctly, it is one of the most pleasant null-equivalents in any language. Used carelessly, it produces exactly the bug class Hoare named for, with a softer landing.

This is a short chapter on a small subject, but the subject is load-bearing. Almost every function you write will need to think about `None` at least once: as a default argument, as a result for "nothing found", as a sentinel that distinguishes "no value given" from "the value given was None". We will cover all three, plus the type-hint conventions that pin

`None` down statically, and a quick look at the family of "almost-None" values (`nan`, `inf`, empty strings, empty lists) that beginners sometimes conflate.

7.1 The Idea of Absence

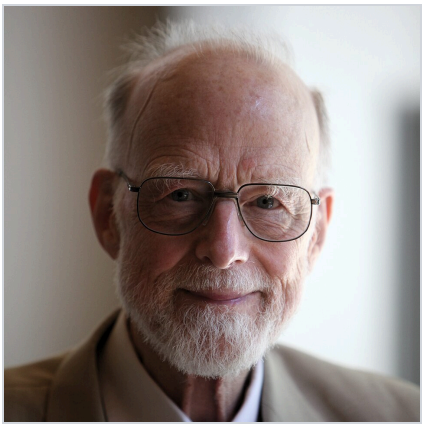


Figure 7.1. *Sir Tony Hoare (born 1934), who in 1965 added the null reference to ALGOL W and forty-four years later called it the billion-dollar mistake.* Tony Hoare

The number zero is not the absence of a number; it is a number with the value zero. The empty string is not the absence of text; it is a string with no characters. To express the genuine absence of a value, distinct from any specific value, you need a separate thing, and Python's separate thing is called `None`. It is what a function returns when it has nothing meaningful to return. It is what a database query gives back when there was no matching row. It is what a default-valued parameter holds when the caller did not supply one.

Why such an idea needs to exist at all is clearer after a moment with a counter-example. Suppose a function looks up a user by id and you decide that the "not found" answer should be the number zero. That works until someone valid is assigned id zero. Then you cannot tell whether the user is missing or simply has id zero, and the system has to be patched. The whole point of `None` is to be reliably outside every other value's domain: no real number is `None`, no real string is `None`, no real user is `None`. The price of this clarity is small but real: you must remember to check for `None` before treating a result as if it were the type you expected.

DEF **None**

The unique instance of the type `NoneType`, used to represent the absence of a value. There is only one `None` object in any running Python program; every `None` you write refers to the same one. It is falsy, hashable, and immutable, which makes it a fine default value and a fine dictionary key. It has no methods of interest beyond what every object has.

→ FURTHER READING: THE IDEA OF NULL

- Tony Hoare's "Null References: The Billion Dollar Mistake" *the full 2009 talk*
- Nullable type *how other languages address the same problem*

7.2 NoneType: One Type, One Value

`None` is unusual among Python's built-in values for being a true singleton: its type, `NoneType`, has exactly one instance, and the interpreter guarantees you will never see another. Attempting to create one fails by design, and the type cannot be subclassed.

```python
type(None)             # <class 'NoneType'>
NoneType = type(None)
NoneType()              # returns None, the same None as before
NoneType() is None     # True, there is no second instance

id(None) == id(None)   # True, every reference points at the one object
```

This uniqueness is what makes the `is None` check (Section 7.3) both fast and reliable. There is also a quirk worth knowing about how `None` behaves with the rest of the language. It is *falsy*, so `bool(None)` is `False` and `None` in an `if` condition takes the false branch. But it cannot be ordered against anything: comparing `None` with `<` or `>` to a number raises `TypeError`, a change Python 3 made deliberately, because comparing apples to absence had no sensible answer in Python 2 either, and the silent "is less than everything" rule it used was a frequent source of bugs.

```python
bool(None)        # False, None is falsy
None == 0          # False, None equals nothing but itself
None == None       # True
None < 5           # TypeError: '<' not supported between 'NoneType' and 'int'
```

A REPL QUIRK

When you type an expression into the REPL, the interpreter prints the value's `repr`. The one exception is when the value is `None`: the REPL hides it, to keep the screen quiet for the many functions that return `None` on purpose. So `print("hi")` shows you only the printed output, not also a separate `None` line; the `None` that `print` actually returned is silently swallowed. This is help-

ful but occasionally confusing: *some* calls that look like they returned nothing did return `None`.

→ **FURTHER READING: NONETYPE**

- None, Python docs *the official reference*
- Ordering comparisons in Python 3 *why None < 5 now raises*

7.3 The `is None` Idiom

To check whether a value is `None`, write `x is None`, never `x == None`. The recommendation appears in PEP 8 and is one of the few style rules with a real technical reason behind it.

Three reasons matter. First, **speed**: `is` compares object identities, which in CPython is a single pointer comparison; `==` calls `__eq__`, which is a method dispatch. The difference is microscopic per call but real. Second, **correctness**: a perverse class could define `__eq__` to return `True` when compared with `None`, so `x == None` might lie. Nothing can fake identity with the one true `None`; `is None` always gives the right answer. Third, **style**: the rest of the Python ecosystem uses `is None`, and consistency is its own value.

EXAMPLE 7.1 Two writes of the same check

```python
# Idiomatic
if result is None:
    handle_missing()

# Works, but slower and stylistically wrong
if result == None:
    handle_missing()

# Negated form: always 'is not None', not 'not ... is None'
if result is not None:
    use(result)
```

The phrasing `x is not None` reads as English and is the canonical positive form. Avoid `not x is None`, which is parsed correctly (`not (x is None)`) but reads ambiguously to a human.

A FREQUENT MISTAKE Beginners sometimes write `if x:` intending to check whether `x` is `None`. This works some of the time, because `None` is falsy, but it also treats `0`, `""`, `[]`, and every other falsy value as if they were missing. A function whose valid result might legitimately be `0` must distinguish "got nothing" from "got zero" by writing `if x is None:` explicitly.

→ **FURTHER READING: THE IS NONE RULE**

- PEP 8: Programming Recommendations *the official styling rule*

7.4 None as the Default Sentinel

The single most useful place for `None` in everyday Python is the default-argument idiom. We met the underlying trap in Chapter 3 and again briefly when introducing functions: a mutable default value (`def f(items=[])`) is created once and shared across every call, with the surprising consequence that the "default" accumulates state across calls. The remedy is to use `None` as the default and build the real default inside the function.

EXAMPLE 7.2 The mutable-default fix, in three steps

```python
# Wrong: the empty list is created ONCE, when the function is defined.
def append_item(item, items=[]):
    items.append(item)
    return items

append_item(1)        # [1]
append_item(2)        # [1, 2], the default REMEMBERED the previous call!

# Right: None signals "no list given", then build a fresh one each call.
def append_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items

append_item(1)        # [1]
append_item(2)        # [2], fresh empty list each time
```

This pattern is so common that you should consider it a reflex: whenever a default value would naturally be a mutable object (a list, a dict, a set, an instance of your own class), the default goes into the signature as `None` and the real value is constructed in the body.

`None` is the right sentinel here for several reasons. It is unique, so the test `if x is None` cannot be confused with a real value. It is immutable, so it cannot accumulate state. It is well-known, so a reader of the function knows immediately what is going on. And it has no methods worth calling by accident.

→ **FURTHER READING: DEFAULT ARGUMENTS**

- The mutable-default-argument trap *Python Guide gotchas*
- "Python Mutable Defaults Are the Source of All Evil" *a memorable explanation*

7.5 When `None` Is a Valid Value: Custom Sentinels

The default-argument idiom of Section 7.4 works perfectly until the function is one that *legitimately* accepts `None` as a value. Imagine a configuration getter that should distinguish three cases: "the caller gave us no default" (return whatever is stored, even `None`), "the caller passed a default of `None`" (return `None` when missing), and "the caller passed a real default" (return that). The first and second cases look identical to `if x is None`, because in the first case the parameter is also `None`. We need a sentinel that is not `None`.

The standard pattern is to create a unique private object that no caller could ever pass by accident.

EXAMPLE 7.3 A custom sentinel for "argument not provided"

PYTHON

```python
_MISSING = object()   # a plain object, unique by identity

def get_config(key, default=_MISSING):
    if key in config_store:
        return config_store[key]
    if default is _MISSING:
        raise KeyError(key)         # caller gave no default → fail loudly
    return default                  # caller's default, which may be None

get_config("theme")              # KeyError if missing
get_config("theme", default=None)  # None if missing, but distinct from "not
```

An empty `object()` instance is the simplest sentinel you can make. Each call to `object()` produces a fresh instance with its own identity, and no caller, anywhere, can ever pass it by accident, because they have no name for it. The `is _MISSING` check then cleanly separates the two cases.

This pattern appears throughout the standard library. The `dict.pop` method takes an optional default with exactly this design: when no default is supplied, popping a missing key raises `KeyError`; when a default is supplied (which may be `None`), the method returns it. The `dataclasses` module exports a `MISSING` sentinel for the same purpose. And `argparse` uses a `SUPPRESS` sentinel to mean "do not even set this attribute".

→ FURTHER READING: SENTINELS

- PEP 661: Sentinel Values *a proposal to standardise the pattern*
- dataclasses.MISSING *a real-world sentinel in the standard library*

7.6 `None` in Type Hints: `Optional` and `T | None`

When you annotate a function's signature, declaring that a parameter or return value *can* be `None` is part of the type. The `typing` module gives you two equivalent ways to say it.

```python
from typing import Optional

# Older style (Python 3.5+):
def find_user(name: str) -> Optional[User]:
    ...

# Modern style (Python 3.10+, PEP 604):
def find_user(name: str) -> User | None:
    ...
```

`Optional[T]` means exactly `Union[T, None]`, which since Python 3.10 can be written as `T | None`. The two are interchangeable; `T | None` is the modern preference because it does not need an import. Use the annotation wherever a function might legitimately return `None` (a lookup that may not find anything), or wherever an argument's default is `None`. A type checker like `mypy` or `pyright` will then refuse to let you call a method on the result without first checking, which is exactly the discipline that converts Hoare's billion-dollar mistake into a compiler-level annoyance.

```python
def find_user(name: str) -> User | None:
    ...

u = find_user("Ada")
u.greet()                # mypy error: u may be None
if u is not None:
    u.greet()            # OK: inside this branch mypy narrows the type to User
```

→ FURTHER READING: OPTIONAL IN TYPING

- PEP 604: Allow writing union types as X | Y *the modern syntax*
- mypy: Optional types and the None type *how the checker narrows types after is-None tests*

7.7 Almost-`None`: `nan`, `inf`, the Empty Container

Several other values look superficially like absence and are sometimes used as if they meant the same thing. They do not, and conflating them is the source of small bugs that can be very hard to track down. A short reference table:

Value	What it really means	Use for
`None`	no value	a missing return, an unset argument, the absent
`float("nan")`	not-a-number, the result of `0/0` or `inf - inf`	signalling a calculation that has no real-number answer
`float("inf")`	infinity, larger than every other float	upper bounds in search problems
`0`	the number zero	a real count or measurement that happens to be zero
`""`	a string with no characters	a real piece of text that happens to be empty
`[]`	a list with no items	a real collection that happens to be empty

Two pairs deserve a closer look. `None` and `nan` are both used for "I do not have a usable answer", but their semantics differ: `None` is the universal absence, while `nan` is specifically the floating-point sentinel for an undefined numeric result, and it propagates through arithmetic (any operation involving `nan` returns `nan`). `None` and `[]` are both falsy, but they mean different things in an API: a function that returns `[]` is telling you "here is the empty result"; one that returns `None` is telling you "I have no result at all".

Code that treats the two as interchangeable will eventually meet a caller who needed the distinction.

EXAMPLE 7.4 nan is unequal to itself

```python
import math
math.nan == math.nan        # False, yes, really
math.nan is math.nan        # True, but only because there is one nan object
math.isnan(math.nan)        # True, the correct way to test for nan

None == None                # True, None behaves the way you expect
```

`nan`'s refusal to equal itself is mandated by IEEE 754 and is not Python's fault; every modern language inherits the rule. The practical consequence is that you cannot deduplicate or sort a list containing `nan` reliably, and a list that might hold both real numbers and absences is usually better expressed with `None` than with `nan`.

→ **FURTHER READING: ALMOST-NONE**

- math.nan, math.inf, math.isnan *how Python exposes the IEEE-754 specials*
- pandas: working with missing data *a real-world library that distinguishes None, NaN, and NaT*

§ § §

Exercises

EXERCISE 7.1 The unique None

Show, by experiment in a REPL, that there is only ever one `None` object in a running program. Use `id()`, `is`, and `type(None)()` in your answer.

EXERCISE 7.2 is None versus == None

Write a class whose `__eq__` method always returns `True`, and demonstrate that `your_object == None` reports `True` while `your_object is None` correctly reports `False`. Explain in one sentence why this is the reason PEP 8 recommends `is None`.

EXERCISE 7.3 The mutable-default trap

The function below has a famous bug. Run it three times, predict and verify the output, then fix it using the `None` idiom from Section 7.4.

```python
def add_tag(tag, tags=[]):
    tags.append(tag)
    return tags
```

EXERCISE 7.4 Distinguishing "not given" from None

Write a function `get(d, key, default=...)` that returns `d[key]` if the key is present, returns the user's `default` if one was given (even when it is `None`), and raises `KeyError` if neither applies. You must distinguish "default not given" from "default given as `None`". Use the sentinel pattern.

EXERCISE 7.5 Type-narrowed access

Given a function annotated `def first(items: list[int]) -> int | None`, write a caller that prints the first item if there is one. Explain, with reference to a type checker, why writing `print(first(xs).bit_length())` directly would be a static error.

EXERCISE 7.6 None versus nan

Build a small list `[1.0, None, 2.0, float("nan"), 3.0]`. (a) Filter out the missing values to keep only real numbers. Take care that the obvious `x is not None` check leaves `nan` in. (b) Write the proper filter that removes both `None` and `nan`.

EXERCISE 7.7 The silent return

What does `print(print("hello"))` output, and why? Trace the two values involved (the printed text and the returned value), and connect the result to the REPL quirk discussed in Section 7.2.

EXERCISE 7.8 A small library question

Look up the documentation of `dict.get(key, default)`. What does it return when the key is missing and no default is given? What if the key is present and its value *is* `None`? Explain why this means you cannot use `dict.get` alone to distinguish "key missing" from "key present, value is None".

Solutions to Chapter 7 Exercises

Solution 7.1: The unique None

```python
id(None) == id(None)        # True
a, b = None, None
a is b                      # True, same object
NoneType = type(None)
NoneType() is None          # True, even constructing one returns the same sing
```

The interpreter guarantees a single instance. Two assignments and a call to `NoneType()` all return references to the same object, and that fact is the basis for the `is None` idiom.

Solution 7.2: is None versus == None

```python
class Lying:
    def __eq__(self, other):
        return True

x = Lying()
print(x == None)    # True, the __eq__ method lied
print(x is None)    # False, identity cannot be faked
```

A class can override `==` to return anything, so `x == None` is only as reliable as the implementation behind it. `is None` compares object identities, which no Python code can lie about, which is why it is the only correct check.

Solution 7.3: The mutable-default trap

```python
# Bug demonstration:
add_tag("a")     # ['a']
add_tag("b")     # ['a', 'b'], the "default" remembered the previous call
add_tag("c")     # ['a', 'b', 'c']

# Fix:
def add_tag(tag, tags=None):
    if tags is None:
        tags = []
    tags.append(tag)
    return tags
```

The mutable list in the original signature is created exactly once, when `def` runs, and shared across every call. Substituting `None` as the signal-only default forces each call to build a fresh list.

Solution 7.4: Distinguishing "not given" from None

```python
_MISSING = object()

def get(d, key, default=_MISSING):
    if key in d:
        return d[key]
    if default is _MISSING:
        raise KeyError(key)
    return default

get({"a": 1}, "a")              # 1
get({"a": 1}, "b")              # KeyError
get({"a": 1}, "b", default=None) # None, but distinct from "not given"
```

`_MISSING` is a fresh, private object whose identity no caller can ever match by accident. The `is _MISSING` test separates "no default passed" from "default passed and it happens to be None", which the plain `default is None` test cannot do.

Solution 7.5: Type-narrowed access

```python
x = first(xs)
if x is not None:
    print(x)
```

The annotated return type is `int | None`, so the static type of `first(xs)` at the call site is the union. Calling `.bit_length()` directly would fail under a type checker because `None` has no `bit_length` method. After `if x is not None:`, the checker narrows the type inside the branch to `int` alone, so calls become legal. This is precisely the discipline Hoare wished he had imposed in 1965.

Solution 7.6: None versus nan

```python
import math
xs = [1.0, None, 2.0, float("nan"), 3.0]

# (a) Naive filter, leaves nan in:
[x for x in xs if x is not None]
# [1.0, 2.0, nan, 3.0], nan is not None, so it passed through

# (b) Correct filter, drops both:
[x for x in xs if x is not None and not math.isnan(x)]
# [1.0, 2.0, 3.0]
```

`nan` is a perfectly real `float` object; only its value is special. The `is not None` check has nothing to grab onto with `nan`, so it lets it through, and the explicit `math.isnan` test is required. Note also the order: thanks to `and`'s short-circuit, the `isnan` call is only attempted on values that are not `None`, which is exactly what we want (calling `math.isnan(None)` would raise `TypeError`).

Solution 7.7: The silent return

`print("hello")` writes `hello` to standard output and returns `None`. The outer `print` then prints that `None` as a second line. The full output is two lines: `hello` and `None`. The REPL trick from Section 7.2 only suppresses `None` when it would be printed as the bare result of an expression; an explicit `print(None)` always shows up because that is what `print` was asked to do.

Solution 7.8: A small library question

`dict.get(key)` returns `None` when the key is missing; `dict.get(key, default)` returns `default` instead. The trouble is that the method returns the same `None` in two genuinely different cases: when the key is missing and no default was supplied, and when the key is present with a value of `None`. There is no way to tell these apart from the return value alone. The standard solutions are to check `key in d` first, or to use the sentinel pattern of Section 7.5 (`d.get(key, _MISSING)` and then test `is _MISSING`), or, when `None` is never a valid stored value, to ignore the problem.

CHAPTER 8 · PART III · THE CONTAINERS

Lists in Depth

The single most-used container in Python, examined from the inside out: what it really is in memory, what makes append fast and prepend slow, how to sort it properly, the famous aliasing bug that ate an afternoon, and the four kinds of comprehension you will write every day.

> *There should be one obvious way to do it.*
>
> — Tim Peters, the Zen of Python (PEP 20)

A great deal of what you will write in Python comes down to working with lists. The lines of a file, the rows of a table, the items in a shopping cart, the moves in a game, the results of a query: all of them are naturally lists. Where C++ asks you to choose between `vector`, `list`, `deque`, and `array`, and Java offers `ArrayList` alongside `LinkedList`, `Vector`, and a handful of others, Python gives you one ordered sequence type with a small, sharp set of operations on it, and an explicit alternative (`collections.deque`) for the one situation where the workhorse is the wrong tool. This chapter is about what that workhorse really is, what it does well, what it does badly, and what a careful programmer does about both.

Eight sections. The first establishes what a list is and why Python has only one of them. The second opens the hood and shows the array of pointers underneath, which explains everything about which operations are fast and which are not. Sections three and four cover the surface: building lists, indexing and slicing them, the dozen mutating methods. Section five takes sorting more seriously than most introductions do, because sorting badly is the single most common performance bug in everyday code. Section six revisits the aliasing trap from Chapter 3 in concrete list form, including its evil nested cousin. Section seven handles list comprehensions properly, including their three lesser-known

forms. The last section is a catalogue of performance pitfalls and the four cases where a list is, in fact, the wrong tool.



Figure 8.0. *Donald Knuth, whose Art of Computer Programming is the lineage every chapter of this Part III walks in.* Donald Knuth

8.1 The Workhorse Container

A **list** in Python is an ordered, mutable sequence of values. It is written in square brackets, its elements may be of any type (even mixed), and its length may change at any moment by appending, inserting, or removing items. That sentence describes the surface; the rest of the chapter unpacks the consequences.

```python
fruits   = ["apple", "banana", "cherry"]      # homogeneous; the common case
mixed    = [42, "hello", True, None, [1, 2]]    # mixed; legal, occasionally usef
empty    = []                                       # a list of nothing
numbers  = list(range(5))                         # [0, 1, 2, 3, 4], from any it
```

The choice to make one list type the default is a Python design decision worth pausing on. In Python, lists are *ordered, growable arrays*: random access by index is fast (constant time), appending at the end is fast (amortised constant), and almost everything you would actually want to do with a sequence is supported. The price you pay is that operations near the *front* of the list are slow, because the array has to shift; for those, you reach for `collections.deque`, which we will meet in Chapter 12. But the alternative cost is sharper: forcing every beginner to learn three sequence types and choose between them, before they have written a useful program, would have been a worse trade for the language. Python's principle here is the Zen line at the head of the chapter.

DEF **List**

An ordered, mutable sequence written as `[v1, v2, ...]`. The type is `list`. Implemented in CPython as a dynamic array of `PyObject *` pointers, which makes random access and append fast (O(1)) and operations near the front slow (O(n)). It is the default container in Python for any "collection of things".

→ FURTHER READING: THE WORKHORSE

- Python tutorial: More on Lists *official, brief, useful*
- list reference *the complete API*

8.2 How a List Is Stored

Almost every performance fact about Python lists falls out of one structural decision: a list is implemented in CPython as a *resizable array of pointers to objects*. It is not a linked list (despite the name) and never has been. Each slot in the array holds a `PyObject *`, an address pointing at an object that lives elsewhere in memory. The list itself owns the array; the objects it points at are owned by the reference-counting system we met in Chapter 2.

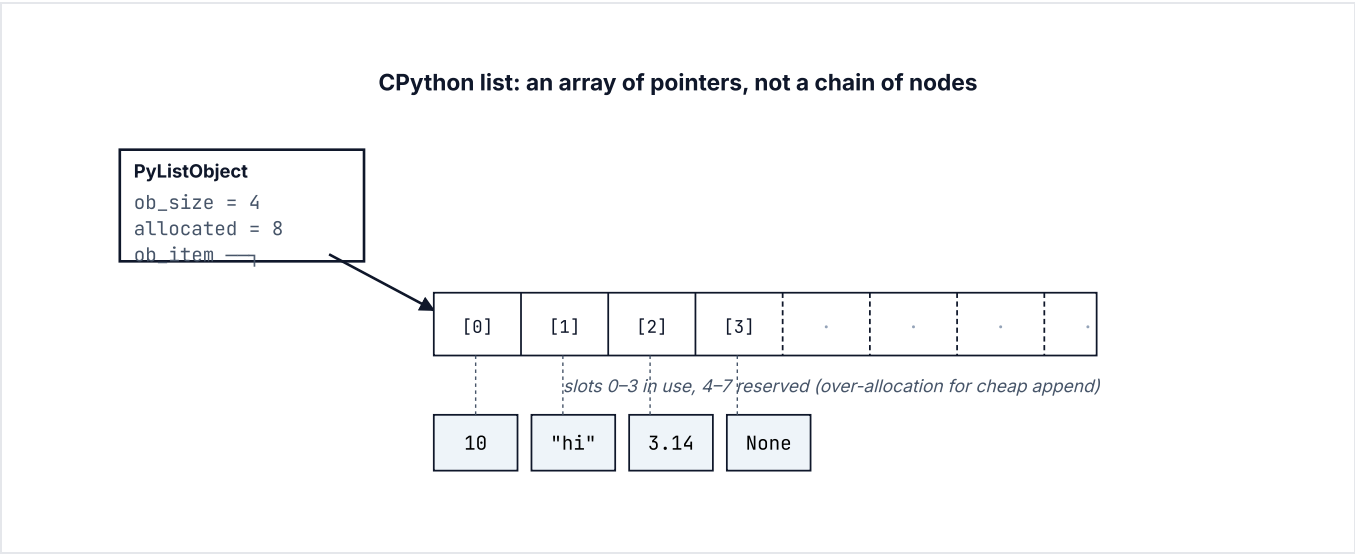


Figure 8.1. A four-element Python list, laid out as CPython actually stores it: a small header pointing at a contiguous array of references, where each slot points at an object somewhere on the heap. The list owns the array; it does not own the objects.

Two practical consequences follow at once. **Random access is fast**, because reaching `lst[k]` requires only adding `k` times a pointer-size to a known base address: constant time, regardless of the list's length. **Append is amortised fast**, because Python over-allocates the underlying array; pushing one item usually just writes into a pre-reserved slot. When the reserved slots run out, Python grows the array geometrically (roughly 12.5% over the current size, rounded to round numbers), copying once. The cost of that copy

spreads across all the appends that filled the array, so on average each append is constant time.

The flip side is also forced: **inserting at or near the front is O(n)**, because every later slot has to shift up by one to make room. **Deleting from the middle is O(n)** for the same reason. And **searching for membership** (`x in lst`) is O(n) because the list has no idea where to look; if you need fast membership, you need a set or a dict instead.

Operation	Cost	Why
`lst[k]`	O(1)	direct array index
`lst[k] = x`	O(1)	direct array write
`lst.append(x)`	O(1) amortised	uses a reserved slot; occasional resize
`lst.pop()`	O(1)	drops the last slot
`lst.insert(0, x)`	O(n)	shifts every element up by one
`lst.pop(0)`	O(n)	shifts every element down by one
`x in lst`	O(n)	linear scan, no index
`len(lst)`	O(1)	stored on the list header
`lst.sort()`	O(n log n)	TimSort (Section 8.5)

A USEFUL NUMBER The over-allocation pattern used by CPython since the early 2000s grows the array by roughly 12.5% on each resize, never less than a small fixed minimum. The exact formula has been tuned over the years; the implementation is in `Objects/listobject.c` in the CPython source. The detail rarely matters, but the principle does: `append` is fast not because each call is cheap but because the cost is amortised over many calls.

→ FURTHER READING: LIST INTERNALS

- Python FAQ: How are lists implemented in CPython? *straight from the source*
- Time Complexity *the operation-by-operation table*
- CPython listobject.c *readable C source for the curious*

8.3 Creating and Indexing

Lists are created in three forms. Most often, you write a *list literal* with square brackets. When the items already live in another iterable (a string, a range, a generator), you call `list()` to materialise them. When you need a fixed-size list initialised with a single value, you multiply.

```python
letters = ["a", "b", "c"]            # literal
chars   = list("hello")             # ['h', 'e', 'l', 'l', 'o']
zeros   = [0] * 5                      # [0, 0, 0, 0, 0]
merged  = [*[1, 2], *[3, 4]]         # [1, 2, 3, 4], splat-merge
```

Once a list exists, you reach into it the same way you reached into a string in Chapter 5. The mechanics are identical (Python's sequence protocol is uniform), so we cover them quickly here.

```python
xs = [10, 20, 30, 40, 50]
xs[0]            # 10                first
xs[-1]           # 50                last (negative wraps)
xs[1:4]          # [20, 30, 40]      slice: half-open [start, stop)
xs[:3]           # [10, 20, 30]      from the start
xs[3:]           # [40, 50]          to the end
xs[::2]          # [10, 30, 50]      every second
xs[::-1]         # [50, 40, 30, 20, 10]  reversed
```

One important detail: **slicing creates a new list**. `xs[:]` is, in fact, the idiomatic way to make a shallow copy of `xs`; we will return to its limits in Section 8.6. And one detail that is unique to lists, not strings: **slice assignment** lets you replace, insert, or delete an entire range in one statement.

EXAMPLE 8.1 Slice assignment, three useful tricks

```python
xs = [1, 2, 3, 4, 5]

# 1. Replace a range with a different number of items:
xs[1:4] = [99]              # [1, 99, 5]

# 2. Insert several items at once without insert():
xs = [1, 2, 3]
xs[1:1] = ["a", "b"]          # [1, 'a', 'b', 2, 3]

# 3. Clear a list in place, useful when other names point at it:
xs[:] = []                        # []  (same object, now empty)
```

The third form matters because it differs from `xs = []`: rebinding the name leaves the old list untouched and still visible through any other name that pointed at it. `xs[:] = []` empties the same object in place, which every aliased name sees.

→ **FURTHER READING: CREATING LISTS**

○ Slicings *the formal grammar of slices*

8.4 The Mutating Methods

A list comes with about a dozen built-in methods that change it in place. The full set fits in a small table, and almost every Python program uses at least three of them.

Method	What it does	Returns
`lst.append(x)`	add one item to the end	None
`lst.extend(iterable)`	add many items, one by one	None
`lst.insert(i, x)`	insert before position i	None
`lst.remove(x)`	delete the first item equal to x (raises ValueError if none)	None

`lst.pop([i])`	remove and return the last (or i-th) item	the item
`lst.clear()`	remove every item	None
`lst.sort()`	sort in place	None
`lst.reverse()`	reverse in place	None
`lst.copy()`	a shallow copy	a new list
`lst.index(x)`	position of the first item equal to x	int
`lst.count(x)`	number of occurrences of x	int

Notice a pattern that catches beginners. **Methods that mutate a list return `None`, not the list.** This is deliberate: it would be a small disaster if `lst.sort()` returned the list, because the very next thing a user would write is `sorted_lst = lst.sort()`, expecting a new sorted list and getting a sorted in place plus a name bound to `None`. The convention forces you to choose, by giving you two different names for the two different actions.

THE CLASSIC MISTAKE

`xs = xs.sort()` sets `xs` to `None`, because `sort` returned nothing. You almost certainly meant `xs.sort()` (which sorts in place and leaves `xs` bound to the sorted list) or `xs = sorted(xs)` (which builds a new list and rebinds `xs` to it). The two are the right answer for different situations: Section 8.5 covers when to use which.

EXAMPLE 8.2 Append versus extend

```python
xs = [1, 2, 3]
xs.append([4, 5])      # [1, 2, 3, [4, 5]] , one item added: the whole list

ys = [1, 2, 3]
ys.extend([4, 5])      # [1, 2, 3, 4, 5]  , many items added: the contents
```

The mistake of using `append` when you meant `extend` is among the most common in beginner code, because they sound similar. `append` takes a value and adds it; if the value happens to be a list, you get a list-inside-a-list. `extend` takes an iterable and pulls its items in one at a time.

→ **FURTHER READING: LIST METHODS**

○ More on lists *the full Python tutorial section*

8.5 Sorting, Properly

Python sorts with a custom algorithm called **TimSort**, designed by Tim Peters in 2002. It is a hybrid of insertion sort (fast for short runs and partially-sorted data) and merge sort (good asymptotic behaviour); it is *stable*, meaning items that compare equal keep their original relative order; and it is the default sort algorithm in Java, Android, and other places besides. Its average and worst case are O(n log n), and on data that is already nearly sorted it can approach O(n).

The two ways to invoke it are well worth distinguishing.

```python
xs = [3, 1, 4, 1, 5]

xs.sort()                  # in-place: mutates xs, returns None
ys = sorted(xs)            # new list: leaves the argument unchanged, returns the

xs.sort(reverse=True)      # descending
```

For anything beyond plain values, the `key` argument is where sorting becomes a real tool. `key=f` means "decide each item's place by what `f` returns for it, not by the item itself". The function is called once per item (not once per comparison), so even an expensive key function is fine.

EXAMPLE 8.3 Sorting by a key, and by several

PYTHON

```python
people = [
    {"name": "Ada",     "age": 28, "city": "London"},
    {"name": "Linus",   "age": 54, "city": "Helsinki"},
    {"name": "Caroline","age": 28, "city": "Paris"},
]

sorted(people, key=lambda p: p["age"])
# Ada, Caroline, Linus, ties broken by insertion order (stable sort)

# Sort by (age, name) to break ties on name:
sorted(people, key=lambda p: (p["age"], p["name"]))

# With operator.itemgetter, faster and cleaner than a lambda:
from operator import itemgetter
sorted(people, key=itemgetter("age", "name"))
```

Two facts to internalise. **The sort is stable**, so when two ages tie, the original order is preserved (Ada stays before Caroline in the first example because she was input first). And **multi-field sorts are a tuple key**: comparing tuples compares element by element, exactly as you want.

STABILITY, USED WELL Because Python's sort is stable, you can sort a list by several criteria by sorting *multiple times*, least significant first. To sort by age (primary) then name (secondary), one elegant way is: sort by name first, then by age. The age sort will preserve the name order for ties. This trick is not always the fastest, but it scales to any number of criteria without building a tuple key.

→ FURTHER READING: SORTING

- Sorting HOWTO *official, with the tuple-key idiom and operator.itemgetter*
- Tim Peters' notes on TimSort *the original design document, in CPython*
- TimSort on Wikipedia *the algorithm in full*

8.6 Copy and the Aliasing Trap

Chapter 3 introduced the rule that names are labels on values, not boxes; `b = a` binds a second name to the same object. The list version of that rule is where it bites hardest, because lists are mutable and mutating one through one name is visible through the other.

```python
a = [1, 2, 3]
b = a               # NOT a copy, b is the same list, with a second name
b.append(4)
print(a)            # [1, 2, 3, 4], a saw it too
```

To make an independent copy, you ask for one explicitly. Three forms all do the same thing for a flat list:

```python
b = a.copy()          # the explicit method (Python 3.3+)
b = a[:]              # the slice idiom, older but still common
b = list(a)           # the constructor
```

Each of these makes a *shallow* copy: the outer list is new, but the items inside it are still the same objects as in the original. For a list of immutable values (numbers, strings, tuples) that is all you ever need. For a list that contains *other* mutable objects, especially other lists, shallow copying is not enough, because both the original and the copy still share the inner objects.

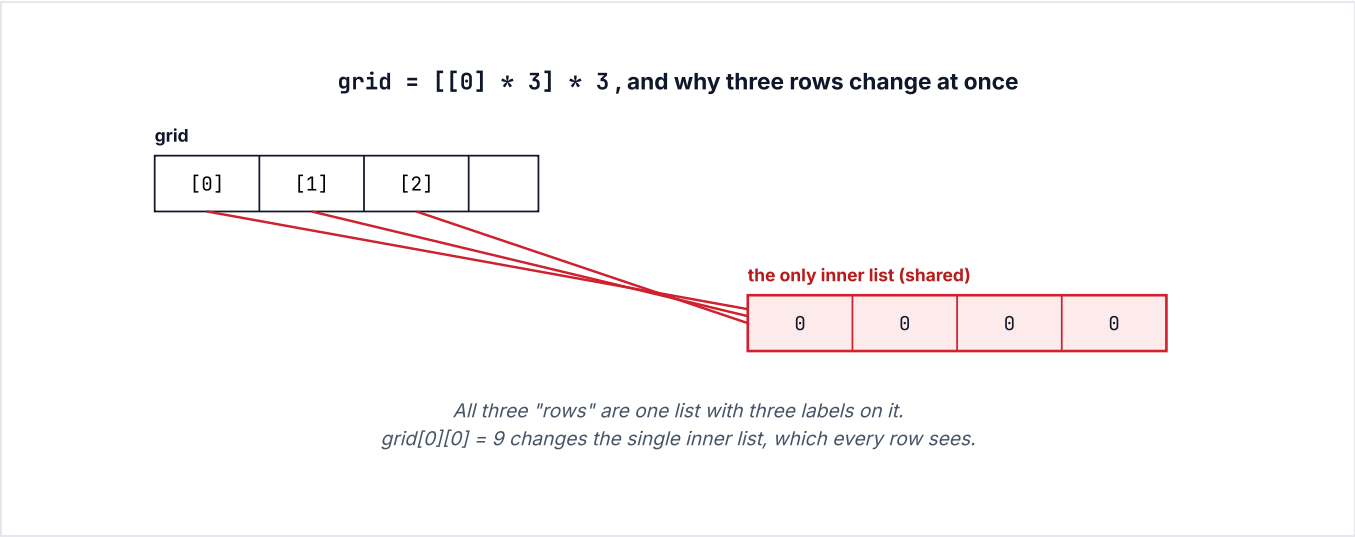


Figure 8.2. The famous `[[0] * 3] * 3` trap. The outer multiplication does not produce three independent rows; it produces three references to the *one* inner list. To get genuine independence, build each row separately with a comprehension: `[[0] * 3 for _ in range(3)]`.

The remedy for nested mutable structures is the `copy.deepcopy` function, which copies all the way down. It is more expensive than a shallow copy, so reach for it only when you need it.

```python
import copy
nested = [[1, 2], [3, 4]]
shallow = nested.copy()
deep    = copy.deepcopy(nested)

nested[0].append(99)
print(shallow)     # [[1, 2, 99], [3, 4]], shallow shares the inner lists
print(deep)        # [[1, 2],     [3, 4]], deep is independent all the way down
```

→ **FURTHER READING: COPYING**

- copy, shallow and deep copy operations *the official reference*
- Ned Batchelder: names and values *the canonical explanation of why this trap exists*

8.7 Comprehensions in Depth

A list comprehension is a compact one-expression form for building a list out of another iterable. The basic shape is `[expression for item in iterable]`; with a filter, `[expres-`

`sion for item in iterable if condition]`. Read aloud it sounds like English ("the square of n, for each n in the range, if n is even"), and once it is in your fingers it becomes the most natural way to think about most transformations.

```python
[x * x for x in range(10)]                    # [0, 1, 4, 9, ..., 81]
[x for x in range(10) if x % 2 == 0]          # [0, 2, 4, 6, 8]
[w.upper() for w in ["a", "b", "c"]]          # ['A', 'B', 'C']
```

The shape generalises in two directions that beginners often miss. **Multiple `for` clauses** nest: the leftmost iterates slowest, exactly as a series of nested loops would, and the values from all clauses are visible in the expression. **Conditional expressions in the value position** (`expr_true if cond else expr_false`) let you transform differently per item.

EXAMPLE 8.4 The five comprehension shapes

```python
# 1. Transform every item
squares = [n * n for n in range(6)]
# [0, 1, 4, 9, 16, 25]

# 2. Filter
evens = [n for n in range(10) if n % 2 == 0]
# [0, 2, 4, 6, 8]

# 3. Branch per item (conditional in the expression position)
parity = ["even" if n % 2 == 0 else "odd" for n in range(5)]
# ['even', 'odd', 'even', 'odd', 'even']

# 4. Cartesian product (two for clauses)
pairs = [(x, y) for x in [1, 2] for y in ["a", "b"]]
# [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]

# 5. Flatten
flat = [x for row in [[1, 2], [3, 4]] for x in row]
# [1, 2, 3, 4]
```

Read multiple `for` clauses left to right, just as if they were nested. The fifth form, flattening, is worth memorising: it is the comprehension version of two nested loops appending into a single list.

Comprehensions are also faster than the equivalent `for`-and-append loop, often by a meaningful margin, because the bytecode is tighter and avoids the repeated attribute lookup on `list.append`. They are not magic, however; they are still `O(n)` in the size of the iterable, and a complicated expression inside the brackets is exactly as expensive as the equivalent code in a loop. The right question to ask before reaching for one is not "is it faster?" but "is it clearer?".

WHEN NOT TO USE A COMPREHENSION If the expression has side effects (printing, writing files, calling an API), use a plain `for` loop. Comprehensions are for building values; loops are for performing actions. A comprehension whose only purpose is to call `print` on each item is universally agreed to be ugly Python. The same goes for any comprehension that needs more than one nested `if`: at that point, an ordinary loop is clearer.

→ **FURTHER READING: COMPREHENSIONS**

- List comprehensions, official tutorial *the full grammar with examples*
- PEP 202: List Comprehensions *the original 2000 proposal*

8.8 Performance Pitfalls, and When Not to Use a List

The list is the right answer to most problems, but it is not the right answer to all problems, and the situations where it is wrong are predictable enough to be worth memorising. Four of them are common and avoidable.

1. Membership tests inside a loop

If you check whether a value is in a list, the cost is O(n). Doing that inside another loop is $O(n^2)$, and at scale it crawls. The fix is to put the values into a `set`, where membership is O(1).

PYTHON

```python
# Slow: O(n * m), n items, each checked against an m-element list
banned = ["alice", "bob", ...]      # thousands
allowed = [u for u in users if u not in banned]
```

```python
# Fast: O(n + m)
banned = set(["alice", "bob", ...])
allowed = [u for u in users if u not in banned]
```

2. Inserting or popping at the front in a loop

Every `insert(0, x)` or `pop(0)` shifts the entire list. If you find yourself doing this in a loop, the right container is `collections.deque` ("double-ended queue"), which gives you O(1) operations at both ends.

PYTHON

```python
from collections import deque
queue = deque([1, 2, 3])
queue.appendleft(0)        # O(1) prepend
queue.popleft()             # O(1) pop from the front
```

3. Modifying a list while iterating over it

Adding to or removing from a list while a `for` loop walks over it leads to skipped items, infinite loops, or both. The fix is to iterate over a copy, or to build a new list with a comprehension.

PYTHON

```python
# Buggy: skips an item every time one is removed
xs = [1, 2, 3, 4]
for x in xs:
    if x % 2 == 0:
        xs.remove(x)
print(xs)   # [1, 3, 4] , the 4 was missed!

# Fix: build a new list rather than mutate while iterating
xs = [x for x in xs if x % 2 != 0]
```

4. Building a long list by concatenation in a loop

The expression `result = result + [x]` builds a new list every iteration, copying everything so far. That makes the whole loop $O(n^2)$. Use `append`, which mutates in place at O(1) amortised, or build with a comprehension.

```python
# Slow, O(n²)
result = []
for x in source:
    result = result + [x]

# Fast, O(n)
result = []
for x in source:
    result.append(x)

# Idiomatic, also O(n)
result = [x for x in source]
```

PYTHON

WHEN TO REACH PAST THE LIST

Use a `set` when order does not matter and membership does. Use a `tuple` when the data is fixed and you want to communicate that. Use a `dict` when you have key-to-value lookups. Use a `deque` when you push and pop at both ends. Use a `numpy.ndarray` when you need numerical work on a million-element array at near-C speed. The list is the default; the others exist for the cases where the default would mislead.

→ **FURTHER READING: CHOOSING A CONTAINER**

- collections.deque *double-ended queue, O(1) at both ends*
- Using lists as queues *the Python tutorial's own warning*

§ § §

Exercises

EXERCISE 8.1 Predict the cost

For each operation, predict whether it is O(1), O(n), or O(n²), and explain in one sentence why: (a) `xs.append(x)`; (b) `xs.insert(0, x)`; (c) `x in xs`; (d) `xs[k]`; (e) `len(xs)`; (f) `xs.sort()`.

EXERCISE 8.2 Append vs extend

Without running them, predict the contents of `xs` after each: (a) `xs = [1, 2]; xs.append([3, 4])`; (b) `xs = [1, 2]; xs.extend([3, 4])`; (c) `xs = [1, 2]; xs += [3, 4]`. Explain which two produce identical results and why.

EXERCISE 8.3 Slice surgery

Starting from `xs = [10, 20, 30, 40, 50]`, perform these in order: replace the middle three items with the single value `99`; insert `0` at the beginning without using `insert`; clear the list in place (do *not* rebind `xs` to a new list). Show each result.

EXERCISE 8.4 Sort by a key

Given `people = [("Ada", 28), ("Linus", 54), ("Caroline", 28)]`, sort them by age, then by name. Then sort them by name length. Use the `key` argument; no manual loops.

EXERCISE 8.5 The aliasing bug

Show, in three lines, the bug in `b = a; b.append(99)`. Then write the fix two different ways (the explicit method and the slice idiom). Predict whether `copy.deepcopy` would be necessary if the list were `a = [[1, 2], [3, 4]]`, and verify by trying both.

EXERCISE 8.6 A 3×3 grid done correctly

Write code that produces a 3×3 grid initialised to zero where modifying `grid[0][0]` does not change `grid[1][0]`. Show why `[[0] * 3] * 3` fails this test and how a list comprehension fixes it.

EXERCISE 8.7 Comprehensions

Write each as a single list comprehension: (a) the squares of the odd numbers from 1 to 10; (b) the words longer than three characters in `"the quick brown fox jumps over the lazy dog".split()`, all uppercased; (c) the (x, y) coordinates of every cell in a 3×3 grid; (d) the elements of `[[1, 2], [3, 4], [5, 6]]` as a single flat list.

EXERCISE 8.8 Container choice

For each scenario, name the right container (list, tuple, set, dict, or deque) and justify your choice: (a) a queue of pending jobs to be processed in order; (b) a collection of unique user IDs you need to test membership against; (c) the coordinates of a single point on a 2D plane; (d) a phone book mapping names to numbers; (e) the rows of a CSV file you will sort and write back.

Solutions to Chapter 8 Exercises

Solution 8.1: Predict the cost

(a) O(1) amortised, because over-allocation makes most appends a single write. (b) O(n), because every later element shifts up by one. (c) O(n), because the list has no index and must scan. (d) O(1), direct array access by offset. (e) O(1), the length is stored on the list header. (f) O(n log n) average, the cost of TimSort. The pattern that ties all six together: list operations on or near the end are fast; operations on or near the front are slow; positional access by index is free.

Solution 8.2: Append vs extend

(a) `[1, 2, [3, 4]]`. `append` adds its argument as a single item, and the argument is a list, so the result is a list-inside-a-list. (b) `[1, 2, 3, 4]`. `extend` iterates over its argument and adds each item. (c) `[1, 2, 3, 4]`. The `+=` operator on a list is equivalent to `extend`. So (b) and (c) are identical; (a) is the trap.

Solution 8.3: Slice surgery

```python
xs = [10, 20, 30, 40, 50]
xs[1:4] = [99]      # [10, 99, 50]
xs[:0] = [0]         # [0, 10, 99, 50]
xs[:] = []             # [], same object, now empty
```

The first form replaces three items with one. The second inserts at position 0 by assigning to an empty slice `xs[:0]`. The third empties the existing object in place; `xs = []` would create a new empty list and leave the old one alive elsewhere.

Solution 8.4: Sort by a key

```python
people = [("Ada", 28), ("Linus", 54), ("Caroline", 28)]

sorted(people, key=lambda p: (p[1], p[0]))
# [('Ada', 28), ('Caroline', 28), ('Linus', 54)]

sorted(people, key=lambda p: len(p[0]))
# [('Ada', 28), ('Linus', 54), ('Caroline', 28)]
```

Tuple keys compare element by element, so `(age, name)` gives the multi-field sort. For name length, the key function is simply `len` applied to the name; the sort is stable so equal-length names keep their insertion order.

Solution 8.5: The aliasing bug

```python
a = [1, 2, 3]
b = a; b.append(99)
print(a)            # [1, 2, 3, 99], a changed too

# Fixes for a flat list:
b = a.copy()
b = a[:]

# For a nested list, both shallow forms still SHARE the inner lists:
a = [[1, 2], [3, 4]]
b = a.copy()
a[0].append(99)    # mutates the inner list, visible through b too
print(b)            # [[1, 2, 99], [3, 4]]
```

```
import copy
b = copy.deepcopy(a) # now truly independent
```

Shallow copy duplicates the outer list but not the inner ones, so a nested structure with mutable elements still shares those elements. `deepcopy` walks the whole tree and is the right answer there.

Solution 8.6: A 3x3 grid done correctly

PYTHON

```
# Wrong: three references to one inner list
grid = [[0] * 3] * 3
grid[0][0] = 9
print(grid)         # [[9, 0, 0], [9, 0, 0], [9, 0, 0]], all three rows changed

# Right: comprehension builds a fresh row each time
grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 9
print(grid)         # [[9, 0, 0], [0, 0, 0], [0, 0, 0]], only one cell changed
```

The trap is in the outer `* 3`, which does not duplicate the inner list, it copies its reference three times. The comprehension evaluates `[0] * 3` separately on each iteration, producing three independent lists.

Solution 8.7: Comprehensions

PYTHON

```
[n * n for n in range(1, 11) if n % 2 == 1]   # [1, 9, 25, 49, 81]

[w.upper() for w in "the quick brown fox jumps over the lazy dog".split() if len(
# ['QUICK', 'BROWN', 'JUMPS', 'OVER', 'LAZY']

[(x, y) for x in range(3) for y in range(3)]
# [(0,0), (0,1), (0,2), (1,0), ..., (2,2)], 9 pairs

[v for row in [[1, 2], [3, 4], [5, 6]] for v in row]
# [1, 2, 3, 4, 5, 6]
```

Solution 8.8: Container choice

(a) `collections.deque`, because you need O(1) pop from the front; a list would be O(n). (b) `set`, because uniqueness and fast membership are exactly what it offers. (c) `tuple`, because the two coordinates are a single record that should not change. (d) `dict`, because name-to-number is a key-value mapping. (e) `list`, because rows have order and may need to be sorted in place; the list is the default sequence and has every method you need.

CHAPTER 9 · PART III · THE CONTAINERS

Tuples & Named Tuples

Tuples are not frozen lists; they are immutable records. Understanding that one distinction reshapes how you choose between the two, why a tuple can be a dictionary key, and why `(5)` is not a tuple but `(5,)` is.

> *Explicit is better than implicit.*
>
> — Tim Peters, the Zen of Python (PEP 20)

The most useful sentence anyone ever wrote about the difference between lists and tuples is this: a *list* is a sequence of homogeneous things you intend to walk over, while a *tuple* is a record of heterogeneous fields you intend to address by position. A list of scores is a list, a row from a database is a tuple. The two types have similar surface syntax, similar indexing rules, and very different intent, and the experienced Python programmer chooses between them by intent, not by which one is "frozen". This chapter is about that distinction, the small set of language facts that make tuples work, and the named-tuple feature that gives positional records back their meaning.

9.1 The Tuple, Properly Understood

A **tuple** is an ordered, immutable sequence of values, conventionally written in parentheses: `(1, 2, 3)`. It looks like a list with rounder brackets, and on the surface it behaves like one: you index it, slice it, iterate over it, and ask its length the same way. The shared surface tempts beginners into thinking of a tuple as "a list you can't change", which works as a first approximation and slowly accumulates confusion afterwards.

The deeper way to read the type is one Guido van Rossum has stated repeatedly: **tuples are for records, lists are for sequences**. A list usually holds many things of the *same*

kind, where every element plays the same role and only its position in the order matters. A tuple usually holds a few things of *different* kinds, where each position has its own meaning. `[12, 7, 19, 4]` is a list of temperatures; `("Ada", 28, "London")` is a tuple that is one person's record. The fact that a tuple cannot be modified is a consequence of this intent rather than its purpose: a record's shape is fixed, so its type is too.

DEF **Tuple**

An ordered, immutable sequence of values, type `tuple`. Written with parentheses (which are optional) and separated by commas (which are not). Tuples are hashable when every element is hashable, which lets them serve as dictionary keys and set members. Use them for records of fixed shape and for "multiple values returned together".

→ **FURTHER READING: TUPLES VS LISTS**

- Tuples and Sequences *the Python tutorial section*
- "What's the difference between lists and tuples?" *the canonical Stack Overflow answer, with Guido's record-vs-sequence quote*

9.2 The Comma Makes It a Tuple, Not the Parens

A small piece of syntax catches every newcomer once, and it is worth understanding now so you never trip on it. *The parentheses do not make a tuple. The commas do.* Parentheses serve their usual grouping role around an expression; when they enclose a comma-separated list of values, the commas are what mark the value as a tuple.

PYTHON

```
t = 1, 2, 3              # a tuple, no parens at all
t = (1, 2, 3)            # a tuple, parens added for clarity

p = (5)                  # NOT a tuple, just the number 5 in parentheses
type(p)                  # <class 'int'>

p = (5,)                 # a one-element tuple, the trailing comma makes it
type(p)                  # <class 'tuple'>

empty = ()                # the one exception: empty tuple needs parens
empty = tuple()           # or call the constructor
```

The empty tuple is the one case where parentheses cannot be omitted, because a comma with nothing on either side has no other meaning. Everywhere else, you can drop the parens; whether you do is a matter of style. Inside a function call, where the parentheses are already taken by the call itself, the trailing-comma rule is what disambiguates a one-element tuple from a parenthesised argument.

THE CLASSIC SLIP Writing `f((5))` when you meant `f((5,))` passes an `int`, not a one-element tuple. The same trap shows up in `return` statements: `return (5)` returns `5`; `return (5,)` returns a one-tuple. If your function's "tuple of one thing" is being treated as the thing itself, look for a missing trailing comma.

9.3 Packing and Unpacking

When several values appear on the right of an `=` separated by commas, Python *packs* them into a tuple. When several names appear on the left, Python *unpacks* a tuple into them. These two operations are the same machinery seen from opposite ends, and together they power some of the most idiomatic Python you will write.

PYTHON

```python
# Packing: the right side becomes a tuple, which is bound to t.
t = 1, 2, 3
type(t), len(t)        # (<class 'tuple'>, 3)

# Unpacking: each name on the left gets one item from the right.
a, b, c = (1, 2, 3)
print(a, b, c)             # 1 2 3

# The swap is just simultaneous packing and unpacking:
a, b = b, a                # the right side packs to (b, a), the left unpacks it
```

The number of names on the left must match the number of values on the right, or Python raises `ValueError` with a precise count. To accept "the rest", whatever its size, use a starred name. The starred name collects everything not claimed by the other names, and the rule is that *at most one star is allowed*, but its position is free.

EXAMPLE 9.1 Starred unpacking, in three positions

```python
first, *rest = (1, 2, 3, 4, 5)
print(first, rest)        # 1 [2, 3, 4, 5]

*head, last = (1, 2, 3, 4, 5)
print(head, last)         # [1, 2, 3, 4] 5

first, *middle, last = (1, 2, 3, 4, 5)
print(first, middle, last)  # 1 [2, 3, 4] 5
```

Two facts to notice. First, the starred name binds a **list**, not a tuple, even though the source was a tuple; the language chose `list` here because the result is the variable-length, "the rest" part of the destructure. Second, you can place the star anywhere in the target, and Python figures out which items belong to it by counting from both ends. The unpacking is greedy but unambiguous.

Unpacking works on any iterable, not just tuples; `a, b, c = "abc"` unpacks a string into three names. This generalisation, called *iterable unpacking*, is why `for x, y in pairs:` and `for i, name in enumerate(names):` read so cleanly: each iteration packs one element of the iterable, then unpacks it into the loop names.

→ **FURTHER READING: UNPACKING**

- PEP 3132: Extended Iterable Unpacking *the starred-name proposal from 2007*
- PEP 448: Additional Unpacking Generalizations *where* `[*xs, *ys]` *and* `{**a, **b}` *came from*

9.4 Records, Not Lists: When to Reach for a Tuple

The clearest way to know when a tuple is the right type is to ask one question: *does each position have its own meaning?* If yes, you are looking at a record, and a tuple is correct. If every position plays the same role, you have a sequence, and a list is correct. Three concrete cases sharpen the distinction.

Data	Right type	Why

A geographic coordinate `(51.5, -0.1)`	tuple	position 0 is latitude, position 1 is longitude, different fields
The cities you visited on a trip	list	every element is "a city"; order is the only structure
An RGB colour `(255, 99, 71)`	tuple	three positions with three meanings (R, G, B)
The pixels of one row of an image	list	thousands of homogeneous values
The return value `(name, age, city)`	tuple	three different fields packed for one transport
A queue of pending jobs	list (or deque)	every element is "a job"; you push and pop

One pragmatic consequence of choosing well: a tuple's immutability documents that a value's *shape* is fixed. A reader who sees `def location(): return (lat, lon)` can be confident that the result will always be a two-element record, while a reader who sees `def location(): return [lat, lon]` has no such guarantee, and any function the result is passed to could quietly extend it.

A TUPLE IS ALSO FASTER AND LIGHTER Because a tuple's length is known at creation and never changes, CPython can store it more compactly than a list (no over-allocation, no separate header for length-vs-capacity, and the items can live in the same memory block). The constant-factor difference rarely matters for individual values, but for code that creates millions of small records it adds up. The right reason to choose a tuple is still semantic; the performance is a side benefit.

9.5 Hashable: Compound Dictionary Keys

Chapter 3 introduced hashability and the rule that mutable objects (lists, dicts, sets) cannot be dictionary keys, while immutable ones (numbers, strings, tuples) can. Tuples are where this rule pays off, because a tuple of hashable items is itself hashable, and the re-

sulting "compound key" lets a dictionary be indexed by structured data without inventing a wrapper.

```python
locations = {
    (51.5, -0.1):  "London",
    (35.7, 139.7): "Tokyo",
    (40.7, -74.0): "New York",
}
locations[(51.5, -0.1)]    # 'London'

# A grid of pieces, keyed by (row, col):
board = {}
board[(0, 0)] = "rook"
board[(0, 4)] = "king"

# A list, attempted as a key, fails:
locations[[51.5, -0.1]]
# TypeError: unhashable type: 'list'
```

The reason a list cannot be a key, you will remember from Chapter 3.7, is the contract between hashing and equality: if a key's hash could change while it sat in the table, the table would lose track of where it had stored the entry. A list is mutable, so its hash is not allowed to exist at all; a tuple is immutable, so its hash is stable forever. The same rule lets tuples of hashable items be set members.

A TUPLE IS HASHABLE ONLY IF ITS ELEMENTS ARE

A tuple containing a list is not hashable. The unhashability is contagious: any container that *contains* a non-hashable element is itself non-hashable, because computing the container's hash requires hashing each of its elements. If you find yourself wanting `(name, friends)` as a key where `friends` is a list, freeze the inner list into a tuple (or `frozenset` when order does not matter) first.

9.6 Multiple Return Values

A function in Python returns exactly one value, but that value can be a tuple, and the calling code can unpack the tuple back into separate names in the same statement. The combination of packing on the way out and unpacking on the way in makes "multiple return values" feel as if Python supported them natively.

```python
def min_max(xs):
    return min(xs), max(xs)          # packs into a tuple

lo, hi = min_max([3, 1, 4, 1, 5, 9])  # unpacks at the call site
print(lo, hi)                          # 1 9
```

The standard library uses this pattern throughout: `divmod(a, b)` returns `(quotient, remainder)`, `str.partition(sep)` returns `(before, sep, after)`, `os.path.split(p)` returns `(head, tail)`. When you write your own function and the answer naturally has two or three parts, returning them as a tuple and letting the caller unpack is more idiomatic than packing them into a list or building a small ad-hoc class.

EXAMPLE 9.2 A function returns three things

```python
def parse_email(addr):
    user, _, domain = addr.partition("@")
    local, _, tag = user.partition("+")
    return local, tag, domain

local, tag, domain = parse_email("ada+work@example.com")
print(local, tag, domain)   # ada work example.com
```

The function returns three values as a tuple; the caller takes the tuple apart in the same line. Notice that inside the function, `str.partition` itself returns a three-element tuple, which the local `user, _, domain` immediately unpacks. Tuples are the lingua franca of compound returns.

The convention of using `_` as a name for "I do not care about this value" is widely understood and respected by linters. It is, of course, still a real variable, but the name signals to a reader "ignore this".

9.7 Named Tuples: When Position Needs a Name

The one limitation of using bare tuples for records becomes obvious the moment a record grows beyond two or three fields. `person[3]` is not self-explanatory; whoever reads it has to remember which position is which, or look up the convention. The standard library has, since Python 2.6, offered a fix called the **named tuple**: a tuple with names for its positions. It costs nothing at runtime and reads as well as it works.

There are now two ways to declare one. The original is `collections.namedtuple`, a factory function. The modern one, since Python 3.5, is the class-based `typing.NamedTuple`, which adds type annotations and a familiar class syntax. Both produce the same kind of object.

EXAMPLE 9.3 The two named-tuple syntaxes

```python
# Original: a factory call
from collections import namedtuple
Point = namedtuple("Point", ["x", "y"])

p = Point(3, 4)
p.x, p.y                # (3, 4), named access
p[0], p[1]            # (3, 4), positional access still works
x, y = p                # unpacking still works

# Modern: a class with type annotations
from typing import NamedTuple

class Point(NamedTuple):
    x: float
    y: float
    def distance_to_origin(self):
        return (self.x ** 2 + self.y ** 2) ** 0.5

p = Point(3, 4)
p.distance_to_origin()     # 5.0
```

The class-based form is the one to prefer in modern code. It documents types, allows methods, and reads like a small class. Behind the scenes both forms produce a subclass of `tuple`, which is why named tuples are still tuples in every sense: indexable, iterable, comparable, hashable when their fields are, and unpackable.

A handful of methods are added by the named-tuple machinery, all prefixed with an underscore to avoid colliding with field names. The three you will actually use:

Method	What it does
`p._asdict()`	convert to a regular `dict` with the field names as keys
`p._replace(x=10)`	return a new named tuple with one or more fields changed
`Point._fields`	the tuple of field names, useful for introspection

`_replace` is worth singling out, because it gives you a way to "update" an immutable record. Tuples cannot be modified, so the only way to get a new tuple with one field different is to create one; `_replace` does that, copying the other fields automatically.

```python
p = Point(3, 4)
q = p._replace(x=10)          # Point(x=10, y=4), p is unchanged
```

NAMED TUPLE VS DATACLASS

If named tuples were not enough, Python 3.7 added `@dataclass` (PEP 557), which serves the same "data holder" role but with a mutable, regular-class body. The two are not redundant; they complement each other. Use a **named tuple** when the record is small, immutable, and frequently created or hashed (it is lighter and works as a dict key). Use a **dataclass** when the record has methods, will be mutated after creation, or has many fields that benefit from default values. We meet dataclasses properly in Chapter 19 (the modern-class chapter).

→ **FURTHER READING: NAMED TUPLES**

- collections.namedtuple *the original factory form*
- typing.NamedTuple *the modern class-based form*
- PEP 557: Data Classes *the close cousin; see Chapter 19*

§ § §

Exercises

EXERCISE 9.1 The comma test

For each, predict the type and value: (a) `x = 1, 2, 3`; (b) `x = (1, 2, 3)`; (c) `x = (5)`; (d) `x = (5,)`; (e) `x = ()`; (f) `x = 5,`. Run them to check.

EXERCISE 9.2 Unpack the line

Given the line `"Lovelace,Ada,1815,London"`, use `str.split` and a single starred-unpacking statement to bind `last`, `first`, and `rest` (a list of any remaining fields). Try the same with the star on the left, in the middle, and on the right; explain each.

EXERCISE 9.3 Choose the type

For each, choose `list` or `tuple` and justify in one sentence: (a) the days a user logged in; (b) a date stored as year-month-day; (c) the cards in a deck; (d) the dimensions of a rectangle (width, height); (e) the items in a shopping cart; (f) an HTTP response stored as (status_code, headers, body).

EXERCISE 9.4 A compound dictionary key

Build a dictionary that maps `(row, col)` coordinates to chess pieces, populating it with a starting rook at `(0, 0)`, a knight at `(0, 1)`, and a queen at `(0, 3)`. Look up the piece at `(0, 1)`. Then try to add a piece using a *list* as the key, observe the error, and explain it.

EXERCISE 9.5 Multiple return values

Write a function `stats(xs)` that returns the count, mean, and standard deviation of a list of numbers as a single tuple. Use it on `[10, 12, 23, 23, 16, 23, 21, 16]` and unpack the result in one line.

EXERCISE 9.6 A named tuple

Using `typing.NamedTuple`, define a `Person` with fields `name: str`, `age: int`, and `city: str`. Create three of them, build a list, and sort by age (use the same `key=` idiom from Chapter 8). Then use `_replace` to make a copy of one person with a different city.

EXERCISE 9.7 Hashability rules

Predict whether each can be used as a dictionary key, and explain: (a) `(1, 2, 3)`; (b) `(1, [2, 3])`; (c) `("Ada", 28)`; (d) `(1, (2, 3))`; (e) `(1, {2, 3})`; (f) `(1, frozenset({2, 3}))`. Try each in a REPL with `hash(...)`.

EXERCISE 9.8 A design question

You are writing a function that returns a user's profile. The profile has 8 fields. Discuss the trade-off in one paragraph between returning a bare tuple, a named tuple, a `dict`, and (looking ahead) a `@dataclass`. Which would you choose, and why?

Solutions to Chapter 9 Exercises

Solution 9.1: The comma test

(a) `tuple`, `(1, 2, 3)`. (b) `tuple`, `(1, 2, 3)`, parens are decorative. (c) `int`, `5`, parens are grouping, not tuple-making. (d) `tuple`, `(5,)`, the trailing comma is what makes it. (e) `tuple`, `()`, the empty case, the one time parens are required. (f) `tuple`, `(5,)`, a bare trailing comma works on the right of an assignment too. The rule to memorise: the comma makes the tuple.

Solution 9.2: Unpack the line

```python
line = "Lovelace,Ada,1815,London"
last, first, *rest = line.split(",")
# last='Lovelace', first='Ada', rest=['1815', 'London']

# Star on the left captures the leading items:
*head, city = line.split(",")
# head=['Lovelace','Ada','1815'], city='London'

# Star in the middle splits front and back:
last, *middle, city = line.split(",")
# last='Lovelace', middle=['Ada','1815'], city='London'
```

The starred name always captures the variable-length portion, with the unstarred names taking from each end as needed. The starred result is always a list.

Solution 9.3: Choose the type

(a) list, variable-length sequence of "a login". (b) tuple, three fields with three meanings. (c) list, homogeneous sequence of "a card". (d) tuple, two named fields. (e) list, variable-length sequence of items. (f) tuple, three differently-typed fields, the canonical compound return.

Solution 9.4: A compound dictionary key

```python
board = {(0, 0): "rook", (0, 1): "knight", (0, 3): "queen"}
board[(0, 1)]          # 'knight'

board[[0, 4]] = "king"  # TypeError: unhashable type: 'list'
```

A list cannot be a dictionary key because it is mutable; its hash value is not allowed to exist at all. A tuple of hashable items is itself hashable and so makes a fine compound key.

Solution 9.5: Multiple return values

```python
import statistics
def stats(xs):
    return len(xs), statistics.mean(xs), statistics.stdev(xs)

n, m, s = stats([10, 12, 23, 23, 16, 23, 21, 16])
print(n, m, s)         # 8 18 5.237...
```

The function returns a three-element tuple; the caller unpacks it into three names in one assignment. This is the idiomatic Python form of "multiple return values".

Solution 9.6: A named tuple

```python
from typing import NamedTuple

class Person(NamedTuple):
    name: str
    age:  int
```

```
    city: str

people = [Person("Ada", 28, "London"),
          Person("Linus", 54, "Helsinki"),
          Person("Caroline", 35, "Paris")]

by_age = sorted(people, key=lambda p: p.age)
ada_in_paris = people[0]._replace(city="Paris")
```

The class-based form gives a type-annotated, methods-allowed declaration; sorting uses the same `key=` idiom as Chapter 8; `_replace` produces a new tuple with one field changed without mutating the original.

Solution 9.7: Hashability rules

(a) Yes, tuple of ints. (b) **No**: contains a list, which is unhashable; the unhashability is contagious. (c) Yes, tuple of str and int. (d) Yes, tuples nest fine. (e) **No**: a regular `set` is mutable and unhashable. (f) **Yes**: `frozenset` is the immutable, hashable counterpart of `set`, so the outer tuple is hashable too. The rule: a tuple is hashable iff every element is.

Solution 9.8: A design question

A bare tuple is the lightest but loses field meaning, so callers must remember the order, fine for two or three fields, but not eight. A `dict` gives named access at the cost of allowing arbitrary keys at runtime and no static checks. A `typing.NamedTuple` gives named access plus immutability plus type annotations, at near-zero cost, and works as a dictionary key. A `@dataclass` goes further by allowing mutation, default values, and behaviour (methods). For an eight-field profile that is mostly read, the choice is between `NamedTuple` and `dataclass`; pick `NamedTuple` if the profile is small, immutable, and frequently hashed or sent across function boundaries, and `dataclass` if the fields are likely to be updated in place or have many sensible defaults. Either is a strong choice over the bare tuple, and either is a clearer choice than a free-form dict.

CHAPTER 10 · PART III · THE CONTAINERS

Dictionaries & Hash Tables

The single most important data structure in Python, used not only by you but by the language itself for every namespace, every object's attributes, every keyword-argument call. Average constant-time lookup, insertion-order memory since 3.7, and a one-page diagram of how that magic actually works.

> *Dictionaries are the cornerstone of Python. Almost every program uses them, and the language itself is built on top of them.*
>
> — the Python tutorial, on the dictionary

If you read no other chapter of this book carefully, read this one. The dictionary is the most consequential data structure in Python by some margin: not only do you use it for the obvious purposes (mapping names to numbers, looking up records by id, counting word frequencies, holding JSON-shaped data), but the language itself is built on dictionaries from the inside. Every module's global namespace is a dictionary; every class's body is a dictionary; every instance's attributes (in the common case) are a dictionary; every keyword-argument call assembles a dictionary. The whole machinery rests on a single algorithmic idea, the hash table, that gives you average constant-time lookup, insertion, and deletion no matter how large the dictionary grows. Understanding that idea will not just make your dict-using code faster; it will explain a dozen behaviours you previously had to memorise.

This chapter covers the idea (how a hash table works), the consequences (why a list cannot be a key, why insertion order is preserved, what "average O(1)" really means), and the daily practice (creating, reading, iterating, the four or five patterns that constitute most real dict code). Eight sections, one central diagram, and the usual set of worked examples and exercises.

10.1 The Most Important Data Structure in Python

A **dictionary** maps *keys* to *values*. You ask for a key and it gives you back the value associated with it. The mechanism is what makes the lookup remarkable: where finding an item in a list of a million takes, on average, half a million comparisons, finding a value in a dictionary of a million keys takes, on average, one. The dictionary does not search; it computes where the key would be, and looks there.

The interface is small. You write a literal in braces, you look up by indexing, you assign by indexing, you delete with `del`, you iterate over keys with a plain `for` loop, and you ask membership with `in`. That is most of what most programs need.

```python
person = {
    "name": "Ada",
    "age":  28,
    "city": "London",
}

person["name"]                # 'Ada'
person["city"] = "Paris"       # update an existing key
person["role"] = "engineer"    # add a new key
del person["age"]             # remove a key

"name" in person              # True, note: 'in' checks KEYS, not values
len(person)                  # 3
```

But the dict's reach beyond that surface is what makes it the most important structure in the language. When you write `x = 5`, Python adds the name `x` to a dictionary that represents the current scope. When you call `obj.method()`, Python looks up `method` in a dictionary attached to `obj`'s type. When you call `f(name="Ada")`, the keyword arguments arrive as a dictionary. You can see all of this, in fact, by asking for it: `globals()` returns the module's namespace dict, `obj.__dict__` returns an instance's attribute dict, `vars(C)` returns a class body. The dictionary is not merely a data type Python provides; it is the substrate on which most of the language runs.

DEF **Dictionary**

A mutable mapping from *hashable* keys to arbitrary values, written `{k1: v1, k2: v2}`. Type is `dict`. Implemented in CPython as a hash table, giving average constant-time

lookup, insertion, and deletion. Since Python 3.7 the language guarantees that iteration follows insertion order. Used by the language itself for every namespace, every object's attributes (typically), and every set of keyword arguments.

→ **FURTHER READING: THE DICT IN PYTHON**

- Dictionaries *the official tutorial section*
- Mapping Types: dict *the complete API reference*

10.2 How a Hash Table Works

Knowing the inside of a dict will not change how you write a dictionary literal, but it will explain almost every fact about how dicts behave. The data structure underneath is called a *hash table*, invented in 1953 by Hans Peter Luhn at IBM, and it is one of the most consequential algorithmic ideas of the twentieth century.

The core trick is this: instead of searching for a key, **compute** where it would be. Given a key, the dictionary calls a function called `hash()` that turns the key into an integer. It then reduces that integer to the range of available slots (modulo the table size) and looks in that slot. If the key it finds there matches, the lookup is done in one step, no matter how many keys the table holds. The whole reason a dictionary is fast is that the position is computed, not searched.

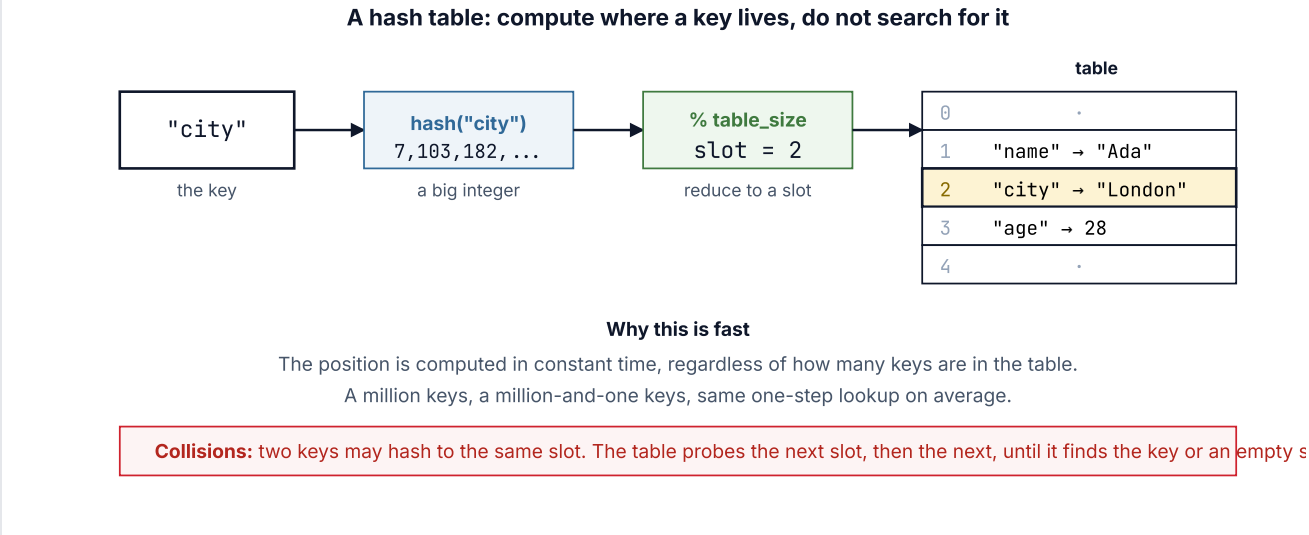


Figure 10.1. The hash-table lookup. CPython uses *open addressing*: when the computed slot is occupied by a different key, the table probes other slots (in a pseudo-random sequence) until it finds the key or a free space. The load factor is kept low (the table grows when about two-thirds full) so collisions stay infrequent.

Two facts about this design force themselves on every dict user. First, **the key must be hashable**, because the whole scheme depends on calling `hash(key)`. Lists, dicts, and sets are mutable, their hash is forbidden to exist, and they cannot be keys. Numbers, strings, tuples-of-hashable, and frozensets are immutable, their hash is stable, and they can. We laid this rule down in Chapter 3.7 and used it again in Chapter 9.5; here is where it finally pays off.

Second, **the average constant-time guarantee is "average"**. On well-behaved keys (the usual case), each lookup probes one or two slots. On pathological data where many keys collide on the same slot, lookups degrade toward linear time. CPython mitigates this in two ways: by growing the table when it gets too full (so collisions stay rare) and by adding randomisation to string hashing (so an attacker cannot craft inputs that collide deliberately). Section 10.8 returns to this.

→ **FURTHER READING: HASH TABLES**

- Hash table *the algorithm, with both chaining and open-addressing variants*
- Raymond Hettinger: "The Mighty Dictionary" *a PyCon talk on the dict's internals*
- CPython dictobject.c *the source, surprisingly readable*

10.3 The Insertion-Order Guarantee

From its creation in 1991 until 2017, Python dictionaries had no defined iteration order; you got the keys in whatever order the hash function happened to put them, and any sane code treated dict ordering as random. This was unusual among modern languages and was the source of subtle bugs. In Python 3.6, the CPython team (notably Raymond Hettinger) rebuilt the dict implementation as a *compact hash table* that happened to preserve the order in which keys were inserted. In Python 3.7 the language specification made this behaviour mandatory across all implementations. Iteration order is now **insertion order**, and you can rely on it.

```python
d = {}
d["first"] = 1
d["second"] = 2
d["third"] = 3

list(d)            # ['first', 'second', 'third'], every time, since 3.7
```

This change had small but real practical consequences. JSON files round-trip with their key order preserved. Keyword arguments arrive in the order they were written. Code that historically used `collections.OrderedDict` for this purpose can now use a plain `dict`; `OrderedDict` still exists but is rarely needed, kept for its few features that go beyond the regular dict (such as `move_to_end` and equality that considers order).

> **A SUBTLETY** "Insertion order" means the order in which keys were *first* added. Updating an existing key with a new value does not move it. To force a key to the end of the iteration order, delete and re-insert it (or use `OrderedDict.move_to_end`).

> → **FURTHER READING: DICT ORDER**
>
> - PEP 468: Preserving Keyword Argument Order *the related kwargs ordering*
> - Guido on dict insertion order *the language-spec announcement*

10.4 Creating Dictionaries

Four common ways to build a dictionary, used roughly in this order of frequency.

```python
# 1. Literal: by far the most common
person = {"name": "Ada", "age": 28}

# 2. Constructor with keyword arguments (keys must be valid identifiers)
person = dict(name="Ada", age=28)

# 3. From an iterable of pairs
person = dict([("name", "Ada"), ("age", 28)])
person = dict(zip(["name", "age"], ["Ada", 28]))

# 4. Dict comprehension
```

```
squares = {n: n * n for n in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Empty: braces only, {} is an empty DICT, not an empty set
empty_dict = {}
empty_set  = set()
```

Two merging operations deserve attention because they have replaced more verbose older code. **The merge operator** `|` (Python 3.9+) builds a new dict combining two:

PYTHON

```
defaults = {"theme": "light", "lang": "en"}
user     = {"lang": "fr", "timezone": "UTC"}

merged = defaults | user
# {'theme': 'light', 'lang': 'fr', 'timezone': 'UTC'}
# right side wins on key conflicts

# The in-place version updates the left-hand dict:
defaults |= user      # equivalent to defaults.update(user)
```

And **unpacking inside a literal** (PEP 448) does the same thing slightly differently:

PYTHON

```
merged = {**defaults, **user}    # same result; works on older Python too
```

→ **FURTHER READING: CREATION**

- PEP 584: Add Union Operators to dict *the | operator's design*
- PEP 448: Additional Unpacking Generalizations *where {**a, **b} came from*

10.5 Reading and Writing: `get`, `setdefault`, and the Avoidance of `KeyError`

The bracket form `d[k]` reads or writes a value. Writing always succeeds: if the key is present, the value is replaced; if not, it is added. Reading is where the subtlety lives. If the key is missing, `d[k]` raises `KeyError`, which is the right behaviour when "missing"

means "something is wrong" and the wrong behaviour when "missing" just means "use a default".

```python
d = {"name": "Ada"}

d["name"]              # 'Ada'
d["missing"]           # KeyError: 'missing'

# .get returns None (or your default) when the key is absent:
d.get("missing")               # None, no exception
d.get("missing", "unknown")    # 'unknown'

# .setdefault both reads and writes: if the key is missing,
# inserts the default and returns it; if present, returns the existing value.
d.setdefault("age", 28)         # 28 (inserted)
d.setdefault("age", 99)         # 28 (already present; 99 ignored)
```

Three other operations worth mastering: `pop`, `update`, and `in`.

```python
d.pop("age")             # removes 'age' and returns its value, or KeyError if ab
d.pop("missing", None)   # with a default: never raises

d.update({"city": "London", "name": "Ada"})
# merges in place; the argument's keys overwrite existing keys

"name" in d            # True, 'in' checks KEYS, never values
"Ada" in d.values()    # True, explicit values-check is O(n)
```

THE `IN` TRAP The expression `x in d` tests whether `x` is one of the dict's **keys**, never one of its values. This is consistent with how `for k in d` iterates over keys, but it surprises some beginners who expect `"Ada" in person` to find the value. To check values, use `x in d.values()` (linear), and to check pairs, use `(k, v) in d.items()`.

→ **FURTHER READING: METHODS**

○ The dict reference *every method, definitively*

10.6 The Three Views: `keys()`, `values()`, `items()`

Iteration over a dict gives you the keys, in insertion order. Two other forms cover the cases when you want values instead, or both at once.

```python
d = {"a": 1, "b": 2, "c": 3}

for k in d:              # default: iterate over keys
    print(k)

for v in d.values():     # iterate over values
    print(v)

for k, v in d.items(): # iterate over both (the workhorse loop)
    print(k, v)
```

The three methods return *views*, not lists. A view is a lightweight, lazy object that reflects the dict as it currently stands: if you mutate the dict, the view updates. Views also support some set-like operations (you can intersect two dicts' keys with `&`, union them with `|`), which is occasionally useful.

```python
v = d.values()
d["d"] = 4
list(v)              # [1, 2, 3, 4], the view saw the new value

# Set operations on key views:
a = {"x": 1, "y": 2}
b = {"y": 9, "z": 3}
a.keys() & b.keys()  # {'y'}, keys in both
a.keys() - b.keys()  # {'x'}, keys only in a
```

DO NOT MODIFY A DICT WHILE ITERATING ITS VIEW

Adding or removing keys during a `for k in d:` loop raises `RuntimeError: dictionary changed size during iteration`. The fix is to iterate over a snapshot: `for k in list(d):`. This builds a list of keys first, then iterates that list, leaving you free to mutate `d`.

10.7 Common Patterns

A small set of dictionary patterns covers most real code. Memorising them is one of the best returns on time in Python.

EXAMPLE 10.1 Counting occurrences

```python
text = "the cat sat on the mat the cat slept"
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
# {'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1, 'slept': 1}

# Or, with collections.Counter (Chapter 12 will revisit this):
from collections import Counter
Counter(text.split()).most_common(3)
# [('the', 3), ('cat', 2), ('sat', 1)]
```

The `get(word, 0) + 1` idiom is canonical: read the current count (zero if absent), add one, write it back. `Counter` wraps the whole pattern into one call.

EXAMPLE 10.2 Grouping items by a key

```python
people = [
    {"name": "Ada",     "team": "A"},
    {"name": "Bo",      "team": "B"},
    {"name": "Cy",      "team": "A"},
]

# Manual grouping with setdefault:
groups = {}
for p in people:
    groups.setdefault(p["team"], []).append(p["name"])
# {'A': ['Ada', 'Cy'], 'B': ['Bo']}

# With defaultdict (cleaner):
from collections import defaultdict
groups = defaultdict(list)
for p in people:
    groups[p["team"]].append(p["name"])
```

`setdefault(key, [])` inserts an empty list when the key is missing and returns it, so the `.append` always has somewhere to go. A `defaultdict(list)` does this implicitly: any missing-key access creates an empty list automatically. Both patterns produce the same result.

EXAMPLE 10.3 Inverting a dict

```python
d = {"one": 1, "two": 2, "three": 3}
inverted = {v: k for k, v in d.items()}
# {1: 'one', 2: 'two', 3: 'three'}
```

A dict comprehension reads as cleanly here as it does for lists. Be aware: if the original dict has two keys with the same value, the inverted dict will collapse them into one, keeping only the last.

EXAMPLE 10.4 Pairs to a dict

```python
pairs = [("a", 1), ("b", 2), ("c", 3)]
d = dict(pairs)          # {'a': 1, 'b': 2, 'c': 3}

# The inverse, dict to pairs, is just .items():
list(d.items())        # [('a', 1), ('b', 2), ('c', 3)]
```

The `dict()` constructor accepts any iterable of two-element pairs. Combined with `zip`, this is how you build a dict from two parallel lists in a single call.

10.8 Performance, Memory, and Hash Security

Three practical facts about dictionaries that occasionally matter beyond the average case.

Memory

A dictionary uses more memory than a list with the same items, because it stores hashes, keys, and values, and the hash table is intentionally not full (CPython keeps the load factor below about two-thirds to limit collisions). For most programs this is invisible; for code that creates a million tiny dicts where a list or named tuple would suffice, the difference is real.

Hash randomisation

Since Python 3.3, the hash of a string is randomised at interpreter start (each `python` process picks a fresh secret). The change was made in response to a real attack: a malicious user could craft a request whose keys all hashed to the same slot, turning every dictionary insertion into an O(n) operation and making the server crawl to a stop. The randomisation breaks the attack by making the hashes unpredictable. The consequence for you: **do not rely on the exact value of `hash(some_string)` across runs**, and do not hard-code dict iteration order across processes if it depends on hash values.

```python
$ python -c "print(hash('hello'))"
# 6058471826385858049
$ python -c "print(hash('hello'))"
# -3284854253523829641, a different value each invocation
```

Worst-case collisions

Even without an attacker, every hash table has a worst case in which all keys land in the same slot. CPython's randomisation, growth strategy, and open-addressing scheme all keep this rare; in practice, dict operations are constant time on the kind of data programs actually use. For pathological inputs (say, a million keys deliberately constructed to collide), performance can degrade.

→ **FURTHER READING: PERFORMANCE & SECURITY**

- PEP 456: Secure and interchangeable hash algorithm *hash randomisation, in full*
- Python security advisories *where hash collision attacks were tracked*

§ § §

Exercises

EXERCISE 10.1 Build a dict three ways

Build the dictionary `{"name": "Ada", "age": 28, "city": "London"}` three different ways: with a literal, with the `dict()` constructor, and with `zip` from two parallel lists. Confirm all three are equal.

EXERCISE 10.2 Word frequencies

Given the sentence `"the quick brown fox jumps over the lazy dog the fox sleeps"`, count word frequencies into a dict using `.get(word, 0) + 1`. Then rewrite with `collections.Counter` and compare.

EXERCISE 10.3 Group by first letter

Given `names = ["Ada", "Alan", "Bo", "Caroline", "Carl"]`, build a dict that maps each first letter to a list of names with that letter, using `setdefault`. Then do it again with `defaultdict(list)`.

EXERCISE 10.4 The `in` trap

Given `d = {"name": "Ada"}`, predict each: (a) `"name" in d`; (b) `"Ada" in d`; (c) `"Ada" in d.values()`; (d) `("name", "Ada") in d.items()`. Run them. Explain in one sentence why `"Ada" in d` is `False`.

EXERCISE 10.5 Merge with the right precedence

Given `defaults = {"theme": "light", "lang": "en"}` and `user = {"lang": "fr"}`, build a merged dict where the user's values override the defaults. Use the `|` operator. Then do the same with `{**a, **b}`. Why does the right operand win?

EXERCISE 10.6 Invert a dict

Invert `{"one": 1, "two": 2, "three": 3}` with a dict comprehension. Then try inverting `{"a": 1, "b": 1, "c": 2}`, where two keys share a value. What happens? Why?

EXERCISE 10.7 A safe iteration

You want to remove every key whose value is `None` from a dict. Show that doing this with a direct `for k in d:` loop and `del d[k]` raises `RuntimeError`. Then write two correct versions: one that iterates over a snapshot, and one that builds a new dict with a comprehension.

EXERCISE 10.8 A research question

The Python team made dict insertion order a language guarantee in 3.7. What problem did this solve, and what older feature became largely obsolete? Read about PEP 468 or Hettinger's compact-dict design and write one paragraph summarising the change.

Solutions to Chapter 10 Exercises

Solution 10.1: Build a dict three ways

```python
a = {"name": "Ada", "age": 28, "city": "London"}
b = dict(name="Ada", age=28, city="London")
c = dict(zip(["name", "age", "city"], ["Ada", 28, "London"]))
a == b == c                # True
```

The literal is the cleanest when you know the keys statically. The constructor with keyword arguments is restricted to keys that are valid Python identifiers. The `dict(zip(...))` form is the canonical way to build a dict from two parallel lists.

Solution 10.2: Word frequencies

```python
text = "the quick brown fox jumps over the lazy dog the fox sleeps"
counts = {}
for w in text.split():
    counts[w] = counts.get(w, 0) + 1

from collections import Counter
counts2 = Counter(text.split())
counts == counts2          # True, Counter is a dict subclass with extra methods
counts2.most_common(3)        # [('the', 3), ('fox', 2), ('quick', 1)]
```

`Counter` is a subclass of `dict` tuned for this exact task. It exposes `most_common` and a small set of arithmetic operations on top of the regular dict API.

Solution 10.3: Group by first letter

```python
names = ["Ada", "Alan", "Bo", "Caroline", "Carl"]

groups = {}
for n in names:
    groups.setdefault(n[0], []).append(n)
# {'A': ['Ada', 'Alan'], 'B': ['Bo'], 'C': ['Caroline', 'Carl']}

from collections import defaultdict
groups = defaultdict(list)
for n in names:
    groups[n[0]].append(n)
```

Both work; the `defaultdict` version is cleaner once you have read it once. The trade-off is that a `defaultdict` silently creates entries on read (any `groups["X"]` creates an empty list), which is what you want here but occasionally hides bugs elsewhere.

Solution 10.4: The `in` trap

(a) `True`, `"name"` is a key. (b) `False`, `in` on a dict checks keys, not values, and `"Ada"` is a value. (c) `True`, explicit values check. (d) `True`, explicit items check, using the tuple form. The `in` operator on a dict is shorthand for `in d.keys()`; this is consistent with how the default iteration yields keys, and is faster than scanning values (which is unavoidably O(n)).

Solution 10.5: Merge with the right precedence

```python
defaults = {"theme": "light", "lang": "en"}
user     = {"lang": "fr"}

defaults | user            # {'theme': 'light', 'lang': 'fr'}
{**defaults, **user}       # same result
```

The right operand wins on conflicts because that is what you almost always want when merging "defaults" with "overrides": the later argument represents the user's preference. `|` codifies this rule by defining the operator to favour the right side; `{**a, **b}` works the same because later keys in a dict literal replace earlier ones.

Solution 10.6: Invert a dict

```python
{v: k for k, v in {"one": 1, "two": 2, "three": 3}.items()}
# {1: 'one', 2: 'two', 3: 'three'}

{v: k for k, v in {"a": 1, "b": 1, "c": 2}.items()}
# {1: 'b', 2: 'c'}, 'a' was overwritten by 'b'
```

A dict has unique keys, so when two original entries share a value, only one survives the inversion. The later one wins (the comprehension processes keys in insertion order, and later writes overwrite earlier ones). If you need to keep both, the inverted value should be a list, built with `setdefault` or `defaultdict`.

Solution 10.7: A safe iteration

```python
d = {"a": 1, "b": None, "c": 3}

# Wrong:
for k in d:
    if d[k] is None:
        del d[k]
# RuntimeError: dictionary changed size during iteration

# Right: iterate over a snapshot of the keys
for k in list(d):
    if d[k] is None:
        del d[k]

# Right: build a new dict
d = {k: v for k, v in d.items() if v is not None}
```

`list(d)` materialises the keys into a list before the loop starts; mutating the dict no longer affects what the loop iterates over. The comprehension version is usually cleaner because it avoids the explicit mutation entirely.

Solution 10.8: A research question

Before 3.7, dictionary iteration order was unspecified, and code that relied on it was buggy. The change made order predictable across implementations: JSON files now round-trip with their keys in original order, keyword arguments arrive in writing order, and code

that previously needed `collections.OrderedDict` can use a plain `dict`. PEP 468 was the keyword-arguments precursor (3.6); the broader compact-dict design by Raymond Hettinger reshaped CPython's dict implementation as a two-array structure (a small index table plus a dense entries array), which both saves memory and preserves insertion order as a side effect. `OrderedDict` still exists but is now used only for its few methods that go beyond regular dicts, primarily `move_to_end` and order-sensitive equality.